

Computación Gráfica y Ambientes Virtuales

Resumen de estudio para el parcial · 2026

Asignatura 0494 · Facultad de Ingeniería
Centro Regional Universitario Córdoba — IUA
Universidad de la Defensa Nacional

Unidades I, II, V, VI y VII
Multiple choice · Preguntas de desarrollo · Simulacro

Indice

Computación Gráfica y Ambientes Virtuales	5
Resumen de estudio para el parcial — 2026	5
Cómo usar este documento	6
Qué revela el parcial anterior	6
Cómo está organizado	6
Priorización basada en evidencia	8
Prioridad MUY ALTA	8
Prioridad ALTA	8
Prioridad MEDIA	9
Prioridad BAJA	9
UNIDAD I — Sistemas gráficos y dispositivos de visualización	10
1.1 Qué es la Computación Gráfica	10
1.2 Evolución de la Computación Gráfica	11
1.3 El tubo de rayos catódicos (CRT)	12
1.4 Display caligráfico o vectorial	13
1.5 Display digital (raster o de trama)	14
1.6 Comparación: display vectorial vs display raster	15
1.7 Arquitectura del sistema raster y buffer de refresco	16
1.8 Tecnologías modernas de display	19
Multiple Choice de práctica — Unidad I	21
Preguntas de desarrollo posibles — Unidad I	23
EL PIPELINE DE RENDERING	25
Definición	25
Explicación conceptual	25
Esquema general	25
Componentes o etapas	26
El mismo pipeline, en su versión práctica (OpenGL)	26
Relación con otros conceptos	27
Errores o confusiones frecuentes	27
Cómo respondería esto en un parcial	28
UNIDAD II — Graficación de primitivas 2D	29
Parte 1 — Digitalización de primitivas lineales	29
Multiple Choice de práctica — Unidad II, parte 1	38

Preguntas de desarrollo posibles — Unidad II, parte 1	40
Parte 2 — Primitivas de área, relleno y antialiasing	41
Multiple Choice de práctica — Unidad II, parte 2	51
Preguntas de desarrollo posibles — Unidad II, parte 2	54
UNIDAD V — Modelado de objetos	55
5.1 Qué es un modelo	55
5.2 Las tres estrategias para modelar	55
5.3 Representaciones geométricas	56
5.4 Los cuatro métodos de representación	56
5.5 Comparación de los cuatro métodos	61
Multiple Choice de práctica — Unidad V	62
Preguntas de desarrollo posibles — Unidad V	64
UNIDAD VI — Transformaciones geométricas	66
6.1 Definición y para qué sirve	66
6.2 Preliminares matemáticos	66
6.3 Transformaciones básicas 2D	67
6.4 Coordenadas homogéneas	68
6.5 Cómo reconocer una matriz de transformación	69
6.6 Composición de transformaciones	70
6.7 Transformaciones 3D	72
Multiple Choice de práctica — Unidad VI	75
Preguntas de desarrollo posibles — Unidad VI	77
UNIDAD VII — Visualización en 3D	79
Parte 1 — Proyecciones y cámara sintética	79
Parte 2 — Transformación de normalización, recorte y proyección	86
Multiple Choice de práctica — Unidad VII	93
Preguntas de desarrollo posibles — Unidad VII	96
BLOQUE PRÁCTICO — Conceptos que iluminan la teoría	97
Los prácticos mencionados el año anterior	97
Qué conceptos teóricos ejercitan estos prácticos	98
Conceptos prácticos con valor teórico	98
PARCIAL DEL AÑO ANTERIOR — RESPUESTAS MODELO	100
Pregunta 1	100
Pregunta 2	101
Pregunta 3	103

Pregunta 4	105
Pregunta 5	107
Pregunta 6	109
Pregunta 7	110
OJO CON ESTO EN EL MULTIPLE CHOICE	112
Sobre displays y frame buffer	112
Sobre algoritmos de digitalización	112
Sobre relleno y antialiasing	113
Sobre modelado	113
Sobre transformaciones	113
Sobre visualización 3D	114
Sobre el pipeline	115
REPASO FINAL — LO QUE NO PUEDO IR SIN SABER	116
Definiciones imprescindibles	116
Comparaciones imprescindibles	117
Procesos que tengo que poder explicar	117
Diagramas que tengo que poder reconstruir	117
Matrices que tengo que reconocer	118
Fórmulas importantes	119
Algoritmos importantes	120
Errores de multiple choice (los 12 más peligrosos)	120
15 preguntas que definitivamente debería poder responder	120
SIMULACRO DE PARCIAL	122
Parte A — Multiple Choice	122
Parte B — Desarrollo	124
RESPUESTAS DEL SIMULACRO	125
Parte A — Respuestas y explicación	125
Parte B — Respuestas modelo y puntos esenciales	126
CHECKLIST DE VERIFICACIÓN	133
Salvedades que corresponde declarar	133

Computación Gráfica y Ambientes Virtuales

Resumen de estudio para el parcial — 2026

Asignatura 0494 · Facultad de Ingeniería · CRUC-IUA · Universidad de la Defensa Nacional

Material elaborado a partir de los 28 PDFs de la cátedra: unidades teóricas I, II, V, VI y VII; presentaciones teórico-prácticas 01 a 06; guías prácticas 01 a 05 y guía áulica; apuntes de clase 1, 2, 3, 6, 7, 8 y 9; y el programa oficial de la asignatura.

Convenciones de este documento

- Las fuentes se citan como Archivo p.N , donde N es la página del PDF.
- **[NOTA DE CLASE]** marca información que está en tus apuntes y **no** en las filminas oficiales. Tiene prioridad especial: refleja qué enfatizó el profesor.
- **Explicación complementaria** marca un agregado que no está en el material, incluido sólo para facilitar la comprensión.
- **⚠ Ojo** marca una errata, ambigüedad o contradicción detectada entre documentos. En esos casos se muestran las dos versiones; no se elige una.

Cómo usar este documento

El profesor indicó que el parcial de este año tendrá **preguntas de multiple choice** y **preguntas a desarrollar**. Como referencia del estilo se dispone de las siete preguntas de un parcial del año anterior.

Qué revela el parcial anterior

Las siete preguntas del año pasado tienen una característica decisiva: **ninguna es numérica**. Todas comienzan con *Realice una comparación...*, *Explique...*, *Describe...*, *Indique...*. No se pidió trazar Bresenham, no se pidió calcular una matriz, no se pidió digitalizar una circunferencia.

El verbo de cada consigna indica el tipo de respuesta esperada:

Verbo	Qué espera el profesor	Ejemplo del parcial anterior
Realice una comparación	Tabla o contraste sistemático punto por punto, y la diferencia conceptual de fondo	Pregunta 1 (vectorial vs raster)
Explique qué es... indicando su función	Definición + ubicación en la arquitectura + para qué sirve	Pregunta 2 (buffer de refresco)
Describe... y realice un esquema	Texto más diagrama : el esquema se pide explícitamente	Pregunta 3 (pipeline)
Explique cómo funcionan	Mecanismo paso a paso, no sólo definición	Pregunta 4 (algoritmos de relleno)
Indique... y explique qué efecto producirá	Reconocer + interpretar geoméricamente	Pregunta 5 (matrices y composición)
Indique cuáles son las especificaciones mínimas	Enumeración completa y justificada	Pregunta 6 (cámara sintética)
Describe el proceso y las transformaciones	Secuencia ordenada de etapas, cada una con su función	Pregunta 7 (visualización 3D)

La consecuencia práctica: el esfuerzo principal debe ir a **poder explicar mecanismos y compararlos**, no a ejercitar cálculo. Los algoritmos hay que saber **explicar cómo funcionan y por qué**, no necesariamente ejecutarlos con números.

Advertencia necesaria. Que el año pasado no haya habido ejercicios numéricos **no garantiza** que este año tampoco los haya. El programa oficial incluye "Guías de problemas" entre los instrumentos de evaluación, y tus apuntes de clase registran un *"hay que saber explicar cómo planteamos"* sobre el algoritmo de punto medio para elipses. Por eso este documento cubre **todo el temario**, con el planteo de los algoritmos desarrollado en profundidad, aunque el énfasis esté puesto en la explicación conceptual.

Cómo está organizado

Cada unidad tiene: desarrollo de los temas → **Cómo respondería esto en un parcial** (respuestas modelo) → **Multiple Choice de práctica** con explicación → **Preguntas de desarrollo posibles** con guía de respuestas.

Al final: las siete preguntas del parcial anterior resueltas, una sección de confusiones típicas para multiple choice, el repaso final y un simulacro completo.

Priorización basada en evidencia

La clasificación surge de cruzar seis criterios: aparición en el parcial anterior · repetición entre PDFs · énfasis explícito del profesor · presencia en tus notas de clase · importancia conceptual · relación con otros temas.

Prioridad MUY ALTA

Tema	Evidencia
Display vectorial vs raster	Pregunta 1 del parcial anterior · aparece en Unidad I p.25, Unidad II-1 p.3, Clase 2 p.2
Buffer de refresco (frame buffer)	Pregunta 2 del parcial anterior · Unidad I p.20-24 · Clase 2 p.1-2
Pipeline de rendering	Pregunta 3 del parcial anterior (pide esquema) · aparece en 8 documentos distintos
Algoritmos de relleno de áreas	Pregunta 4 del parcial anterior · Unidad II-2 p.18-22
Matrices de transformación y composición	Pregunta 5 del parcial anterior · Unidad VI completa · Clase 9 · se aplica en 4 prácticos
Cámara sintética	Pregunta 6 del parcial anterior · Unidad VII-1 p.18-32
Proceso completo 3D → 2D (normalización)	Pregunta 7 del parcial anterior · Unidad VII-2 completa · Práctico 06
Coordenadas homogéneas	Base de las preguntas 5 y 7 · Unidad VI p.11-14
Algoritmo de punto medio para elipses	[NOTA DE CLASE] único "hay que saber explicar" de todos tus apuntes (Clase 3 p.9)

Prioridad ALTA

Tema	Evidencia
Algoritmo de Bresenham: planteo y fundamento	8 páginas de filminas + 6 de apuntes; es el modelo de los otros dos algoritmos
Punto medio para circunferencias	Mismo patrón, con la diferencia de los incrementos variables
Scanline con GET y AET	13 páginas de filminas con el algoritmo numerado paso a paso
Proyecciones planares: clasificación completa	Unidad VII-1 p.3-17; el árbol de clasificación es una figura entera
Aliasing, Nyquist y antialiasing	Unidad II-2 p.23-32 + Clase 7
Los cuatro métodos de representación 3D	Unidad V completa; formato naturalmente comparativo
Transformaciones 3D y rotación sobre eje arbitrario	Unidad VI p.19-33
Recorte (clipping) y mapeo a pantalla	Unidad VII-2 p.24-27; parte de la pregunta 7
Determinación interior/exterior (par-impar, winding)	Unidad II-2 p.4; antecede a la pregunta 4

Prioridad MEDIA

Tecnologías de display (LCD, TFT, LED, OLED) · Modelado por barrido y superficies de revolución · Muestreo y reconstrucción · Concepto de coherencia · Casos especiales del scanline · Conceptos de OpenGL que iluminan la teoría (VBO/VAO, shaders, la equivalencia teórico↔código) · Conteo de vértices según atributos.

Prioridad BAJA

Historia detallada por décadas · Detalle constructivo del CRT · Distancia focal (la propia filmina la marca como *opcional*) · Durero 1525 · Planos acotados y proyección gnomónica (aparecen sólo en el árbol, sin desarrollo) · Historia de OpenGL · Comparación OpenGL vs Vulkan · Régimen del proyecto integrador.

Cobertura del programa. El programa oficial tiene **nueve** unidades. Entre los PDFs entregados **no están** las Unidades III (Color), IV (Imágenes: muestreo, Fourier, convolución, filtrado), VIII (Eliminación de superficies ocultas) ni IX (Iluminación, sombreado y textura). Este documento cubre las cinco unidades con material: **I, II, V, VI y VII**. Si el parcial incluye las otras cuatro, este resumen no alcanza: conviene confirmar el alcance antes de estudiar.

UNIDAD I — Sistemas gráficos y dispositivos de visualización

Prioridad MUY ALTA. Aportó dos de las siete preguntas del parcial anterior.

1.1 Qué es la Computación Gráfica

Definición

"La computación gráfica es la ciencia (y el arte) de comunicar visualmente por medio de una pantalla y los dispositivos de interacción de una computadora." — *Unidad I p.3*

Según **Fetter**, es la *creación, manipulación y almacenamiento de modelos de objetos e imágenes*.

Explicación conceptual

El punto de partida es que las computadoras crecieron enormemente en capacidad de cálculo y realizan tareas cada vez más complejas; esa complejidad exige que el usuario acceda a **gran cantidad de información**. La solución es un **proceso de comunicación bidireccional**:

- **Computadora → usuario:** visual, auditiva y movimientos.
- **Usuario → computadora:** dispositivos de entrada (teclado, mouse, joystick, dispositivo touch, movimientos).

La materia estudia los **aspectos técnicos del proceso de comunicación asociado a la representación visual de la información**, buscando que el usuario **infiera información a través de la simple observación de la pantalla**.

Para comunicar visualmente se usa el concepto de **imagen**. Según la RAE: "*figura, representación, semejanza y apariencia de algo*".

Componentes: un campo multidisciplinario

Disciplina	Qué aporta
Física	Modelos de la luz y simulación
Matemática	Descripción de las formas
Percepción humana	Determina los recursos computacionales a utilizar
Ingeniería	Optimización de memoria y tiempos de procesamiento
Diseño gráfico y arte	Hacen más eficiente la interacción computadora ↔ usuario

Cómo se generan las imágenes

Hay **dos grandes categorías**:

- **Capturadas.** Se realiza un proceso que transforma la luz que reflejan los objetos de la realidad en información que se almacena en algún soporte para su visualización. Ejemplos: pintura artística, fotografía analógica, fotografía digital.

- **Sintetizadas.** La información de la realidad que se quiere representar se genera mediante algún **modelo físico** o usando algún dispositivo, y luego se almacena para su visualización. Las imágenes son "**creadas**".
← **Es lo que estudia la materia.**

Clasificación de las imágenes sintetizadas

Criterio	Tipo	Características
Resultado visual	Realista / foto realista	Se ve como la vida real; se usan modelos que representan la física
	Artístico / no foto realista	Se ve como lo que podría generar un artista; se modela el proceso de creación del artista , física levemente; generar algo que se vea bien
Tiempo de procesamiento	En tiempo real	10 – 60 cuadros por segundo. Interfaces, simulaciones visuales, juegos
	En diferido / producción	Segundos a horas por cuadro. Industria cinematográfica, arquitectura, arte, propaganda

Concepto de rendering

Rendering (plasmado): el proceso que permite obtener una representación estática 2D (una imagen) de un mundo abstracto 3D. — *Unidad II-1 p.2*

Para lograrlo hay que estudiar tres cosas: **cómo modelar las formas**, **cómo modelar la apariencia de las formas** y **cómo visualizar las formas**. Esas tres preguntas son, respectivamente, las Unidades V, IX y VII.

Errores o confusiones frecuentes

- Confundir **capturada** con **sintetizada**: la fotografía digital es capturada aunque el soporte sea digital. Lo que define la categoría es **de dónde proviene la información**, no el medio de almacenamiento.
- Creer que "realista" y "tiempo real" son el mismo eje. Son **dos criterios independientes**: un juego es realista y en tiempo real; un render de POV-Ray es realista y en diferido.

Lo que debería saber explicar en el parcial

La definición textual de computación gráfica; las dos formas de generar imágenes con un ejemplo de cada una; los dos ejes de clasificación de las sintetizadas **con sus rangos numéricos**; y qué es el rendering.

1.2 Evolución de la Computación Gráfica

Prioridad BAJA-MEDIA. Conviene saber los tres hitos fundacionales; la lista completa de algoritmos por década es contexto.

Los hitos que sí hay que saber

- **≈1950 — Los inicios.** La computación gráfica está presente desde el inicio de la computación. Interacción por **tarjetas perforadas**; dispositivos de salida: *strip tracers* (ej. electrocardiograma), *pen plotters* (precursores de las impresoras) y **CRT**. Imágenes sencillas usando conversor A/D; los CRT eran **lentos en actualizar** la imagen.

- **1960 — William Fetter.** Se le atribuye la creación del término "**Computación Gráfica**". Lo usa para describir el método de diseño que estaba usando en **Boeing**: con un plotter crea imágenes del diseño de una cabina de avión usando el modelo 3D de un cuerpo humano.
- **1963 — Ivan Sutherland.** Crea **Sketchpad** como trabajo de tesis doctoral en el MIT. Utiliza un **trazador caligráfico (vectorial) modificado** y un **lápiz óptico**. Objetivo: permitir que el usuario controle el contenido, la estructura y apariencia de los objetos mediante un rápido reconocimiento visual. **Crea el primer Sistema Gráfico Interactivo.**

Componentes de un Sistema Gráfico Interactivo básico: entrada (mouse, lápiz, tabla de digitalización, scanner) → **procesamiento** (y almacenamiento) → salida/muestra (pantalla, impresora, video grabador).

Evolución por décadas (contexto)

Período	Hardware / estándares	Algoritmos
1960-70	Display vectorial; imágenes de líneas (malla de alambre); primeros procesadores de pantalla dedicados	Visibilidad de líneas: Roberts (1963), Appel (1967). Visibilidad de superficies: Warnock (1968), Watkins (1968)
1970-80	Display raster ; líneas y polígonos rellenos; propuestas de estándares	Iluminación difusa y sombreado continuo: Gouraud (1971) ; z-buffer: Catmull (1974) ; iluminación especular: Phong (1975) ; <i>environment mapping</i> y texturas: Blinn (1976); anti-aliasing : Crow (1977)
1980-90	Hardware especial; estándares PHIGS y RenderMan; X Window System	Ray tracing: Whitted (1980); radiosidad: Goral/Torrance (1984), Cohen (1985); ecuación del renderizado: Kajiya (1986); ruido de Perlin (1985)
1990-2000	APIs OpenGL y DirectX ; primera película completa por computadora (Toy Story); texture mapping, blending, stencil buffers	Renderizado no foto realista; volume rendering
2000-2010	Pipeline gráfico programable : shaders (vertex, geometry, tessellation, fragment); hardware con paralelización	Iluminación por píxel en vez de por vértice; bump/normal mapping; efectos procedurales
2010-presente	GPGPU (CUDA, OpenCL); hardware dedicado de ray-tracing; APIs de bajo overhead (Vulkan, Direct3D 12)	PBR en tiempo real; iluminación global en tiempo real; plasmado híbrido raster + ray-tracing

Lo que debería saber explicar en el parcial

Los tres nombres con año y aporte (Fetter 1960 acuña el término; Sutherland 1963 crea Sketchpad y el primer sistema gráfico interactivo; Bresenham 1965 su algoritmo); la transición **vectorial** → **raster** en los años 70; y que el pipeline programable aparece en los 2000.

1.3 El tubo de rayos catódicos (CRT)

Cómo funciona

El CRT es la base de los dos displays precursores. Su funcionamiento:

- **Cuerpo principal:** una botella de vacío.
- **Cañón de electrones y película de fósforo.**
- Los electrones emitidos son **atraídos hacia un disco (ánodo)** conformando un **rayo**.
- **Elementos deflectores** del rayo, vertical y horizontal, lo dirigen.
- El rayo de electrones **impacta en la capa de fósforo**, haciendo que ésta **emita luz**.

Primera aplicación: el **trazador aleatorio** (osciloscopio), que se utilizaba para representar señales eléctricas analógicas.

Relación con otros conceptos

El CRT por sí solo sólo sirve para señales analógicas. Para representar **imágenes arbitrarias** hubo que modificarlo, y de ahí salen las **dos estrategias** que compara la pregunta 1 del parcial anterior: el display caligráfico (Sutherland lo modifica en 1963) y el display raster.

1.4 Display caligráfico o vectorial

Prioridad MUY ALTA. Mitad de la pregunta 1 del parcial anterior.

Definición y funcionamiento

Sutherland tuvo que **modificar el hardware de visualización (CRT)** para poder generar una representación de imágenes arbitrarias. Las modificaciones fueron:

- **Modificar el control de los elementos deflectores** (para dirigir el rayo a cualquier posición).
- **Modificar el control de emisión del rayo** (para poder encenderlo y apagarlo).

Con eso, **se pueden dibujar líneas en cualquier orden**, con la misma metodología que usa un *pen-plotter*.

Arquitectura de hardware

Sutherland definió una **lista de comandos almacenada en memoria** para dibujar las líneas, es decir, para controlar el rayo de electrones.

El problema es la **persistencia**: el fósforo se apaga. Para que las líneas permanezcan en pantalla es necesario **volver a ejecutar la lista de comandos**.

- **Solución:** ejecutar en un **loop** la lista de comandos.
- **Problema:** la velocidad de actualización **depende de la cantidad de comandos**, es decir, de la **complejidad de la imagen** representada.
- **Límite práctico:** la velocidad de actualización debe ser de **30 a 60 veces por segundo** para evitar el problema de **parpadeo (flicker)**.

Características

- Se **dibujan segmentos de línea** para representar las formas.
- La velocidad de pintado **depende de la complejidad** de la imagen.
- Es **muy costoso rellenar formas**: hay que generar nuevos segmentos de línea.

Interpretación conceptual

La clave del vectorial es que **la imagen es una lista de instrucciones de dibujo**, no una grilla de valores. Cada línea es un comando. Por eso más líneas = más tiempo = riesgo de parpadeo, y por eso rellenar un área es carísimo: un relleno son cientos de líneas paralelas, cada una con su comando.

1.5 Display digital (raster o de trama)

Prioridad MUY ALTA. La otra mitad de la pregunta 1.

El concepto de barrido (scanning)

- Una imagen se puede interpretar como una **secuencia de intensidades de luz**.
- Es posible **muestrearla** para poder reconstruirla después.
- **Barrer (scanning)** una escena generada o una imagen real consiste en **muestrear (digitalizar) una secuencia de funciones de intensidad continuas**.
- El proceso genera **una función por "línea"** de la imagen.

Proceso de muestreo y reconstrucción

1. Se **barre** una imagen para obtener una secuencia de funciones de intensidades (una por línea).
2. Se **muestra a intervalos regulares** — en el proceso **se pierde información de intensidad y espacial**.
3. Se **reconstruye la señal**, por ejemplo por medio de una onda de tensión analógica continua.

¿Cuál es la frecuencia de muestreo necesaria para recuperar la imagen? Cuantas más muestras se disponga de la señal, más información se posee de ella; pero hay que optimizar el almacenamiento y el tiempo de procesamiento, lo que limita la cantidad de muestras. La respuesta la da el **Teorema de muestreo de Nyquist-Shannon** (se desarrolla en la Unidad II.2).

[NOTA DE CLASE] (Clase 2 p.1) — Tu apunte registra: "*Creo que mientras más aumentás la frecuencia de muestreo, menor información pierdo.*" Es **correcto** y coincide con la filmina. Podés borrar la duda.

Hardware del display digital

- Componentes principales **similares al display vectorial**.
- Incorpora una "**máscara de sombra**".
- Requiere **precisión geométrica** del rayo de electrones.
- Se debe mantener **correspondencia** entre: la **persistencia del fósforo**, la **relación de aspecto** (ancho/alto) y la **resolución** (cantidad de puntos).
- Se pueden representar imágenes **con colores** usando **3 cañones**.

Funcionamiento

- El rayo de electrones **recorre toda la pantalla con un patrón regular**.
- Es un **proceso determinístico e invariante**:
- **No depende de la complejidad** de la imagen.
- **No depende del código de pintado**.

- El **tiempo se mantiene constante**.
- Es posible **repintar toda la pantalla 60 veces por segundo**.
- **Ventajas:** transiciones suaves, mayor resolución.
- Se puede mejorar la velocidad **duplicando los cañones** de electrones (**modo entrelazado**).

Características

- Se **dibujan puntos** que unidos representan las formas.
- Velocidad de pintado **independiente de la complejidad** de la imagen.
- Es posible **rellenar formas de manera sencilla y sin costo de pintado adicional**.

1.6 Comparación: display vectorial vs display raster

Ésta es literalmente la pregunta 1 del parcial anterior.

Característica	Vectorial (caligráfico)	Raster (trama)
Qué dibuja	Segmentos de línea que representan las formas	Puntos (píxeles) que unidos representan las formas
Cómo se almacena la imagen	Como una lista de comandos de dibujo en memoria	Como una matriz de bits en el frame buffer
Velocidad de pintado	Depende de la complejidad de la imagen (cantidad de líneas)	Independiente de la complejidad; tiempo constante
Recorrido del rayo	Arbitrario: se dibujan líneas en cualquier orden	Patrón regular: recorre toda la pantalla siempre igual
Naturaleza del proceso	Depende del código de pintado	Determinístico e invariante
Relleno de formas	Muy costoso: hay que generar nuevos segmentos de línea	Sencillo y sin costo de pintado adicional
Refresco	Re-ejecutar la lista de comandos en loop; 30-60 veces/s para evitar parpadeo	Repintar 60 veces por segundo, siempre en el mismo tiempo
Problema típico	Parpadeo (<i>flicker</i>) con imágenes complejas	Aliasing (escalones); consumo de memoria del frame buffer
Calidad de línea	Líneas continuas y nítidas	Líneas escalonadas (<i>jagged</i>) por la discretización
Origen	Sutherland modifica el CRT (1963)	Incorpora máscara de sombra y barrido regular

Diferencia fundamental

En el **vectorial** la imagen **es** una lista de instrucciones de dibujo, y por lo tanto el tiempo de refresco crece con la cantidad de elementos, hasta producir parpadeo. En el **raster** la imagen es una **matriz de valores almacenada en el frame buffer**, y el rayo la recorre siempre igual: el tiempo es constante e independiente de lo que haya dibujado. Ese cambio de "imagen como procedimiento" a "imagen como dato" es lo que permite el relleno gratuito, y a la vez lo que introduce el aliasing.

Cómo respondería esto en un parcial

El **display caligráfico o vectorial** surge de la modificación que Sutherland hizo al CRT: alteró el control de los elementos deflectores y el de emisión del rayo, de modo de poder dibujar líneas en cualquier orden, con la misma metodología de un pen-plotter. La imagen se almacena como una **lista de comandos** que controla el rayo de electrones; como el fósforo se apaga, esa lista debe volver a ejecutarse en un loop para que las líneas persistan. De ahí su limitación central: **la velocidad de actualización depende de la cantidad de comandos**, es decir, de la complejidad de la imagen. Si el refresco cae por debajo de 30 a 60 veces por segundo aparece el parpadeo. Además, rellenar una forma resulta muy costoso, porque hay que generar nuevos segmentos de línea para cubrir el área.

El **display digital de trama o raster** parte de interpretar la imagen como una secuencia de intensidades de luz que puede muestrearse por líneas. El rayo de electrones **recorre toda la pantalla con un patrón regular**, en un proceso determinístico e invariante: no depende de la complejidad de la imagen ni del código de pintado, y el tiempo se mantiene constante, permitiendo repintar 60 veces por segundo. La imagen se almacena como una matriz de bits en el **buffer de refresco**. Su hardware incorpora una máscara de sombra y exige precisión geométrica del rayo, manteniendo correspondencia entre persistencia del fósforo, relación de aspecto y resolución; con tres cañones se obtiene color.

La diferencia de fondo es que el vectorial **dibuja segmentos** mientras el raster **dibuja puntos**, y eso determina todo lo demás: en el vectorial el costo crece con la complejidad y el relleno es caro; en el raster el costo es constante y el relleno es gratuito. A cambio, el raster introduce la discretización y con ella el **aliasing**, y demanda memoria para el frame buffer.

1.7 Arquitectura del sistema raster y buffer de refresco

Prioridad MUY ALTA. Es la pregunta 2 del parcial anterior, que pide explícitamente "**su función en la arquitectura del dispositivo**".

El problema que resuelve

Los **primeros sistemas raster** utilizaban un **único controlador** para la generación y el pintado de la imagen.

Problema: la velocidad de cambio de imágenes quedaba limitada, porque **había que sincronizar la velocidad de generación de la imagen con la de pintado**. El componente más lento imponía su ritmo al otro.

La solución: separar en tres dispositivos

Se propone ampliar la arquitectura con:

Componente	Función
Controlador de display	Encargado de la generación de la imagen
Controlador de video	Encargado del control del pintado en pantalla
Buffer de refresco (frame buffer)	Almacena la imagen en formato de bits

Esto **permite tener velocidades de generación y pintado diferentes**.

Esquema de la arquitectura



Qué es el buffer de refresco

Es la **memoria que contiene la representación de la imagen en formato de bits**: un valor por cada píxel de la pantalla. Conceptualmente se ubica **entre el proceso que genera la imagen y el proceso que la pinta**, y cumple tres funciones:

1. **Almacenamiento**: guarda el estado de todos los píxeles de la pantalla.
2. **Desacople**: permite que la generación y el pintado ocurran a velocidades distintas, resolviendo el problema de sincronización de la arquitectura original.
3. **Refresco**: el controlador de video lo **lee repetidamente** para repintar la pantalla, porque el fósforo se apaga. Es la fuente de datos de cada refresco.

Tipos de frame buffer

Tipo	Cómo funciona	Resultado
Bi-level (1 bit)	Implementación más simple; el fósforo está encendido o apagado	Imágenes monocromáticas
N-level (n bits)	Permite 2ⁿ intensidades por píxel	1 cañón → escala de grises: $2^8 = 256$ tonos. 3 cañones → colores: $2^{24} \approx 16,7 \times 10^6$
Con tabla de colores (look-up table, LUT)	En una imagen no se utilizan todos los colores posibles (2^n). Se organizan los colores de la imagen en una tabla separada , y el frame buffer contiene los índices en esa tabla para el color de cada píxel	Ahorra memoria y acelera el sistema
"True color" de 24 bits	Cada canal de color (R, G, B) tiene su propio frame buffer de 8 bits , y cada frame buffer tiene su propia tabla de colores de 8 bits	16,7 millones de colores

[NOTA DE CLASE] (Clase 2 p.2) — El profesor justificó la LUT con un ejemplo: "Si estamos pintando un paisaje nocturno seguramente no tenga un color tipo verde lima, van a haber colores que no están presentes. Entonces si escribo unos tonos digamos acotados puedo acelerar el sistema. Entonces guardo en una tabla separada los colores que voy a ir representando."

Fórmula relevante

$$\text{niveles} = 2^n$$

donde **n** = cantidad de bits por píxel (por cañón). Con 3 cañones de 8 bits cada uno: $2^{24} \approx 16,7 \times 10^6$ colores.

Error típico: confundir bits por canal con bits totales. 8 bits **por cañón** con 3 cañones da 2^{24} , no 2^8 .

Doble frame buffer

Escenas complejas con objetos en movimiento demandan tiempos de procesamiento altos. Esto puede generar errores visuales denominados "**artifacts**" por la velocidad del proceso de pintado.

Estrategia: usar **dos frame buffers**.

- **Front buffer** → donde **lee** el controlador de video.
- **Back buffer** → donde **escribe** el dispositivo de procesamiento.

Cuando el cuadro está completo, **se intercambian**. La implementación puede ser **tanto a nivel de hardware como de software**.

[NOTA DE CLASE] (Clase 2 p.2) — "Esto me desacopla la frecuencia entre ambos dispositivos, ya que en un buffer el controlador va a escribir y en el otro va a estar leyendo; cuando termina de leer le dice de switchear para seguir leyendo lo del otro frame."

Screen tearing

Artifact generado por des-sincronización en la lectura/escritura del frame buffer. La pantalla muestra **partes de dos cuadros distintos al mismo tiempo**, produciendo una imagen "rasgada".

[NOTA DE CLASE] (Clase 2 p.2) — "Cuando el controlador de video lee más rápido, antes de que genere la nueva imagen el dispositivo que escribe." Y la cita en inglés que anotaste: "Screen tearing happens when your screen shows parts of two different frames at the same time. This occurs because your graphics card and your monitor are not synced up."

El **doble buffer lo resuelve**, porque nunca se lee un buffer que se está escribiendo.

Explicación complementaria: en OpenGL/GLFW el intercambio de buffers es la llamada `glfwSwapBuffers`, que el template de los prácticos ejecuta en cada vuelta del render loop (*Práctico 01 p.26*). Es el mismo concepto de la Unidad I, aplicado.

Relación con otros conceptos

El frame buffer es la **penúltima etapa del pipeline de rendering**: recibe el resultado del procesamiento y alimenta al dispositivo de visualización. Y es justamente la etapa que, según tu apunte, el profesor decidió no desarrollar en profundidad:

[NOTA DE CLASE] (Clase 3 p.1) — "Durante todo el semestre vamos a estar hablando de estos bloques menos del frame buffer, ya que es más a nivel de hardware."

Esto acota el alcance: el frame buffer hay que saber **qué es, qué contiene y qué función cumple en la arquitectura**, que es exactamente lo que pidió la pregunta 2 del parcial anterior, pero no se profundizó más allá.

Errores o confusiones frecuentes

- **El frame buffer no "dibuja"**: sólo almacena. Quien genera es el controlador de display; quien pinta es el controlador de video.

- **La LUT no reduce la cantidad de colores mostrables**, reduce la cantidad de **bits por píxel** necesarios para indexarlos.
- **El doble buffer no acelera nada**: elimina artifacts de sincronización.

Cómo respondería esto en un parcial

El **buffer de refresco** o *frame buffer* es la memoria que almacena la imagen en formato de bits, es decir, un valor por cada píxel de la pantalla. Conceptualmente se ubica entre el proceso que **genera** la imagen y el proceso que la **pinta**.

Su existencia responde a un problema concreto de los primeros sistemas raster, que usaban un único controlador tanto para generar como para pintar la imagen: eso obligaba a **sincronizar ambas velocidades** y limitaba la velocidad de cambio de imágenes. La solución fue separar la arquitectura en tres dispositivos: un **controlador de display** encargado de la generación, un **controlador de video** encargado del control del pintado, y el **buffer de refresco** que almacena la imagen entre ambos. De este modo, generación y pintado pueden funcionar a **velocidades diferentes**.

Su función en el refresco es central: como el fósforo de la pantalla se apaga, el controlador de video debe **releer el buffer** para repintar la imagen, típicamente 60 veces por segundo. El buffer es la fuente de datos de cada uno de esos repintados.

Según su organización puede ser **bi-level** de 1 bit (monocromático, fósforo encendido o apagado), **n-level** que permite 2^n intensidades (con un cañón de 8 bits da 256 tonos de gris; con tres cañones, $2^{24} \approx 16,7$ millones de colores), o **con tabla de colores** (*look-up table*), donde se aprovecha que una imagen no usa todos los colores posibles: se los guarda en una tabla separada y el buffer almacena sólo los **índices**, ahorrando memoria. El sistema *true color* de 24 bits usa un frame buffer de 8 bits por canal, cada uno con su propia tabla.

Finalmente, cuando la escena es compleja pueden aparecer artifacts por la velocidad del pintado. La estrategia del **doble frame buffer** usa dos buffers: el controlador de video lee del **front buffer** mientras el procesamiento escribe en el **back buffer**, y se intercambian al completar el cuadro. Esto evita el *screen tearing*, que es el artifact en el que la pantalla muestra partes de dos cuadros distintos simultáneamente por desincronización entre lectura y escritura.

1.8 Tecnologías modernas de display

Prioridad MEDIA. Descriptivo, sin cálculo. Conviene saber el mecanismo y las diferencias.

LCD — display de cristal líquido

Construcción: moléculas orgánicas encerradas entre **dos electrodos transparentes** (vidrios) y **dos filtros polarizados a 90°**.

Funcionamiento:

- **En ausencia de voltaje:** las moléculas se alinean en una **estructura helicoidal**, rotando la luz 90° entre los cristales, lo que **alinea la polarización de la luz con el segundo filtro** → **punto gris** (pasa luz).
- **Cuando se aplica voltaje:** las moléculas **se reorganizan como si fueran un líquido**, dejando pasar luz que quedará polarizada **perpendicular** al segundo filtro → **punto oscuro**.

Característica clave: los dispositivos LCD **modifican la luz, no son emisores**. Por eso existen tres variantes:

- **Transmisivos:** usan una luz en plano trasero.
- **Reflexivos:** dependen de una fuente de luz externa.
- **Trans-reflectivos:** combinación de los dos anteriores.

Se pueden crear displays a color interponiendo un **filtro de color** (rojo, verde y azul) entre los cristales.

Matriz pasiva vs matriz activa (TFT)

Característica	Matriz pasiva	Matriz activa (TFT)
Organización	Líneas × columnas (igual que los CRT)	Líneas × columnas
Control del píxel	Se energiza la línea i y la columna j	Cada píxel está conectado a un transistor y un capacitor
Mantenimiento del estado	Las moléculas mantienen su estado poco tiempo	El capacitor mantiene activamente el estado mientras se recorre la grilla
Refresco	Deben ser refrescadas constantemente	Permite mantener el estado de los píxeles durante el recorrido
Resultado	Menor brillo y contraste	Imágenes más brillantes y con mejor contraste

La mayoría de los displays modernos usan la tecnología **thin film transistor (TFT)** para la grilla.

LED

Basado en la tecnología de **semiconductores emisores de luz**. Tienen un procesamiento simple del cambio de estado y están contruidos con una **película fina de polímeros**.

Ventajas: son **emisores de luz**, de bajo voltaje · electrónica simple, flexible, cambio de estado veloz · transparente, excelente brillo, full-color · gran ángulo de visualización · permite cualquier tipo de tamaño.

OLED

Diodos orgánicos emisores de luz. Están formados por **capas electroluminiscentes emisivas de material orgánico compuesto** que emiten luz en respuesta a una corriente eléctrica.

La grilla puede ser **pasiva** (PMOLED) o **activa**, que utiliza la tecnología TFT (**AMOLED**).

Ventajas: bajo consumo eléctrico (**no tiene luz trasera**) · tiempos de respuesta altos (mejores que los LCD) · gran contraste dinámico (*dynamic range*).

Diferencia fundamental

El **LCD no emite luz: la modula**, por eso necesita retroiluminación y su contraste está limitado por la luz que se filtra. El **LED y el OLED sí son emisores**, cada píxel produce su propia luz, lo que elimina la luz trasera, baja el consumo y permite negros reales y por lo tanto mucho mayor contraste dinámico. La distinción **pasiva vs activa (TFT)** es independiente de eso: se refiere a cómo se sostiene el estado de cada píxel mientras se recorre la grilla.

Cómo respondería esto en un parcial

El **LCD** está formado por moléculas orgánicas encerradas entre dos electrodos transparentes y dos filtros polarizados a 90° . Sin voltaje las moléculas adoptan una estructura helicoidal que rota la luz 90° y la alinea con el segundo filtro, dejándola pasar; al aplicar voltaje se reorganizan y la luz queda polarizada perpendicular al segundo filtro, que la bloquea. Lo esencial es que **el LCD no emite luz, la modifica**, por lo que requiere una fuente: transmisivos con luz trasera, reflexivos con luz externa, o trans-reflectivos.

Su control puede ser por **matriz pasiva**, donde para cambiar un píxel se energiza la línea i y la columna j y las moléculas mantienen su estado poco tiempo, obligando a un refresco constante; o por **matriz activa (TFT)**, donde cada píxel tiene un transistor y un capacitor que mantienen activamente su estado mientras se recorre la grilla, logrando imágenes más brillantes y con mejor contraste. Es la tecnología de la mayoría de los displays modernos.

Los **LED** se basan en semiconductores emisores de luz contruidos con película fina de polímeros; **son emisores**, de bajo voltaje, con electrónica simple, cambio de estado veloz, excelente brillo y gran ángulo de visualización. Los **OLED** usan capas electroluminiscentes de material orgánico que emiten luz ante una corriente eléctrica, con grilla pasiva (PMOLED) o activa con TFT (AMOLED); al no tener luz trasera logran bajo consumo, tiempos de respuesta mejores que el LCD y gran contraste dinámico.

Multiple Choice de práctica — Unidad I

1. El término "Computación Gráfica" se atribuye a:

A) Ivan Sutherland, al crear Sketchpad en 1963 B) William Fetter, en 1960, para describir el método de diseño que usaba en Boeing C) Jack Bresenham, en 1965, en IBM D) Edwin Catmull, en 1974, al desarrollar el z-buffer

2. ¿Cuál de las siguientes imágenes es **sintetizada**?

A) Una fotografía digital tomada con un sensor CCD B) Una radiografía obtenida por rayos X C) Un render de POV-Ray generado a partir de un modelo 3D D) Una pintura artística digitalizada con un scanner

3. Una imagen "en tiempo real", según el criterio de la materia, se caracteriza por:

A) Generarse en menos de un segundo por cuadro B) Una duración de 10 a 60 cuadros por segundo C) Usar modelos que representan la física de la luz D) No requerir frame buffer

4. En un display **vectorial**, la velocidad de actualización de la pantalla depende de:

A) La resolución del dispositivo y la relación de aspecto B) La cantidad de líneas a dibujar y la persistencia del fósforo C) El tamaño del buffer de refresco D) La frecuencia de muestreo utilizada en el barrido

5. ¿Por qué el display raster puede repintar la pantalla en un tiempo constante?

A) Porque almacena la imagen comprimida en la tabla de colores B) Porque el rayo recorre la pantalla con un patrón regular, independiente de la complejidad de la imagen y del código de pintado C) Porque utiliza dos cañones de electrones en modo entrelazado D) Porque el fósforo tiene mayor persistencia que en el display vectorial

6. El problema que motivó la separación entre controlador de display, controlador de video y buffer de refresco fue:

A) El aliasing producido por el muestreo discreto de la imagen B) La necesidad de sincronizar la velocidad de generación de la imagen con la de pintado C) La imposibilidad de rellenar áreas en sistemas vectoriales D) El

parpadeo producido por la baja persistencia del fósforo

7. Un frame buffer de **3 cañones de 8 bits** permite representar:

A) 256 colores B) $2^{24} \approx 16,7$ millones de colores C) $2^8 = 256$ tonos de gris por cañón, pero un solo color por píxel D) 1024 colores organizados en una tabla

8. La **tabla de colores** (*look-up table*) de un frame buffer sirve para:

A) Aumentar la cantidad total de colores que el dispositivo puede generar B) Almacenar los colores efectivamente usados en una tabla separada, de modo que el frame buffer guarde sólo índices C) Corregir el aliasing aplicando pesos a los píxeles vecinos D) Sincronizar el front buffer con el back buffer

9. El **screen tearing** se produce porque:

A) La frecuencia de muestreo es menor al doble de la frecuencia máxima de la señal B) El fósforo se apaga antes de completarse el refresco C) Se desincronizan la lectura y la escritura del frame buffer, mostrándose partes de dos cuadros distintos D) La relación de aspecto del frame buffer no coincide con la de la pantalla

10. En un esquema de **doble frame buffer**:

A) El controlador de video escribe en el front buffer y lee del back buffer B) El controlador de video lee del front buffer y el dispositivo de procesamiento escribe en el back buffer C) Ambos controladores escriben alternadamente en el mismo buffer D) El back buffer almacena la tabla de colores y el front buffer los índices

11. ¿Cuál de estas afirmaciones sobre el **LCD** es correcta?

A) Es un dispositivo emisor de luz de bajo voltaje B) Modifica la luz en lugar de emitirla, por lo que requiere una fuente luminosa C) Cada píxel posee un diodo orgánico que emite luz ante una corriente D) Su grilla sólo puede organizarse como matriz activa

12. La diferencia entre matriz **pasiva** y matriz **activa (TFT)** es que en la activa:

A) Se utilizan tres cañones de electrones en lugar de uno B) Cada píxel tiene un transistor y un capacitor que mantienen activamente su estado C) La imagen no necesita refrescarse nunca D) Los filtros polarizados están a 45° en lugar de 90°

13. El **rellenado de áreas** es sencillo y sin costo adicional de pintado en sistemas:

A) Vectoriales, porque las líneas se dibujan en cualquier orden B) Raster, porque la imagen se compone de puntos que ya se recorren en el barrido regular C) En ambos por igual, ya que depende sólo de la resolución D) En ninguno de los dos: siempre requiere generar nuevos segmentos

14. El **trazador aleatorio** (osciloscopio) como primera aplicación del CRT servía para:

A) Representar señales eléctricas analógicas B) Digitalizar imágenes por barrido de líneas C) Almacenar la lista de comandos de dibujo D) Convertir coordenadas 3D en coordenadas de pantalla

15. El **primer Sistema Gráfico Interactivo** y sus componentes básicos corresponden a:

A) El pen-plotter de Boeing, con entrada por tarjetas perforadas B) Sketchpad de Sutherland, con entrada, procesamiento/almacenamiento y salida C) El sistema PHIGS, con controlador de display y de video D) El frame buffer con tabla de colores de 24 bits

Respuestas y explicación — Unidad I

1. **B.** Fetter acuña el término en 1960 en Boeing, describiendo el proceso de creación, manipulación y almacenamiento de modelos e imágenes (*Unidad I p.10*). Sutherland (A) creó el primer sistema gráfico **interactivo**, que es otra cosa.

2. **C.** En las sintetizadas la información **se genera** mediante un modelo o dispositivo. A, B y D son **capturadas**: transforman luz o radiación reflejada por objetos reales. Que el soporte sea digital no las convierte en sintetizadas.
3. **B.** El criterio de la materia es explícitamente **10 a 60 cuadros por segundo** (*Unidad I p.8*). La opción C describe el criterio de *realismo*, que es un eje de clasificación **independiente**.
4. **B.** En el vectorial hay que re-ejecutar la lista de comandos en loop; más líneas implican más tiempo, y la persistencia del fósforo marca el plazo disponible (*Unidad I p.14*). C y D corresponden al raster.
5. **B.** El barrido es **determinístico e invariante** (*Unidad I p.19*). El modo entrelazado (C) es una mejora adicional de velocidad, no la causa de la invariancia.
6. **B.** La arquitectura original usaba **un único controlador**; había que sincronizar generación y pintado (*Unidad I p.20*). D es el problema del display **vectorial**, no del raster.
7. **B.** Tres cañones de 8 bits dan $2^{24} \approx 16,7 \times 10^6$. La opción A confunde bits por canal con bits totales — es el error más común de este tema.
8. **B.** Se aprovecha que una imagen **no usa todos los 2^n colores posibles**; la ganancia es de **memoria**, no de gama. Por eso A es falsa.
9. **C.** Es la definición literal del artifact (*Unidad I p.24*). A describe una violación del teorema de Nyquist (aliasing), que es otro fenómeno.
10. **B.** Front buffer = donde **lee** el controlador de video; back buffer = donde **escribe** el dispositivo de procesamiento (*Unidad I p.23*). La opción A invierte exactamente los roles.
11. **B.** "Los dispositivos LCD modifican la luz (no son emisores)" (*Unidad I p.27*). A describe al LED y C al OLED.
12. **B.** El transistor y el capacitor mantienen el estado mientras se recorre la grilla, logrando más brillo y contraste (*Unidad I p.28*). C es falsa: sigue habiendo refresco.
13. **B.** Es una de las ventajas centrales del raster frente al vectorial (*Unidad I p.25*).
14. **A.** El trazador aleatorio se usaba para representar **señales eléctricas analógicas** (*Unidad I p.12*); recién con la modificación de Sutherland se pudieron dibujar imágenes arbitrarias.
15. **B.** Sketchpad, con los tres componentes: entrada (mouse, lápiz, tabla de digitalización, scanner), procesamiento y almacenamiento, y salida (*Unidad I p.11*).

Preguntas de desarrollo posibles — Unidad I

1. Realice una comparación entre el display caligráfico o vectorial y el display digital de trama o raster, indicando cómo se genera la imagen en cada uno y qué consecuencias tiene esa diferencia.
2. Explique qué es el buffer de refresco en los dispositivos de trama, indicando su función dentro de la arquitectura del dispositivo y cómo interviene en el proceso de refresco de la pantalla.
3. Describa la arquitectura de hardware de un sistema raster, justificando por qué fue necesario separarla en distintos componentes.
4. Explique los distintos tipos de organización de un buffer de refresco y qué ventaja aporta cada uno.
5. Describa el proceso de muestreo y reconstrucción en un display digital, indicando qué información se pierde y por qué.
6. Explique qué es el *screen tearing*, cuál es su causa y qué estrategia se utiliza para evitarlo.

7. Compare las tecnologías LCD, LED y OLED, indicando cuál emite luz y cuál la modifica, y qué consecuencias tiene esa diferencia.
8. Indique cómo se clasifican las imágenes sintetizadas y explique los criterios utilizados.

Guía de respuestas — Unidad I

1. Debe incluir: qué dibuja cada uno (segmentos vs puntos); cómo se almacena la imagen (lista de comandos vs matriz de bits en el frame buffer); dependencia o independencia respecto de la complejidad; costo del relleno; problema típico de cada uno (parpadeo vs aliasing); y el origen de cada tecnología. Cerrar con la diferencia conceptual: imagen como **procedimiento** vs imagen como **dato**.
 2. Definición (memoria que almacena la imagen en formato de bits); ubicación conceptual entre generación y pintado; el problema de sincronización que resuelve; los tres componentes de la arquitectura y la función de cada uno; su rol en el refresco (el controlador de video lo relee porque el fósforo se apaga); mención de los tipos (bi-level, n-level, LUT, true color) y del doble buffer.
 3. Situación previa (un único controlador, con el problema de sincronización); los tres componentes y sus funciones; la consecuencia (velocidades de generación y pintado independientes); un esquema del flujo.
 4. Los cuatro tipos con su mecanismo y su ventaja: bi-level (simplicidad), n-level (2^n intensidades, con los números concretos), LUT (ahorro de memoria por indexación, con la justificación de que no se usan todos los colores), true color 24 bits (8 bits por canal con tabla propia).
 5. Interpretación de la imagen como secuencia de intensidades; el barrido genera una función por línea; muestreo a intervalos regulares; **se pierde información de intensidad y espacial**; reconstrucción por onda de tensión analógica continua; mención de Nyquist como criterio de la frecuencia necesaria.
 6. Definición del artifact (partes de dos cuadros simultáneamente); causa (desincronización lectura/escritura del frame buffer); solución (doble buffer con front y back, e intercambio al completar el cuadro); mención de que puede implementarse en hardware o software.
 7. LCD: mecanismo de las moléculas y los filtros polarizados, **no emisor**, variantes transmisivo/reflexivo/trans-reflectivo, color por filtros RGB. LED: semiconductor emisor, ventajas. OLED: capas electroluminiscentes orgánicas, PMOLED/AMOLED, bajo consumo por ausencia de luz trasera, gran contraste dinámico. Distinguir el eje **emisor/no emisor** del eje **pasiva/activa**.
 8. Dos criterios independientes: por **resultado visual** (realista/foto realista vs artístico/no foto realista, con la distinción de que uno modela la física y el otro el proceso del artista) y por **tiempo de procesamiento** (tiempo real, 10-60 fps, con sus aplicaciones; diferido/producción, segundos a horas por cuadro, con las suyas).
-

EL PIPELINE DE RENDERING

Prioridad MUY ALTA. Es la pregunta 3 del parcial anterior, que pide explícitamente "**realice un esquema general del mismo indicando las operaciones en cada etapa**". Además es el hilo que une todas las unidades: aparece en **ocho documentos distintos**.

Definición

Pipeline de rendering: el conjunto de procesos que permiten visualizar un modelo 3D abstracto por medio de una representación 2D. — *Unidad II-1 p.4, Unidad V p.2, Unidad VI p.2, Unidad VII-1 p.2*

Explicación conceptual

Es la **transformación del modelo matemático del mundo a píxeles en la pantalla**. Tres propiedades lo definen:

- El proceso requiere **múltiples pasos que se realizan en secuencia**.
- **La información de salida de una etapa sirve como información de entrada de la siguiente**.
- Cada etapa resuelve un problema distinto y bien delimitado.

Su valor conceptual para la materia es que **cada unidad del programa corresponde a una etapa del pipeline**. Si se entiende el esquema, se sabe dónde encaja cada tema.

Esquema general



Componentes o etapas

Etapas	Qué recibe	Qué operación realiza	Qué produce	Por qué es necesaria	Unidad
Modelado del mundo virtual (3D)	La descripción del objeto o fenómeno a representar	Construye la descripción geométrica de las formas y las ubica, orienta y dimensiona en la escena mediante transformaciones	Una escena 3D con todos los objetos en coordenadas del mundo	Sin una descripción de la forma no hay nada que proyectar	V (formas) y VI (transformaciones)
Proyección 2D	La escena 3D y los parámetros de visualización (cámara)	Determina qué parte del mundo se ve, recorta lo que queda fuera y aplasta la geometría 3D sobre un plano	Primitivas 2D (segmentos, polígonos) en coordenadas de dispositivo	Hay que reducir tres dimensiones a las dos de la pantalla	VII
Modelo de imagen 2D	Las primitivas 2D proyectadas	Representa la imagen todavía en términos continuos: líneas, circunferencias, polígonos	La descripción 2D de lo que debe pintarse	Es la frontera entre el mundo continuo y el discreto	II
Muestreo / procesamiento	El modelo de imagen 2D continuo	Digitaliza (rasteriza) las primitivas: decide qué píxeles se pintan y con qué intensidad; rellena áreas; aplica antialiasing	Una matriz de valores de píxel	La pantalla es una grilla discreta, no un plano continuo	II
Frame buffer	Los valores de píxel calculados	Almacena la imagen en formato de bits	La imagen lista para ser leída por el controlador de video	Desacopla generación y pintado	I
Dispositivo de visualización	El contenido del frame buffer	Lee repetidamente el buffer y refresca la pantalla	La imagen visible	El fósforo se apaga: hay que repintar	I

El mismo pipeline, en su versión práctica (OpenGL)

(Práctico 01 p.4) — La versión concreta que implementa el hardware:

vértices VBO+VAO	->	vertex shader	->	ensamblado de primitivas	->	recorte (clipping)	->	rasterizado "acá vive Bresenham"	->	fragment shader	->	frame buffer
[PROGR.]		[PROGR.]		[FIJA]		[FIJA]		[FIJA]		[PROGR.]		[FIJA]

- **Tres puntos programables:** los **datos** (vértices y sus atributos), el **vertex shader** (qué hacer con cada vértice) y el **fragment shader** (de qué color sale cada fragmento).
- **Cuatro etapas fijas:** armar triángulos con los vértices, recortar lo que sale de la pantalla, rasterizar (decidir qué píxeles cubre cada triángulo) y escribir el resultado en el framebuffer.

Idea clave que une teoría y práctica. Los algoritmos de la Unidad II (Bresenham, relleno por scanline, recorte) **ya no se programan: están fijos en el hardware de la GPU**, en la etapa de rasterizado. *"Acá vive Bresenham" (Práctico 01 p.4).*

Relación con otros conceptos

El pipeline es el organizador de toda la materia:

Pregunta que responde la etapa	Unidad
¿Cómo describo la forma del objeto?	V — mallas, parches, CSG, voxels
¿Cómo ubico, oriento y dimensiono cada objeto en la escena?	VI — transformaciones
¿Qué ve la cámara y cómo lo aplasto a 2D?	VII — proyecciones y normalización
¿Qué píxeles se prenden para cada primitiva? ¿Cómo evito los serruchos?	II — rasterización y antialiasing
¿Cómo se guarda la imagen y cómo llega a la pantalla?	I — frame buffer y display

Errores o confusiones frecuentes

- **No es un ciclo:** es una secuencia unidireccional; la salida de una etapa entra a la siguiente y no vuelve.
- **La proyección ocurre antes de la rasterización**, no después. Primero se decide *qué y dónde*, después *qué píxeles*.
- **El recorte (clipping) pertenece a la etapa de visualización**, no al modelado.

Cómo respondería esto en un parcial

El **pipeline de rendering** es el conjunto de procesos que permiten visualizar un modelo 3D abstracto por medio de una representación 2D. Consiste en la transformación del modelo matemático del mundo en píxeles en la pantalla, y requiere múltiples pasos que se realizan **en secuencia**, donde la información de salida de una etapa sirve como información de entrada de la siguiente.

(Acá va el esquema.)

En la etapa de **modelado del mundo virtual**, en 3D, se construye la descripción geométrica de los objetos y se los ubica, orienta y dimensiona en la escena mediante transformaciones geométricas. Su salida es la escena completa en coordenadas del mundo.

En la etapa de **proyección** se especifica un observador mediante una cámara sintética, se determina qué objetos caen dentro del volumen de visualización, se recorta lo que queda fuera y se aplasta la geometría sobre un plano de proyección. Su salida son primitivas bidimensionales.

El **modelo de imagen 2D** es todavía una descripción continua: segmentos de recta, circunferencias, polígonos. Es la frontera entre el dominio continuo y el discreto.

La etapa de **muestreo y procesamiento** realiza la digitalización o conversión por barrido: decide qué píxeles deben pintarse para aproximar cada primitiva, rellena las primitivas de área y aplica técnicas de antialiasing para compensar la pérdida de información del muestreo. Su salida es una matriz de valores de píxel.

Esos valores se escriben en el **frame buffer**, que almacena la imagen en formato de bits y desacopla la generación del pintado. Finalmente el **dispositivo de visualización** lee repetidamente ese buffer para refrescar la pantalla.

Cada unidad de la materia corresponde a una etapa: el modelado a la Unidad V, las transformaciones a la VI, la proyección a la VII, la rasterización a la II y el almacenamiento y display a la I.

UNIDAD II — Graficación de primitivas 2D

Parte 1 — Digitalización de primitivas lineales

Prioridad ALTA. No apareció en el parcial anterior, pero ocupa 46 páginas de filminas, 10 de tus apuntes, y contiene el único "hay que saber explicar" de todo tu material.

2.1 El problema de la conversión por barrido

Definición

"Dados dos puntos P y Q en el plano se desea dibujar la línea que une ambos puntos en un sistema raster." — Unidad II-1 p.6

Explicación conceptual

La línea tiene una **descripción matemática continua**, pero la pantalla es una **grilla discreta**. Entonces:

- Hay que **decidir cuáles píxeles deben pintarse** en pantalla.
- Se realiza un proceso de transformación para pasar de un **dominio continuo a un dominio discreto** (muestreo).
- Hay que **escribir los píxeles resultantes siguiendo las líneas del sistema raster**.

Por esto se denomina "**conversión por barrido de línea**" (*scan-conversion line*).

Los elementos básicos para representar formas en 2D son las **líneas**: segmentos de recta, cónicas (círculos, elipses, parábolas), polinomios y splines.

Fórmulas relevantes

Ecuación de la recta, forma algebraica:

$$y(x) = m \cdot x + b \quad m = \frac{y_f - y_0}{x_f - x_0} \equiv \frac{\Delta y}{\Delta x}$$

Ecuación de la recta, forma implícita:

$$F(x, y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b = 0$$

Propiedades de la forma implícita:

- $y(x)$ y $F(x, y) = 0$ representan la **misma línea**.
- Cualquier punto (x, y) que satisfaga $F(x, y) = 0$ **pertenece a la línea**.
- F **no dice cómo calcular y dado x** (ni al revés).
- **El signo de $F(x, y)$ define en qué región del plano se encuentra el punto:**

Signo	Región
$F(x, y) < 0$	Por encima de la línea
$F(x, y) = 0$	Sobre la línea
$F(x, y) > 0$	Por debajo de la línea

⚠ **Ojo:** el criterio es contraintuitivo. Con esta definición de F , **positivo significa debajo**. Es una fuente clásica de error.

2.2 Algoritmo incremental DDA

Definición

DDA = *Digital Differential Analyzer*. En este algoritmo **la línea se muestrea a intervalos unitarios en una dirección y se calcula el valor entero más cercano en la otra coordenada**, según la dirección de la línea.

Cómo funciona

Hay **dos estrategias de avance**, y hay que elegir según la pendiente:

Caso $|m| \leq 1$ — avance unitario en x :

$$\frac{\Delta y}{\Delta x} = m \Rightarrow \Delta y = m \cdot \Delta x ; \text{ como } \Delta x = 1 \Rightarrow \Delta y = m$$
$$y_{i+1} = y_i + m \quad \text{pixel} = (x_{i+1}, \text{round}(y_{i+1}))$$

Caso $|m| > 1$ — avance unitario en y :

$$\Delta x = \frac{1}{m} \cdot \Delta y ; \text{ como } \Delta y = 1 \Rightarrow \Delta x = \frac{1}{m}$$
$$x_{i+1} = x_i + \frac{1}{m} \quad \text{pixel} = (\text{round}(x_{i+1}), y_{i+1})$$

Redondeo: $\text{round}(x) = \text{floor}(x + 0,5)$

Por qué hacen falta dos casos

Si con $m > 1$ se avanzara en x , la coordenada y **saltaría más de un píxel por paso** y la línea quedaría **con huecos**. Siempre se avanza por el **eje de mayor recorrido**.

[NOTA DE CLASE] (Clase 3 p.3) — El profesor explicó el paso intermedio de la derivación: "El problema que tiene esto es que avanzamos por línea de scan; deberíamos guardar el píxel de donde estamos parados para que luego, en la siguiente vuelta de la línea de scan, sepa de dónde arrancar la próxima."

Procedimiento (versión final)

1. Calcular $dx = x_1 - x_0$ y $dy = y_1 - y_0$.
2. $\text{steps} = \max(|dx|, |dy|) \rightarrow$ se avanza por el **eje de mayor recorrido**.
3. $x_{inc} = dx/\text{steps}$, $y_{inc} = dy/\text{steps}$ — uno de los dos vale ± 1 .
4. Partir de (x_0, y_0) y repetir **steps+1** veces: pintar $(\text{round}(x), \text{round}(y))$ y sumar los incrementos.

```

void drawLineDDA(int x0, int y0, int x1, int y1) {
    int dx = x1 - x0, dy = y1 - y0;
    int steps = fabs(dx) > fabs(dy) ? fabs(dx) : fabs(dy);
    float xIncrement = dx / float(steps);
    float yIncrement = dy / float(steps);
    float x = x0, y = y0;
    for (int i = 0; i <= steps; i++) {
        write_pixel(round(x), round(y));
        x += xIncrement; y += yIncrement;
    }
}

```

Ventajas y desventajas

Ventajas	Desventajas
Implementación más rápida que si se implementa la ecuación $y = m \cdot x + b$ (reemplaza una multiplicación por una suma)	Requiere operaciones de punto flotante , que pueden ser costosas
	Se acumula error de redondeo , que puede hacer diverger las posiciones de los píxeles para líneas largas

2.3 Algoritmo de Bresenham (punto medio para rectas)

Definición

Desarrollado en **1965 por Jack Bresenham en IBM**. Es un algoritmo para muestrear una línea de manera más eficiente usando **únicamente aritmética de números enteros**.

- Adecuado para ser implementado en **hardware gráfico de bajo nivel**.
- Este algoritmo, y algunas variantes, es el que **se encuentra implementado en los procesadores gráficos actuales**.

Cómo funciona

Hipótesis de la versión básica:

- La pendiente m es positiva y menor que uno: $0 < m < 1$.
- La recta crece hacia la derecha: $x_0 < x_1$ e $y_0 < y_1$.
- La línea pasa por el punto (x_0, y_0) .

Planteo del algoritmo:

- Hay que decidir cuál es el próximo píxel a pintar: el **adyacente** $E = (x + 1, y)$ o el **adyacente superior** $NE = (x + 1, y + 1)$.
- Se toma el **punto medio** entre los dos posibles píxeles y se evalúa el **signo** de $F(x, y)$.
- El valor que tome $F(x, y)$ se llama **variable de decisión "D"**.
- **La magnitud no importa, ya que nos interesa el signo**. Por eso se puede multiplicar por cualquier constante positiva.

Derivación de la variable de decisión

Se evalúa F en el primer punto medio, $(x_0 + 1, y_0 + \frac{1}{2})$:

$$F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y(x_0 + 1) - \Delta x(y_0 + \frac{1}{2}) + \Delta x b$$

$$= \underbrace{(\Delta y x_0 - \Delta x y_0 + \Delta x b)}_{F(x_0, y_0) = 0} + (\Delta y - \frac{\Delta x}{2}) = \Delta y - \frac{\Delta x}{2}$$

Multiplicando por **2** ambos miembros para evitar la división:

$$D_0 = 2\Delta y - \Delta x$$

Regla de decisión

Signo de D	Dónde está el punto medio	Píxel a pintar
$D > 0$	Por debajo de la línea	$(x_0 + 1, y_0 + 1)$ — adyacente superior (NE)
$D < 0$	Por encima de la línea	$(x_0 + 1, y_0)$ — adyacente (E)

Derivación de los incrementos

Si se pintó el adyacente (E), el próximo punto medio estará en $(x_0 + 2, y_0 + \frac{1}{2})$:

$$F(x_0 + 2, y_0 + \frac{1}{2}) - F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y \Rightarrow \Delta D_{ady} = 2\Delta y$$

Si se pintó el adyacente superior (NE), el próximo punto medio estará en $(x_0 + 2, y_0 + \frac{3}{2})$:

$$F(x_0 + 2, y_0 + \frac{3}{2}) - F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y - \Delta x \Rightarrow \Delta D_{aSup} = 2(\Delta y - \Delta x)$$

Ambos incrementos son constantes: todo el bucle son sumas de enteros.

```
void drawLineBresenham(int x0, int y0, int x1, int y1){
    int dx = x1 - x0, dy = y1 - y0;
    int des_var = 2 * dy - dx;
    const int dd_ady = 2 * dy;
    const int dd_asup = 2 * (dy - dx);
    int x = x0, y = y0;
    write_pixel(x, y);
    for (int i = 0; i < dx; i++) {
        x++;
        if (des_var < 0) { des_var += dd_ady; }
        else { des_var += dd_asup; y++; }
        write_pixel(x, y);
    }
}
```

Ejemplo conceptual: de (2,1) a (7,3)

$\Delta x = 5, \Delta y = 2 \rightarrow D_0 = 2(2) - 5 = -1; \Delta D_{ady} = 4; \Delta D_{aSup} = -6.$

Paso	D usado	Decisión	Píxel	D nuevo
inicio	—	—	(2,1)	-1
1	-1	E	(3,1)	3
2	3	NE	(4,2)	-3
3	-3	E	(5,2)	1
4	1	NE	(6,3)	-5
5	-5	E	(7,3)	-1

Verificación: la recta es $y = 1 + 0,4(x - 2)$; en $x = 4$ vale 1,8 $\rightarrow$ redondea a 2 ✓.

Extensión para todos los casos

- **Pendientes mayores que 1 ($m > 1$):** se usa la **misma estrategia que el algoritmo DDA**: se avanza en dirección "y" y se calcula "x".
- **Pendientes negativas:** se utiliza el concepto de **simetría** y se hace **disminuir** la posición en "x" o en "y" del píxel según el signo del Δx o del Δy respectivamente. Esta modificación permite también **alterar el sentido de trazado** de la línea ($x_0 > x_1$ y/o $y_0 > y_1$).

Comparación DDA vs Bresenham

Característica	DDA	Bresenham
Aritmética	Punto flotante	Sólo enteros
Operación por paso	Suma en flotante + redondeo	Suma de enteros + comparación de signo
Error	Acumula error de redondeo ; diverge en líneas largas	No acumula : la decisión es exacta
Implementación en hardware	Poco adecuada	Muy adecuada ; está en las GPU actuales
Resultado	Los mismos píxeles	Los mismos píxeles

Diferencia fundamental

Los dos algoritmos producen **exactamente los mismos píxeles**; lo que cambia es **cómo se llega a ellos**. El DDA calcula la posición real en punto flotante y la redondea; Bresenham nunca calcula la posición: sólo evalúa el **signo** de una función implícita en el punto medio entre los dos candidatos, lo que se reduce a sumas de enteros. De ahí sus dos ventajas: no hay error acumulado y es implementable en hardware.

2.4 Punto medio para circunferencias

El problema

"Dado un punto P en el plano se desea dibujar una línea curva cuyos puntos se encuentran a una distancia r respecto de P (circunferencia) en un sistema raster."

Ecuación: $r^2 = (x - x_c)^2 + (y - y_c)^2$

Métodos directos y su problema

Con incrementos unitarios en x: $y(x) = y_c \pm \sqrt{r^2 - (x - x_c)^2}$

En coordenadas polares: $x = x_c + r \cos \theta$, $y = y_c + r \sin \theta$, con $\theta_{k+1} = \theta_k + \Delta\theta$

Conclusiones sobre los algoritmos directos:

- **Costosos computacionalmente:** punto flotante, raíz cuadrada, operaciones trigonométricas.
- **Espaciado no uniforme** de los puntos.

Reducción del costo mediante simetría

Una circunferencia presenta **simetría por cuadrantes y por octantes**. Aprovechando esto se pueden calcular los píxeles entre $x = 0$ y $x = y$ (**2.º octante**) y luego construir el resto. Cada punto (x, y) calculado genera **8 puntos**: $(\pm x, \pm y)$ y $(\pm y, \pm x)$.

Estrategia e hipótesis

- **Hipótesis:** valores enteros para la posición del centro (x_c, y_c) y el radio r .
- Se calculan las posiciones para una circunferencia **centrada en el origen**, y luego se desplaza sumando las coordenadas del centro.
- Se calculan los píxeles para el **octante 2** ($0 \leq x \leq y$), donde la pendiente varía entre 0 y -1.

Fórmulas

Ecuación implícita: $F(x, y) = x^2 + y^2 - r^2$

Signo	Región
$F < 0$	Adentro de la circunferencia
$F = 0$	Sobre la circunferencia
$F > 0$	Fuera de la circunferencia

Variable de decisión — punto medio entre el adyacente $(x_k + 1, y_k)$ y el adyacente inferior $(x_k + 1, y_k - 1)$, es decir $(x_k + 1, y_k - \frac{1}{2})$:

$$F(x_k + 1, y_k - \frac{1}{2}) = (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2 = F(x_k, y_k) + (2x_k - y_k + \frac{5}{4})$$

Multiplicando por 4 para evitar la división:

$$D_k = 4F(x_k, y_k) + (8x_k - 4y_k + 5)$$

Regla de decisión:

Signo	Punto medio	Píxel
$D > 0$	Fuera de la circunf.	$(x_k + 1, y_k - 1)$ — adyacente inferior
$D < 0$	Dentro de la circunf.	$(x_k + 1, y_k)$ — adyacente

Incrementos (ya multiplicados por 4):

$$(\Delta D_k)_{aInf} = 8(x_k + 1) - 8y_k + 12 \quad (\Delta D_k)_{ady} = 8(x_k + 1) + 4$$

Valores iniciales, evaluando en $(x_0, y_0) = (0, r)$:

$$D_0 = 5 - 4r \quad (\Delta D_0)_{aInf} = 20 - 8r \quad (\Delta D_0)_{ady} = 12$$

La diferencia clave con la recta

En la recta los incrementos son **constantes**. En la circunferencia **dependen de x_k e y_k** , por lo que **hay que recalcularlos en cada paso**. Siguen siendo operaciones enteras.

```

void drawCircleMP(int r){
    int des_var = 5 - 4*r, dd_ady = 12, dd_ainf = 20 - 8*r;
    int x = 0, y = r;
    write_pixel(x, y);    x++;
    while (y > x) {
        if (des_var < 0) { des_var += dd_ady; }
        else { des_var += dd_ainf; y--; }
        dd_ady = 8*x + 12;          // = 4*(2(x+1)+1)
        dd_ainf = 8*x - 8*y + 20;   // = 4*(2(x+1)-2y+3)
        write_pixel(x, y);    x++;
    }
}

```

Ejemplo conceptual: $r = 10$

$$D_0 = 5 - 40 = -35.$$

x	1	2	3	4	5	6	7
D usado	-35	-23	-3	25	-11	33	21
decisión	ady	ady	ady	inf	ady	inf	inf
píxel	(1,10)	(2,10)	(3,10)	(4,9)	(5,9)	(6,8)	(7,7)

Se termina cuando $y \leq x$ (fin del octante). Luego se aplican las 8 simetrías y se traslada al centro.

2.5 Punto medio para elipses

Prioridad MUY ALTA por tus apuntes. [NOTA DE CLASE] (Clase 3 p.9-10) — "Hay que saber explicar cómo planteamos, dado el gráfico de abajo a la izquierda, cómo planteamos para saber qué próximo píxel pintar (función implícita, variable de decisión, etc.)." Es el **único "hay que saber explicar"** de todos tus apuntes. La consigna pide **el planteo**, no un cálculo numérico.

El problema

"Dados dos puntos P y Q en el plano se desea dibujar en un sistema raster una línea curva en donde la suma de las distancias a los puntos P y Q para cualquiera de sus puntos es constante (elipse)."

- P y Q se denominan **focos** de la elipse.
- El **eje mayor** es el segmento de recta que pasa por los dos focos.
- El **eje menor** es el segmento perpendicular al eje mayor que lo bisecta.

Ecuación general: $Ax^2 + By^2 + Cxy + Dx + Ey + F = 0$

Posición estándar (para simplificar el planteo): eje mayor horizontal (paralelo a x), lo que implica eje menor vertical.

$$\left(\frac{x - x_c}{r_x}\right)^2 + \left(\frac{y - y_c}{r_y}\right)^2 = 1$$

⚠ **Ojo — inconsistencia del material.** Unidad II-1 p.37 dice que r_x es el semieje **mayor** y r_y el menor; pero p.40 asume $r_x < r_y$. Las dos afirmaciones son incompatibles. El algoritmo funciona con la hipótesis de p.40. Conviene mencionarlo si el tema aparece.

Por qué la elipse es distinta de la circunferencia

Dos diferencias, y hay que nombrarlas las dos:

1. **La elipse sólo tiene simetría de cuadrantes**, no de octantes. Cada punto calculado genera **4** puntos, no 8. Por eso hay que recorrer **un cuadrante entero**.
2. Dentro de ese cuadrante **la pendiente cambia de carácter**: arranca casi horizontal y termina casi vertical. Si se avanzara siempre en x , la parte final quedaría con huecos. Por eso hay que **partir el cuadrante en dos regiones**.

Hipótesis

- Valores **enteros** para el centro (x_c, y_c) y las dimensiones de los semiejes r_x y r_y .
- Elipse en **posición estándar**.
- Se asume $r_x < r_y$.
- Se calculan los píxeles para el **cuadrante 1** ($0 \leq x \leq r_x$).

Ecuación implícita

$$F(x, y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$$

Signo	Región
$F < 0$	Adentro de la elipse
$F = 0$	Sobre la elipse
$F > 0$	Fuera de la elipse

Determinación de la región del cuadrante

El límite de las dos regiones se define **cuando se alcanza el valor** $dy/dx = -1$ para la derivada. Derivando implícitamente:

$$\frac{dy}{dx} = -\frac{2r_y^2 x}{2r_x^2 y}$$

Recorriendo el cuadrante desde $x = 0$ hacia $x = r_x$, el cambio de región 1 a región 2 se da cuando:

$$2r_y^2 x \geq 2r_x^2 y$$

La región determina cómo realizar el avance:

- **Región 1** ($|m| < 1$): hay que **aumentar en la dirección "x"** (+1).
- **Región 2** ($|m| > 1$): hay que **disminuir en la dirección "y"** (-1).

Variables de decisión

Región 1 — candidatos $(x_k + 1, y_k)$ y $(x_k + 1, y_k - 1)$; punto medio $(x_k + 1, y_k - \frac{1}{2})$:

$$F(x_k + 1, y_k - \frac{1}{2}) = r_y^2 x_k^2 + r_x^2 y_k^2 - r_x^2 r_y^2 + [r_y^2(2x_k + 1) + r_x^2(\frac{1}{4} - y_k)]$$

Multiplicando por 4:

$$(D_k)_1 = 4F(x_k, y_k) + [4r_y^2(2x_k + 1) + r_x^2(1 - 4y_k)]$$

Incrementos de región 1 (ya $\times 4$):

$$((\Delta D_k)_{ady})_1 = 4r_y^2(2x_k + 1) + 8r_y^2$$

$$((\Delta D_k)_{aInf})_1 = 4r_y^2(2x_k + 1) + 8r_y^2 + 2r_x^2(1 - 4y_k) + 6r_x^2$$

Región 2 — candidatos $(x_k, y_k - 1)$ y $(x_k + 1, y_k - 1)$; punto medio $(x_k + \frac{1}{2}, y_k - 1)$:

$$F(x_k + \frac{1}{2}, y_k - 1) = r_y^2 x_k^2 + r_x^2 y_k^2 - r_x^2 r_y^2 + [r_y^2(x_k + \frac{1}{4}) + r_x^2(1 - 2y_k)]$$

Multiplicando por 4:

$$(D_k)_2 = 4F(x_k, y_k) + [r_y^2(4x_k + 1) + 4r_x^2(1 - 2y_k)]$$

Incrementos de región 2 (ya $\times 4$):

$$((\Delta D_k)_{ady})_2 = 4r_x^2(1 - 2y_k) + 8r_x^2$$

$$((\Delta D_k)_{aInf})_2 = 2r_y^2(4x_k + 1) + 6r_y^2 + 4r_x^2(1 - 2y_k) + 8r_x^2$$

La clave conceptual que hay que poder decir: **el punto medio siempre se toma entre los dos candidatos, y los candidatos dependen de la dirección de avance**. Por eso hay dos fórmulas distintas y no una.

Extensión para todos los casos

- **Elipse completa:** se calculan el resto de los píxeles aplicando las **condiciones de simetría de cuadrantes**.
- **Posición del centro:** se calcula centrada en el origen y luego se realiza una **transformación de desplazamiento** usando (x_c, y_c) .
- Luego de la traslación **se puede rotar** la elipse respecto de su centro para mostrar los ejes alineados a una dirección arbitraria.

2.6 El patrón común a los tres algoritmos

Éste es el concepto que conviene tener claro por encima de las fórmulas particulares.

Paso	Recta	Circunferencia	Elipse
1. Forma implícita	$\Delta y x - \Delta x y + \Delta x b$	$x^2 + y^2 - r^2$	$r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$
2. Punto medio entre los candidatos	$(x_0 + 1, y_0 + \frac{1}{2})$	$(x_k + 1, y_k - \frac{1}{2})$	R1: $(x_k + 1, y_k - \frac{1}{2})$ · R2: $(x_k + \frac{1}{2}, y_k - 1)$
3. Factor que elimina la fracción	$\times 2$	$\times 4$	$\times 4$
4. Incrementos	Constantes	Se recalculan	Se recalculan
5. Simetría aprovechada	—	Octantes (8 puntos)	Cuadrantes (4 puntos)

Cómo respondería esto en un parcial

Los tres algoritmos responden al **mismo planteo**, que consta de cinco pasos.

Primero se escribe la **forma implícita** $F(x, y)$ de la primitiva, es decir, la ecuación llevada a la forma " $= 0$ ". Su utilidad es que **el signo de F clasifica el plano en tres regiones**: adentro/arriba, sobre la curva, y afuera/abajo. A diferencia de la forma explícita, F no permite calcular una coordenada en función de la otra, pero eso no hace falta: sólo hay que **decidir entre dos píxeles candidatos**.

Segundo, se identifican los dos píxeles candidatos —que dependen de la dirección de avance— y se **evalúa F en el punto medio entre ambos**. Si el punto medio cae de un lado, la curva pasa más cerca de un candidato; si cae del otro, del otro. Al valor de F en ese punto se lo llama **variable de decisión D** , y **sólo interesa su signo**, no su magnitud.

Tercero, al desarrollar los cuadrados aparece una fracción: $\frac{1}{2}$ en la recta, $\frac{5}{4}$ en la circunferencia, $\frac{1}{4}$ en la elipse. Como el signo no cambia al multiplicar por una constante positiva, se **multiplica por 2 o por 4** para eliminar la división y trabajar **sólo con enteros**.

Cuarto, se calcula el **incremento** de D para cada una de las dos decisiones posibles, evaluando F en los dos próximos puntos medios y restando. Así, en cada paso no hay que reevaluar F : basta con sumar una cantidad entera. En la recta esos incrementos son **constantes**; en la circunferencia y la elipse **dependen de la posición actual y hay que recalcularlos**, aunque siguen siendo enteros.

Quinto, se **aprovecha la simetría** de la figura —octantes en la circunferencia, cuadrantes en la elipse— y se calcula la primitiva centrada en el origen, trasladándola al final sumando las coordenadas del centro.

Multiple Choice de práctica — Unidad II, parte 1

- El proceso de determinar qué píxeles deben encenderse para representar una primitiva continua se denomina:
A) Muestreo de Nyquist B) Conversión por barrido (*scan-conversion*) C) Transformación de normalización D) Coherencia espacial
- En la forma implícita $F(x, y) = \Delta y x - \Delta x y + \Delta x b$, un valor $F(x, y) > 0$ indica que el punto está:
A) Sobre la línea B) Por encima de la línea C) Por debajo de la línea D) Fuera del segmento, pero sobre su prolongación
- ¿Por qué el algoritmo DDA necesita dos estrategias de avance según la pendiente?
A) Porque con $|m| > 1$ la coordenada calculada saltaría más de un píxel por paso y la línea quedaría con huecos B) Porque la forma implícita cambia de signo cuando $m > 1$ C) Porque el redondeo sólo es válido para pendientes menores que 1 D) Porque con $|m| > 1$ el error de punto flotante se vuelve negativo
- La principal desventaja del DDA frente a Bresenham es:
A) Que requiere calcular una raíz cuadrada por paso B) Que usa punto flotante y acumula error de redondeo C) Que produce píxeles distintos a los de Bresenham D) Que no puede extenderse a pendientes negativas
- En el algoritmo de Bresenham, la variable de decisión se multiplica por 2 para:
A) Duplicar la precisión del resultado B) Eliminar la división manteniendo el signo, que es lo único que interesa C) Compensar el error acumulado en líneas largas D) Permitir el uso de pendientes mayores que 1
- En Bresenham, si $D < 0$ el píxel que corresponde pintar es:

A) El adyacente superior $(x + 1, y + 1)$, porque el punto medio quedó debajo de la recta B) El adyacente $(x + 1, y)$, porque el punto medio quedó por encima de la recta C) El adyacente inferior $(x + 1, y - 1)$ D) Depende del signo de Δx

7. Los valores $\Delta D_{ady} = 2\Delta y$ y $\Delta D_{aSup} = 2(\Delta y - \Delta x)$ del algoritmo de Bresenham:

A) Deben recalcularse en cada iteración B) Son constantes durante todo el trazado del segmento C) Sólo son válidos para el primer octante de una circunferencia D) Cambian de signo al cruzar la mitad del segmento

8. ¿Cuál es la diferencia esencial entre el algoritmo de punto medio para rectas y el de circunferencias?

A) Que el de circunferencias no usa forma implícita B) Que en circunferencias los incrementos de la variable de decisión deben recalcularse en cada paso C) Que el de circunferencias no puede implementarse con enteros D) Que el de rectas aprovecha simetría de octantes

9. En el algoritmo de punto medio para circunferencias, la variable de decisión se multiplica por 4 porque:

A) Se calculan cuatro cuadrantes B) Al desarrollar el punto medio aparece un término $5/4$ C) Se usan cuatro cañones de electrones D) Hay cuatro píxeles candidatos en cada paso

10. El algoritmo de punto medio para circunferencias calcula los píxeles de:

A) Toda la circunferencia, píxel por píxel B) Un cuadrante, y completa el resto por simetría C) El segundo octante, completando el resto con las ocho simetrías D) La mitad superior, reflejando luego sobre el eje x

11. El primer cuadrante de la elipse se divide en **dos regiones** porque:

A) La elipse tiene dos focos B) La pendiente pasa por el valor -1 , y a partir de ahí conviene avanzar en y en lugar de en x C) Los semiejes r_x y r_y son distintos D) Hay que aplicar dos traslaciones diferentes

12. El criterio que marca el paso de la región 1 a la región 2 en la elipse es:

A) $x \geq r_x/2$ B) $2r_y^2x \geq 2r_x^2y$ C) $F(x, y) > 0$ D) $r_x^2 + r_y^2 \geq x^2 + y^2$

13. A diferencia de la circunferencia, la elipse:

A) Sólo presenta simetría de cuadrantes, generando 4 puntos por cálculo B) Presenta simetría de octantes, generando 8 puntos por cálculo C) No admite una formulación implícita D) Requiere operaciones en punto flotante obligatoriamente

14. En la región 2 del algoritmo para elipses, el punto medio que se evalúa es:

A) $(x_k + 1, y_k - \frac{1}{2})$ B) $(x_k + \frac{1}{2}, y_k - 1)$ C) $(x_k + \frac{1}{2}, y_k + \frac{1}{2})$ D) $(x_k + 2, y_k - \frac{3}{2})$

15. Para extender Bresenham a pendientes negativas se utiliza:

A) Una segunda variable de decisión independiente B) El concepto de simetría, decrementando x o y según el signo de Δx o Δy C) La conversión previa a coordenadas polares D) El algoritmo DDA, porque Bresenham no admite pendientes negativas

Respuestas y explicación — Unidad II, parte 1

1. **B.** Es el nombre que le da la filmina: *"por esto se denomina conversión por barrido de línea"* (Unidad II-1 p.7).

2. **C.** Con esta definición de F , **positivo = debajo**. Es contraintuitivo y por eso conviene memorizarlo tal cual (Unidad II-1 p.15).

3. **A.** Siempre se avanza por el **eje de mayor recorrido**; si no, la línea sale discontinua.

4. **B.** Las dos desventajas que enuncia la filmina son el punto flotante y el error acumulado (Unidad II-1 p.13). La opción C es falsa: **dan los mismos píxeles**.

5. **B.** "La magnitud no importa ya que nos interesa el signo" (Unidad II-1 p.16); multiplicar por una constante positiva no lo altera.
6. **B.** $D < 0$ significa punto medio **por encima** de la línea, entonces la recta pasa más abajo y corresponde el adyacente E (Unidad II-1 p.18).
7. **B.** En la recta son **constantes** — ésa es justamente la diferencia con circunferencia y elipse.
8. **B.** En la circunferencia los incrementos dependen de x_k e y_k (Unidad II-1 p.34). D es falsa: la simetría de octantes es de la circunferencia, no de la recta.
9. **B.** $(y_k - \frac{1}{2})^2$ aporta un $\frac{1}{4}$ que sumado al 1 da $\frac{5}{4}$; multiplicar por 2 dejaría $\frac{5}{2}$, aún fraccionario.
10. **C.** "Sólo calcula los píxeles para 1/8 de la circunferencia (2do octante)" (Unidad II-1 p.34).
11. **B.** El límite está donde $dy/dx = -1$; después de ahí un paso en x saltaría más de un píxel en y (Unidad II-1 p.41).
12. **B.** Surge de igualar la derivada a -1 (Unidad II-1 p.41).
13. **A.** La elipse no tiene simetría diagonal, por eso se recorre el cuadrante completo dividido en dos regiones.
14. **B.** En región 2 se avanza **disminuyendo en y**, así que los candidatos son $(x_k, y_k - 1)$ y $(x_k + 1, y_k - 1)$, cuyo punto medio es $(x_k + \frac{1}{2}, y_k - 1)$ (Unidad II-1 p.43). La opción A es el punto medio de la **región 1**.
15. **B.** "Se utiliza el concepto de simetría y se hace disminuir la posición en x o y según el signo del Δx o del Δy respectivamente" (Unidad II-1 p.22).

Preguntas de desarrollo posibles — Unidad II, parte 1

1. Explique en qué consiste la conversión por barrido de una primitiva y por qué es necesaria en un sistema raster.
2. Describa el algoritmo incremental DDA, indicando sus dos estrategias de avance y justificando por qué son necesarias. Indique ventajas y desventajas.
3. Explique el planteo del algoritmo de Bresenham: forma implícita, criterio del punto medio, variable de decisión e incrementos.
4. Compare el algoritmo DDA con el algoritmo de Bresenham.
5. Explique cómo se plantea el algoritmo de punto medio para circunferencias e indique qué diferencia presenta respecto del algoritmo para rectas.
6. Explique el planteo del algoritmo de punto medio para elipses, justificando por qué el cuadrante se divide en dos regiones.
7. Indique qué elementos son comunes a los tres algoritmos de punto medio estudiados.

Guía de respuestas — Unidad II, parte 1

1. Descripción continua vs grilla discreta; hay que decidir qué píxeles pintar; es un proceso de muestreo de continuo a discreto; los píxeles se escriben siguiendo las líneas del raster, de ahí el nombre.
2. Idea del muestreo a intervalos unitarios; las dos estrategias con sus fórmulas ($y_{i+1} = y_i + m$ y $x_{i+1} = x_i + 1/m$); la justificación del eje de mayor recorrido (evitar huecos); el redondeo; ventaja (más rápido que evaluar $y = mx + b$) y desventajas (punto flotante costoso, error acumulado que diverge en líneas largas).
3. Autor y año (Bresenham, IBM, 1965) y por qué importa (sólo enteros, apto para hardware, está en las GPU actuales); hipótesis; forma implícita y sus tres regiones; los dos candidatos E y NE; evaluación en el punto

medio y por qué sólo importa el signo; derivación de $D_0 = 2\Delta y - \Delta x$; los dos incrementos y su constancia; la regla de decisión; mención de las extensiones.

4. Tabla comparativa: aritmética, error, aptitud para hardware, resultado. Cerrar con la diferencia fundamental: producen los mismos píxeles pero Bresenham nunca calcula la posición, sólo evalúa un signo.

5. Problema y ecuación; por qué los métodos directos no sirven (costo computacional y espaciado no uniforme); simetría de octantes; hipótesis y estrategia de centrar en el origen; $F = x^2 + y^2 - r^2$ con sus regiones; punto medio y derivación de $D_0 = 5 - 4r$; el factor 4; **la diferencia clave: los incrementos se recalculan.**

6. Definición de elipse por los focos; posición estándar; las **dos** diferencias con la circunferencia (simetría de cuadrantes y cambio de pendiente); ecuación implícita; derivación del criterio de cambio de región desde $dy/dx = -1$; los dos puntos medios según la dirección de avance; las dos variables de decisión; extensión por simetría y traslación.

7. Los cinco pasos del patrón común, con la tabla comparativa de los tres casos: forma implícita, punto medio, factor multiplicador, constancia o no de los incrementos, simetría aprovechada.

Parte 2 — Primitivas de área, relleno y antialiasing

Prioridad MUY ALTA. Los algoritmos de relleno fueron la **pregunta 4** del parcial anterior, que pidió explícitamente "*explique cómo funcionan*" — es decir, el mecanismo, no la definición.

2.7 Primitivas de área vs primitivas lineales

	Primitivas lineales	Primitivas de área
Ejemplos	Líneas, círculos, arcos	Polígonos, círculos, regiones irregulares
Cómo se definen	Por contornos y perímetros	Requieren determinar los puntos interiores
Qué hay que decidir	Qué píxeles aproximan la curva	Qué píxeles están adentro, y con qué valor rellenar
Algoritmos	DDA, Bresenham, punto medio	Paridad, scanline, boundary fill, flood fill

Polígono: figura plana que se encuentra definida por **tres o más vértices** que se encuentran unidos, en secuencia, por medio de segmentos de recta denominados **lados**.

- **Polígonos convexos:** todos los ángulos interiores $< 180^\circ$.
- **Polígonos cóncavos:** algún ángulo interior $> 180^\circ$.

Clasificación de los algoritmos (Unidad II-2 p.2):

- **Para relleno de polígonos:** relleno por paridad (*parity fill*) y relleno general (scanline).
- **Para relleno de regiones irregulares:** relleno de contornos (*boundary fill*) y relleno por difusión o inundación (*flood fill*).

2.8 Determinación interior/exterior

Antes de rellenar hay que poder decidir si un punto está adentro. Hay dos reglas.

Regla par-impar (even-odd)

1. **Trazar un rayo** desde un punto arbitrario P hacia el infinito.
2. **Contar las intersecciones** con el contorno.
3. **Impar = interior. Par = exterior.**

Regla nonzero winding

1. **Trazar un rayo** desde el punto arbitrario P hacia el infinito.
2. Inicializar el *winding number* en **0**; se acumula analizando los cruces del rayo con los lados.
3. En cada cruce se evalúa el **producto cruz** $\hat{L} \times \hat{S}$ (rayo $\times$ lado).
4. El **signo de la componente z** del producto cruz determina el aporte: **+1 si $z > 0$, -1 si $z < 0$.**
5. Si el winding number final es **distinto de cero**, P es **interior**; si es **cero**, es **exterior**.

Diferencia fundamental

Las dos reglas **coinciden en polígonos simples** y **difieren en los auto-intersectados**. Una región "rodeada dos veces" produce 2 cruces, que es par, y por lo tanto es **exterior** para par-impar; pero su winding number vale ± 2 , distinto de cero, y por lo tanto es **interior** para winding. La diferencia de fondo es que par-impar sólo cuenta **cuántas veces** se cruza la frontera, mientras que winding también considera **en qué sentido** se la cruza.

2.9 Algoritmo de relleno por paridad

Cómo funciona

Para cada polígono:

1. **Identificar los extremos del polígono:** - $(x_{min}, x_{max}) \rightarrow$ columnas que ocupa el polígono. - $(y_{min}, y_{max}) \rightarrow$ scanlines a recorrer.
2. **Recorrer cada scanline** de izquierda a derecha, **desde afuera** del polígono.
3. Al iniciar cada scanline, fijar un estado como un **bit de paridad en 0 (par)**.
4. **Al encontrar un borde, invertir el estado.**
5. **Pintar el píxel si el estado es interno (impar).**

Ventajas y desventajas

Ventajas	Desventajas
Funciona bien para polígonos convexos y cóncavos en general	Hay que procesar cada polígono de manera aislada
Es rápido y se adapta bien para implementar en hardware	No da buenos resultados con figuras que se auto-intersectan

2.10 Algoritmo general de scanline para polígonos

El concepto de coherencia

Coherencia: relación, conexión o unión de unas cosas con otras, o aquello que interconecta o mantiene unidas las partes de un todo. — *Unidad II-2 p.7*

Tipo	Definición
Coherencia de la escena	Las diferentes partes de una escena que podrían brindar una ventaja al momento de procesar el pintado
Coherencia espacial	Las primitivas no cambian de píxel a píxel dentro de un intervalo, o de una línea de scan a otra
Coherencia de línea de scan	En algunas figuras, como los rectángulos, las líneas de scan consecutivas son iguales. En figuras generales con líneas iguales se denomina coherencia de bordes/lados

La coherencia es la **justificación conceptual de todas las optimizaciones** que siguen.

Planteo del algoritmo

1. **Recorrer el polígono de abajo hacia arriba.**
2. Para cada línea de scan ($y = k$), **determinar las intersecciones** con los lados del polígono.
3. **Ordenar** las intersecciones de manera ascendente según "x" (izquierda a derecha).
4. **Recorrer la línea de scan** de izquierda a derecha: - Si la coordenada "x" actual está en la lista de intersecciones, **invertir el estado**. - **Pintar el píxel si el estado es interno (impar)**.

Búsqueda de intersecciones (paso 2)

Como los lados son segmentos rectos, $y = mx + b$, las intersecciones con la scanline y_k se encuentran resolviendo:

$$x = \frac{y}{m} - \frac{b}{m}, \quad m = \frac{\Delta y}{\Delta x} \Rightarrow x_k = \frac{\Delta x}{\Delta y} y_k - \frac{\Delta x}{\Delta y} b$$

Estos valores **se ordenan y se almacenan** para usarlos en el algoritmo.

Manejo de casos especiales

Problema: scan-line que pasa por un vértice compartido.

- Puede generar un **número impar de intersecciones**.
- Resultado **incorrecto** en la determinación interior/exterior.

Solución: análisis topológico.

Situación	Criterio	Cuenta como
Extremo local	Los bordes quedan en el mismo lado de la scanline (un "pico" o un "valle")	2 intersecciones
Continuación	Los lados quedan en zonas opuestas : la frontera "sigue"	1 intersección

Implementación: recorrer los lados del polígono; determinar si los lados que comparten un vértice "continúan"; **en ese caso, acortar el lado inferior en 1 unidad.**

Problema: lados horizontales. Generan **múltiples intersecciones** con su propia scanline. **Solución simple: ignorar el lado.**

Optimización 1 — Intersecciones incrementales (coherencia espacial)

Se utiliza **coherencia de bordes** para reconstruir los lados durante el cálculo de las intersecciones. Se puede calcular la siguiente intersección **a partir de la actual**, con un algoritmo incremental:

$$x_{k+1} = x_k + \frac{1}{m}, \quad m = \frac{\Delta y}{\Delta x} \Rightarrow x_{k+1} = x_k + \frac{\Delta x}{\Delta y}$$

Error típico: sumar m en lugar de $1/m$. Se avanza en **y**, no en **x**.

Optimización 2 — Tablas de lados

Tabla de lados global (GET — *Global Edge Table*)

- Se utiliza una estructura de datos para usar un **algoritmo de ordenamiento de casilleros** (*bucket sort* o *bin sort*).
- Almacena los lados en una tabla global **ordenados por coordenada** y_{min} .
- Cada entrada de la tabla es un **casillero** (*bucket*).
- En cada casillero se almacenan los **datos mínimos para reconstruir los lados** que inician o terminan en la línea de scan correspondiente.
- Los datos de reconstrucción se encuentran **ordenados por coordenada** $x_{min} = f(y_{min})$.

Datos de reconstrucción: $y_{max} \cdot x_{min} = f(y_{min}) \cdot 1/m$

Tabla de lados activos (AET — *Active Edge Table*)

- Se almacenan los datos de reconstrucción de **todos los lados que intersecta la línea de scan activa** (y_k).
- Los datos se **ordenan de acuerdo al valor de la coordenada "x"**.
- Cada entrada: $y_{max} \cdot x_{actual} \cdot 1/m$

Algoritmo optimizado — el orden de los pasos

Este orden se pide. Conviene memorizarlo.

- Crear** la tabla de lados global (GET).
- Inicializar** la scanline $y_k = y_{min}$ (índice del primer casillero no vacío de GET).
- Inicializar** la tabla de lados activos (AET) en cero.
- Repetir** hasta que GET y AET se encuentren vacías: - a. **Mover** los lados de GET con $y_{min} = y_k$ a la AET. - b. **Ordenar** la AET según x (eventualmente la AET incluirá valores nuevos y viejos). - c. **Pintar** los píxeles para y_k utilizando **pares** de coordenadas x de la AET. - d. **Eliminar** las entradas de la AET con $y_{max} = y_k$ (lados terminados). - e. **Avanzar** a la siguiente línea de scan ($y_k + 1$). - f. **Actualizar** x en las entradas remanentes de la AET (sumar $1/m$).

```
struct Edge {
    int   ymax;    // Coordenada y maxima
    float x;       // Interseccion x actual
    float dx;      // 1/pendiente
};

void scanLineFill(Polygon poly, Color fillColor) {
    GET = buildSortedEdgeTable(poly);
    AET = NULL;
    for (y = ymin; y <= ymax; y++) {
        addNewEdges(GET[y], AET);           // bordes que inician en y
        fillSpans(AET, y, fillColor);       // rellenar entre pares de intersecciones
        removeCompletedEdges(AET, y);      // bordes que terminan en y
        updateActiveEdges(AET);            // x += 1/m
    }
}
```

Verificación conceptual: en cada scanline la cantidad de intersecciones debe ser **par**. Si da impar, falta aplicar el análisis topológico de los vértices.

2.11 Algoritmos de relleno de regiones irregulares

Ésta es la pregunta 4 del parcial anterior.

Contexto

- Los algoritmos intentan rellenar figuras con **bordes curvos o irregulares** (no polígonos).
- Usualmente son utilizados por **artistas gráficos**, que bosquejan los bordes de una figura y luego seleccionan el color o patrón de relleno desde un menú.
- **Comienzan con un punto conocido dentro de la figura** y luego proceden a rellenar la figura **hacia el exterior** del punto.
- En paquetes de pintado interactivo son las funciones de pintado tipo **balde de pintura**.

Conectividad de píxeles

- **4-conectado**: arriba, abajo, izquierda, derecha.
- **8-conectado**: incluye además las **diagonales**.

Explicación complementaria: con 8-conectividad el relleno puede "escaparse" por una diagonal si el borde está trazado con 4-conectividad.

Relleno de contorno (*Boundary Fill*)

Concepto:

- **Partir de un punto interior conocido.**
- **Expandir hasta encontrar el color de contorno.**
- Útil para regiones con **bordes irregulares**.

Mecanismo: para el píxel actual se lee su color; si **no es el color de contorno** y **no es el color de relleno**, se lo pinta con el color de relleno y se repite recursivamente sobre sus vecinos.

```
void boundaryFill4(int x, int y, int fillColor, int borderColor) {
    int interiorColor;
    getPixel(x, y, interiorColor);
    if ((interiorColor != borderColor) && (interiorColor != fillColor)) {
        setPixel(x, y, fillColor);
        boundaryFill4(x + 1, y, fillColor, borderColor);
        boundaryFill4(x - 1, y, fillColor, borderColor);
        boundaryFill4(x, y + 1, fillColor, borderColor);
        boundaryFill4(x, y - 1, fillColor, borderColor);
    }
}
```

La doble condición es esencial: la comparación contra `borderColor` es la que **detiene** la expansión; la comparación contra `fillColor` es la que evita **volver sobre píxeles ya pintados** y caer en recursión infinita.

Ventajas	Desventajas
Funciona con cualquier forma de contorno	Requiere un stack recursivo profundo
Implementación intuitiva	Puede ser lento para áreas grandes
Ideal para aplicaciones interactivas	Problemas si hay píxeles ya coloreados

Relleno por difusión (*Flood Fill*)

Diferencias con el relleno de contorno:

- No busca un color de contorno específico.
- Reemplaza el color interior existente.
- Útil para **regiones multicolor**.

Mecanismo: para el píxel actual se lee su color; si **es igual al color interior** buscado, se lo pinta con el color de relleno y se repite recursivamente sobre sus vecinos. Se detiene ante **cualquier píxel que no sea del color interior**.

```
void floodFill4(int x, int y, int fillColor, int interiorColor) {
    int color;
    getPixel(x, y, color);
    if (color == interiorColor) {
        setPixel(x, y, fillColor);
        floodFill4(x + 1, y, fillColor, interiorColor);
        floodFill4(x - 1, y, fillColor, interiorColor);
        floodFill4(x, y + 1, fillColor, interiorColor);
        floodFill4(x, y - 1, fillColor, interiorColor);
    }
}
```

Aplicaciones: cambio de color en regiones existentes · herramienta "balde de pintura" · selección de regiones similares.

Casos especiales: regiones con múltiples colores interiores · pre-procesamiento para unificar colores.

Consideraciones importantes: verificar que $\text{fillColor} \neq \text{interiorColor}$ · manejar los límites del área de dibujo · optimización similar por spans horizontales.

Optimización por spans (para ambos)

- Rellenar líneas horizontales completas.
- Apilar sólo las posiciones iniciales de los spans.
- Reducir el uso de memoria del stack.

Comparación

Característica	Boundary fill	Flood fill
Punto de partida	Punto interior conocido	Punto interior conocido
Condición de avance	El píxel no es el color de contorno ni el de relleno	El píxel es el color interior buscado
Se detiene en	El color de contorno	Cualquier color que no sea el interior
Parámetro que recibe	fillColor y borderColor	fillColor e interiorColor
Tipo de región	Región con borde de un color definido	Región de color interior uniforme , borde multicolor
Uso típico	Rellenar una figura bosquejada con un contorno	Recolorear una zona existente
Riesgo	Recursión infinita si falta comparar contra fillColor	Recursión infinita si fillColor = interiorColor

Diferencia fundamental

Los dos parten de un punto semilla y se expanden por conectividad; la diferencia está en **qué condición usan para detenerse**. El *boundary fill* mira **hacia el borde**: avanza mientras no encuentre el color de contorno, por lo que necesita que el contorno sea de un color definido y uniforme. El *flood fill* mira **hacia adentro**: avanza mientras el píxel sea del color interior que se quiere reemplazar, por lo que sirve aunque el borde sea multicolor, pero exige que el interior sea homogéneo.

Cómo respondería esto en un parcial

Los algoritmos de relleno de áreas se utilizan para rellenar figuras con **bordes curvos o irregulares**, que no son polígonos y por lo tanto no pueden tratarse con el algoritmo de scanline. Ambos **parten de un punto conocido dentro de la figura** —el punto semilla— y proceden a rellenar hacia el exterior por **conectividad**, que puede ser **4-conectada** (arriba, abajo, izquierda y derecha) u **8-conectada** (incluyendo las diagonales). Son la base de la herramienta "balde de pintura" de los paquetes de pintado interactivo.

El **relleno de contorno** (*boundary fill*) parte del punto interior y **expande hasta encontrar el color de contorno**. En cada píxel lee su color y verifica dos condiciones: que **no sea el color de contorno**, que es lo que detiene la expansión, y que **no sea el color de relleno**, que evita volver sobre píxeles ya pintados y caer en recursión infinita. Si ambas se cumplen, pinta el píxel y se llama recursivamente sobre sus vecinos. Su ventaja es que funciona con cualquier forma de contorno y es intuitivo e ideal para aplicaciones interactivas; sus desventajas son que requiere un stack recursivo profundo, puede ser lento en áreas grandes y presenta problemas si hay píxeles previamente coloreados.

El **relleno por difusión** (*flood fill*) se diferencia en que **no busca un color de contorno específico**, sino que **reemplaza el color interior existente**. En cada píxel verifica si su color coincide con el color interior buscado; si coincide, lo pinta y continúa recursivamente. Se detiene ante **cualquier** color distinto del interior. Esto lo hace útil para regiones cuyo borde es **multicolor**, y su aplicación típica es recolorear una zona existente. Requiere verificar que el color de relleno sea distinto del interior, o también cae en recursión infinita.

En ambos casos, la recursión píxel a píxel consume mucho stack, por lo que se aplica una **optimización por spans**: se rellenan líneas horizontales completas y se apilan únicamente las posiciones iniciales de los spans de las filas superior e inferior, reduciendo notablemente el uso de memoria.

La diferencia esencial entre ambos es la **condición de detención**: el boundary fill se detiene en un color de contorno definido, y el flood fill se detiene en todo color que no sea el interior que está reemplazando.

2.12 Aliasing y antialiasing

Definición

Aliasing: la **pérdida de información causada por submuestreo** (*undersampling*). Los errores causados por el aliasing se denominan **"artifacts"**.

Artifacts clásicos

- **Perfiles escalonados** (*jagged*).
- **Detalles faltantes** o renderizado incorrecto.
- **Texturas que se deforman**.

Causa

- Los algoritmos de primitivas de trama (Bresenham, punto medio) generan objetos con **apariencia escalonada**.
- Esta distorsión se debe al **proceso de muestreo discreto de modelos continuos**.
- **Problema fundamental**: representar objetos con **parámetros continuos** usando **píxeles discretos**.

Teoría del muestreo — Teorema de Nyquist

Para evitar la pérdida de información hay que muestrear con una frecuencia de **al menos dos veces la frecuencia máxima** del proceso:

$$f_s \geq 2f_{max} \quad \Longleftrightarrow \quad \Delta x_s \leq \frac{\Delta x_{ciclo}}{2}$$

Variable	Significado
f_s	Frecuencia de muestreo mínima
f_{max}	Frecuencia máxima del objeto
Δx_s	Intervalo de muestreo de Nyquist

Errores típicos: decir "el doble de la frecuencia de muestreo" en vez de "el doble de la frecuencia **máxima de la señal**"; e invertir el sentido de la desigualdad al pasar de frecuencia ($\geq$) a intervalo ($\leq$).

Limitación fundamental: para representar objetos con parámetros continuos necesitaríamos **intervalos de muestreo arbitrariamente pequeños**. Un borde nítido tiene frecuencias arbitrariamente altas.

Limitaciones de las soluciones por hardware

- **Incrementar la resolución mejora pero no elimina** el problema.
- Limitaciones del frame buffer: tamaño máximo limitado por la tecnología.
- Mantener 60+ fps requiere compromisos.
- Costo computacional y de memoria.
- → **Necesidad de técnicas de software: antialiasing** como solución complementaria.

Las tres técnicas de antialiasing

Técnica	Idea	Detalle
1. Supersampling (<i>postfiltering</i>)	Muestreo a alta resolución y combinación de resultados	Cálculo de intensidades subpíxel
2. Area sampling (<i>prefiltering</i>)	Cálculo de áreas de solapamiento	Determinación directa de la intensidad, sin cálculos subpíxel
3. Técnicas de filtrado	Funciones de peso continuas	Integración sobre la superficie del píxel; mayor precisión matemática

Criterios de selección: calidad visual · costo computacional · memoria requerida · disponibilidad en el hardware.

Supersampling de líneas rectas

Idea básica: renderizar a resolución mayor, luego filtrar y submuestrear.

Metodología:

1. **Dividir cada píxel en $n \times n$ subpíxeles.**
2. **Aplicar el algoritmo de Bresenham** en los subpíxeles.
3. **Contar los subpíxeles que solapan la línea.**
4. **Establecer la intensidad proporcional al conteo.**

Niveles de intensidad: $3 \times 3 \rightarrow$ hasta 3 niveles $\cdot 4 \times 4 \rightarrow$ hasta 4 niveles $\cdot 5 \times 5 \rightarrow$ hasta 5 niveles.

△ **Ojo:** ese conteo de niveles corresponde a una línea de **ancho cero**, que atraviesa como mucho n subpíxeles por píxel. La propia filmina aclara en la página siguiente que con **ancho finito** todos los subpíxeles pueden contribuir y hay más niveles.

Líneas de ancho finito: todos los subpíxeles pueden contribuir $\rightarrow$ mayor distribución de intensidad $\rightarrow$ se pueden considerar colores de fondo:

$$\text{color}_{\text{pixel}} = \frac{n_{\text{linea}} \cdot \text{color}_{\text{linea}} + n_{\text{fondo}} \cdot \text{color}_{\text{fondo}}}{n_{\text{total}}}$$

Máscaras de peso (weighting masks): dar **más importancia a los subpíxeles centrales**. Se implementan como arrays de pesos aplicados durante el cálculo de intensidades promedio.

Ejemplo de máscara 3×3 :

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad m_{22} = 4, \quad \sum m_{ij} = 16 \Rightarrow w_{22} = \frac{4}{16} = \frac{1}{4}$$

Area sampling de líneas

Idea básica: intensidad proporcional al **área de solapamiento píxel-línea**.

Metodología:

1. Tratar la línea como un rectángulo de ancho finito.
2. Calcular la intersección polígono-píxel.
3. Determinar el porcentaje de cobertura.
4. Establecer la intensidad proporcional.

$$\% \text{cobertura} = \frac{S_{\text{trap}}}{S_{\text{pixel}}} = m x_k + b - y_k + \frac{1}{2}$$

[NOTA DE CLASE] (Clase 7 p.2) — El profesor explicó de dónde sale: "Calcula el área matemáticamente; para una recta es bastante fácil si usamos el algoritmo de Bresenham. Para calcular el área del segundo gráfico nos podemos dar cuenta que es un trapecio; esto lo explica para llegar al porcentaje de cobertura."

Funciones de filtro

Concepto: superficie de peso continua sobre el píxel.

Tipos comunes: **Box Filter** (peso uniforme) · **Cone Filter** (peso decreciente radial) · **Gaussian Filter** (distribución gaussiana).

$$I_{\text{pixel}} = \iint_{\text{pixel}} f(x, y) \cdot w(x, y) dx dy$$

donde $f(x, y)$ es la función objeto y $w(x, y)$ el filtro.

[NOTA DE CLASE] (Clase 7 p.3) — "Lo que hago es integrar sobre la distribución del píxel donde estoy parado (?). Esto lo dejamos para la parte de imagen." El signo de pregunta es tuyo, y la aclaración final es importante: **el tema de filtrado se difiere a la Unidad IV (Imágenes), que no está entre el material entregado.** No se espera profundidad acá.

[NOTA DE CLASE — contenido faltante] (Clase 7 p.1) — En tus apuntes figura el título "Otra forma de calcular el supersampling" **sin ningún contenido debajo.** Es un método que el profesor desarrolló en clase y quedó sin registrar. **Conviene preguntarlo en consulta.**

Diferencia fundamental entre supersampling y area sampling

Ambos buscan asignarle al píxel una intensidad **proporcional a cuánto lo cubre la primitiva**, pero llegan por caminos opuestos. El **supersampling** es *postfiltering*: **discretiza más fino** —subdivide el píxel en una grilla $n \times n$, rasteriza ahí y cuenta—, de modo que la cobertura se **estima por conteo**. El **area sampling** es *prefiltering*: **no subdivide nada**, calcula **analíticamente** el área de solapamiento entre la primitiva y el píxel. El primero es más general y directo de implementar; el segundo es exacto y más barato, pero requiere poder resolver la geometría de la intersección.

Multiple Choice de práctica — Unidad II, parte 2

1. La diferencia esencial entre una primitiva lineal y una de área es que la de área:
A) Siempre tiene bordes curvos B) Requiere determinar los puntos interiores y elegir el valor de relleno C) No puede rasterizarse con algoritmos incrementales D) Sólo existe en sistemas vectoriales
2. Según la regla **par-impar**, un punto es interior cuando el rayo trazado desde él hacia el infinito corta el contorno:
A) Un número par de veces B) Un número impar de veces C) Exactamente dos veces D) Ninguna vez
3. Las reglas par-impar y nonzero winding dan resultados distintos en:
A) Polígonos cóncavos B) Polígonos con lados horizontales C) Polígonos que se auto-intersectan D) Polígonos con más de ocho vértices
4. En el algoritmo de relleno por paridad, el bit de paridad se inicializa en 0 y:
A) Se invierte al encontrar cada borde; se pinta cuando el estado es impar B) Se incrementa en cada píxel; se pinta cuando supera el umbral C) Se invierte en cada píxel; se pinta cuando es par D) Se fija según el signo del producto cruz con el lado
5. En la tabla de lados global (GET), los lados se almacenan ordenados por:
A) Su longitud, de mayor a menor B) Su coordenada y_{min} , en casilleros (*buckets*) C) Su pendiente m D) El orden en que aparecen los vértices del polígono
6. Los datos mínimos de reconstrucción que se guardan de cada lado son:
A) Los dos vértices completos del lado B) y_{max} , $x_{min} = f(y_{min})$ y $1/m$ C) x_{min} , x_{max} y la pendiente m D) El color de relleno, y_{min} y y_{max}
7. Cuando una scanline pasa por un vértice donde los dos lados quedan **en zonas opuestas** (continuación), ese vértice debe contarse como:
A) 2 intersecciones, sin corrección adicional B) 1 intersección, acortando el lado inferior en 1 unidad C) 0 intersecciones, ignorando ambos lados D) 2 intersecciones, duplicando el lado superior

8. El tratamiento que reciben los **lados horizontales** en el algoritmo de scanline es:

A) Se les asigna pendiente infinita y se procesan al final B) Se ignoran C) Se subdividen en segmentos verticales equivalentes D) Se cuentan como dos intersecciones

9. En el algoritmo optimizado de scanline, dentro del bucle principal el orden correcto es:

A) Ordenar · mover de GET a AET · eliminar · pintar · actualizar x B) Mover de GET a AET · ordenar por x · pintar · eliminar los terminados · avanzar · actualizar x C) Pintar · mover de GET a AET · ordenar · actualizar x · eliminar D) Mover de GET a AET · pintar · ordenar · actualizar x · eliminar

10. La actualización incremental de la intersección de un lado con la scanline siguiente se hace sumando:

A) m B) $1/m = \Delta x / \Delta y$ C) $\Delta y / \Delta x$ D) 1

11. El algoritmo de **relleno de contorno** (*boundary fill*) detiene su expansión cuando encuentra:

A) Cualquier píxel de color distinto al interior B) El color de contorno C) El borde del área de dibujo únicamente D) Un píxel cuyo winding number es cero

12. ¿Cuál es la diferencia central entre *boundary fill* y *flood fill*?

A) Boundary fill usa 4-conectividad y flood fill usa 8-conectividad B) Boundary fill se detiene en el color de contorno; flood fill reemplaza un color interior y se detiene ante cualquier otro color C) Flood fill no es recursivo D) Boundary fill sólo funciona con polígonos convexos

13. En el `boundaryFill4`, la comparación `interiorColor != fillColor` es necesaria para:

A) Detectar el contorno de la figura B) Evitar volver sobre píxeles ya pintados y caer en recursión infinita C) Determinar la conectividad del píxel D) Calcular el porcentaje de cobertura

14. La **optimización por spans** en los algoritmos de relleno de áreas consiste en:

A) Rellenar líneas horizontales completas y apilar sólo las posiciones iniciales de los spans B) Dividir el área en subpíxeles y promediar C) Ordenar los píxeles por coordenada x antes de pintar D) Reemplazar la recursión por una tabla de lados activos

15. El **aliasing** se define como:

A) La distorsión producida por el redondeo en el algoritmo DDA B) La pérdida de información causada por submuestreo C) El desfase entre lectura y escritura del frame buffer D) El error de composición al multiplicar matrices en orden incorrecto

16. Según el teorema de Nyquist, para no perder información hay que muestrear con una frecuencia:

A) Igual a la frecuencia máxima de la señal B) De al menos el doble de la frecuencia máxima de la señal C) De al menos el doble de la frecuencia de muestreo previa D) Inversamente proporcional al intervalo de muestreo del ciclo

17. Aumentar la resolución del dispositivo frente al aliasing:

A) Lo elimina por completo si se supera el límite de Nyquist B) Mejora el resultado pero no elimina el problema, y tiene costo de memoria y fps C) No tiene ningún efecto sobre el aliasing D) Sólo sirve para líneas de ancho finito

18. La diferencia entre *supersampling* y *area sampling* es que:

A) El supersampling subdivide el píxel y cuenta subpíxeles; el area sampling calcula analíticamente el área de solapamiento B) El supersampling es prefiltering y el area sampling es postfiltering C) El area sampling requiere una grilla de $n \times n$ subpíxeles D) El supersampling sólo se aplica a polígonos y el area sampling a líneas

19. En una máscara de pesos 3×3 con valores $\{1, 2, 1; 2, 4, 2; 1, 2, 1\}$, el peso del subpíxel central es:

A) 4 B) $4/9$ C) $4/16 = 1/4$ D) $1/3$

20. La fórmula $\% \text{cobertura} = m x_k + b - y_k + \frac{1}{2}$ corresponde a:

A) La variable de decisión de Bresenham B) La estimación del área de solapamiento en area sampling, considerando la región como un trapecio C) La intensidad resultante del filtro gaussiano D) La actualización incremental de la AET

Respuestas y explicación — Unidad II, parte 2

1. **B.** Las lineales se definen por contornos y perímetros; las de área requieren **determinar los puntos interiores** y **elegir el valor de relleno** (Unidad II-2 p.2). La opción A es falsa: los polígonos son primitivas de área y tienen lados rectos.

2. **B.** Impar = interior, par = exterior (Unidad II-2 p.4).

3. **C.** Coinciden en polígonos simples; en los auto-intersectados una zona rodeada dos veces da 2 cruces (par $\rightarrow$ exterior para par-impar) pero winding $\pm 2 \neq 0$ (interior).

4. **A.** Bit de paridad a 0 al iniciar cada scanline, invertir al encontrar un borde, pintar si es impar (Unidad II-2 p.5).

5. **B.** Ordenados por y_{min} en casilleros, usando *bucket sort* (Unidad II-2 p.13).

6. **B.** Son exactamente los tres campos que enuncia la filmina (Unidad II-2 p.13). La opción C confunde m con $1/m$ y omite y_{max} .

7. **B.** Continuación = 1 intersección, y la implementación es **acortar el lado inferior en 1 unidad** (Unidad II-2 p.10). La opción A corresponde al **extremo local**.

8. **B.** "Solución: simple — ignorar el lado" (Unidad II-2 p.11).

9. **B.** Es el orden de la filmina (Unidad II-2 p.16). El punto crítico: **se pinta ANTES de eliminar** los lados terminados.

10. **B.** Se avanza en y , así que x avanza $1/m = \Delta x / \Delta y$ (Unidad II-2 p.12). Sumar m (opción A) es el error clásico.

11. **B.** Es la definición del boundary fill (Unidad II-2 p.19). La opción A describe al **flood fill**.

12. **B.** Es la diferencia conceptual central. La conectividad (opción A) es independiente: ambos admiten 4 u 8.

13. **B.** Sin esa comparación, al volver sobre un píxel ya pintado la condición seguiría siendo verdadera y la recursión no terminaría.

14. **A.** (Unidad II-2 p.20). La opción B describe supersampling; la D, el algoritmo de scanline.

15. **B.** Definición literal (Unidad II-2 p.24).

16. **B.** $f_s \geq 2f_{max}$, donde f_{max} es la frecuencia máxima **de la señal**. La opción C es el error de redacción típico.

17. **B.** "Incrementar resolución: mejora pero no elimina el problema", con las limitaciones de frame buffer, fps y costo (Unidad II-2 p.26).

18. **A.** Supersampling = postfiltering (subdivide y cuenta); area sampling = prefiltering (calcula el área directamente, **sin cálculos subpíxel**). La opción B invierte los nombres.

19. **C.** $m_{22} = 4$ y $\sum m_{ij} = 16$, entonces $w_{22} = 4/16 = 1/4$ (Unidad II-2 p.30). La opción B olvida que los pesos no son todos 1.

Preguntas de desarrollo posibles — Unidad II, parte 2

1. Explique cómo funcionan los algoritmos de relleno de áreas: relleno de contorno y relleno por difusión.
2. Explique las dos reglas de determinación interior/exterior e indique en qué caso producen resultados distintos.
3. Describa el algoritmo de scanline para polígonos, indicando el orden de los pasos y cómo se manejan los casos especiales.
4. Explique qué es la coherencia y cómo se aprovecha en las optimizaciones del algoritmo de relleno.
5. Explique qué son la GET y la AET, qué datos almacena cada una y cómo se utilizan.
6. Explique qué es el aliasing, cuál es su causa y por qué no puede eliminarse aumentando la resolución.
7. Describa las tres técnicas de antialiasing, indicando en qué se diferencian.
8. Compare el supersampling con el area sampling.

Guía de respuestas — Unidad II, parte 2

1. Contexto (figuras irregulares, no polígonos; punto semilla; balde de pintura); conectividad 4 y 8; boundary fill con su **doble condición** y por qué cada una; flood fill con su condición y por qué sirve para bordes multicolor; ventajas y desventajas de cada uno; optimización por spans; cerrar con la diferencia en la **condición de detención**.
 2. Par-impar: rayo, contar cruces, impar = interior. Winding: rayo, acumular ± 1 según el signo de la componente z del producto cruz $\hat{L} \times \hat{S}$, $\neq 0$ = interior. Diferencia en auto-intersectados, con el ejemplo de la zona rodeada dos veces.
 3. Los cuatro pasos del planteo; el cálculo de intersecciones; los dos casos especiales (vértice compartido con extremo local vs continuación, y lados horizontales) con su solución; las dos optimizaciones; el bucle de 6 pasos en orden.
 4. Definición de coherencia; los tres tipos (de la escena, espacial, de línea de scan/bordes); cómo la coherencia de bordes habilita el cálculo incremental $x_{k+1} = x_k + 1/m$ y cómo la coherencia justifica reconstruir lados en vez de recalcular intersecciones.
 5. GET: buckets por y_{min} , bucket sort, datos mínimos de reconstrucción (y_{max} , $x(y_{min})$, $1/m$), ordenados por x_{min} . AET: los lados que corta la scanline activa, ordenados por x actual. Cómo se mueven lados de una a otra en el bucle.
 6. Definición (pérdida de información por submuestreo); artifacts concretos; causa (muestreo discreto de modelos continuos); Nyquist con las dos formas; la limitación fundamental (un borde nítido tiene frecuencias arbitrariamente altas); por qué el hardware no alcanza (tamaño del frame buffer, 60 fps, costo).
 7. Supersampling/postfiltering con su metodología de 4 pasos; area sampling/prefiltering con la fórmula de cobertura; filtrado con la integral y los tres tipos de filtro; los cuatro criterios de selección.
 8. Tabla o contraste: cuándo se aplica el filtrado (post vs pre), si subdivide o no, cómo obtiene la cobertura (conteo vs cálculo analítico), costo. Cerrar con la diferencia fundamental.
-

UNIDAD V — Modelado de objetos

Prioridad ALTA. No apareció en el parcial anterior, pero es una unidad de **formato naturalmente comparativo** (cuatro métodos con ventajas y desventajas), y el profesor ya demostró que usa preguntas de comparación.

5.1 Qué es un modelo

Definición

Modelo: representación real o abstracta que captura las **características sobresalientes** (dato y comportamiento) de un objeto o fenómeno que está siendo modelado. — *Unidad V p.3*

"Modelar es copiar con complejidad."

Tipos de modelo

Tipo	Característica	Sub-tipos
Modelo real	Tiene geometría intrínseca	Representación física: objetos reales · Representación no física: funciones matemáticas (ej. datos del tiempo)
Modelo abstracto	No tiene geometría intrínseca, pero puede visualizarse	Por su disposición: organización de una compañía, diagramas · Cuantitativo: gráficos de ventas de stock

[NOTA DE CLASE] (*Clase 7 p.4*) — Sobre por qué hay que reducir el modelo: *"Nosotros no podemos, por ejemplo, modelar el clima, ya que no tenemos los datos suficientes... como no tengo la capacidad de cálculo del mundo, entonces lo reduzco para que para mi objetivo tenga sentido. La idea no es copiar todo sino saber qué copiar y qué no."*

5.2 Las tres estrategias para modelar

Estrategia	En qué consiste
1. Descomposición	Modelar objetos complejos por medio de objetos más simples : reducir a un conjunto de objetos que tienen descripción sencilla (primitivas)
2. Organización	Las formas primitivas deben estructurarse para establecer las relaciones de composición: establecer una jerarquía de componentes geométricos
3. Composición	Las formas primitivas deben ser "manipuladas" matemáticamente (transformadas) para ensamblar el objeto final

[NOTA DE CLASE] (*Clase 7 p.4-5*) — El profesor usó el ejemplo de un **tornillo**: se descompone en cabeza, tronco y punta; se organiza como un **árbol** (raíz = tornillo, que tiene cuerpo y cabeza; el cuerpo se subdivide en tronco y punta, que son las hojas); se compone generando los modelos geométricos individuales y adjuntándolos. *"Las funciones que aparecen en línea azul son funciones de transformación."*

Relación con otros conceptos: estas tres estrategias son exactamente lo que se hace en el Práctico 04 con la aeronave: **despiece** (descomposición), **jerarquía avión→ala→alerón** (organización) y **matrices de modelo** (composición). Y la etapa de composición es toda la Unidad VI.

5.3 Representaciones geométricas

El modelo más simple disponible es el **punto**, representado por una coordenada espacial (x, y, z) . A partir de él se construyen: **polilíneas**, **poliedros**, **funciones explícitas** y **funciones paramétricas**.

Según las dimensiones de los elementos de diseño:

Elementos / Modelo	Resultado
Elementos 1D / Modelo 2D	Dibujo lineal
Elementos 1D / Modelo 3D	Malla de alambre (<i>wireframe</i>)
Elementos 2D / Modelo 3D	Superficies

De qué depende el método de representación

El método a usar depende de: **la naturaleza del objeto** · **la técnica utilizada para modelar la apariencia** · **la aplicación**.

Y esas características, junto con la representación elegida, **determinan**:

- La **estructura de datos**, la forma de los algoritmos de procesamiento y el diseño de los programas en hardware.
- El **costo del procesamiento** del objeto a lo largo del pipeline 3D.
- La **apariencia final** del objeto: algunas formas se aproximan mejor que otras.
- La **facilidad de edición** de la forma del objeto.

Criterios de clasificación: si modelan la **superficie** o el **volumen** total del objeto · si modelan por representación **aproximada** o **exacta**.

5.4 Los cuatro métodos de representación

Método	Qué modela	¿Exacto?	Idea central
Poligonal	Superficie	Aproximado	Los objetos son aproximados por medio de una malla de facetas poligonales planas
Parches paramétricos bicúbicos	Superficie	Aproximado / interpolado	Similar a la malla poligonal, excepto que los polígonos individuales son superficies curvas (cuadriláteros curvos)
Geometría sólida constructiva (CSG)	Volumen	Exacto (dentro de los límites de las formas específicas)	El objeto surge de una combinación de formas elementales sólidas o primitivas geométricas
Subdivisión del espacio	Volumen	Aproximado (discreto)	Divide el espacio del objeto en cubos elementales conocidos como voxels

5.4.1 Representación poligonal

Cómo funciona. La superficie del objeto es **aproximada por medio de una secuencia de polígonos**, estructurados jerárquicamente: los polígonos se agrupan en **caras**, las caras en **superficies** y las superficies en **objetos**. Permite representar con gran fidelidad un objeto con cualquier forma.

Se la denomina usualmente "**representación de contorno**" porque es una **descripción geométrica y topológica de la superficie** de un objeto.

[NOTA DE CLASE] (Clase 7 p.9) — "Si queremos calidad = muchos vértices. Si queremos velocidad = menos vértices."

Las cuatro estrategias de modelado. Las estrategias son "básicamente fuerza bruta": se trata de **fijar puntos (vértices)** mediante alguna técnica, asegurando una aproximación adecuada del objeto.

Estrategia	Cómo funciona
1. Modelado manual interactivo	Software especializado para el manejo de objetos tridimensionales (SketchUp, Blender). El proceso es interactivo : el usuario edita en tiempo real. Gran cantidad de operaciones disponibles
2. Modelado manual con digitalizador 3D	El operador usa su experiencia y buen juicio para la captura de puntos del objeto, que serán los vértices de los polígonos. Estrategia común: dibujar una malla sobre la superficie del objeto
3. Modelado automático (explorador láser)	Asegura una malla de alta resolución y muy confiable . El objeto se coloca en una base rotativa interceptando la ruta del rayo; el láser retorna un conjunto de curvas de contorno (planos paralelos); se requiere un algoritmo para unir pares de puntos de contornos consecutivos y convertirlos en polígonos
4. Modelado matemático	El objeto es generado por medio de una descripción matemática que es barrida a lo largo de una ruta . La resolución poligonal se controla fácilmente por el algoritmo. Las superficies de revolución son un caso especial

[NOTA DE CLASE] (Clase 7 p.9-10) — Dos observaciones del profesor: sobre la estrategia 1, "no es viable porque es muy complicado". Sobre el digitalizador, "genera una nube de puntos (puntos por todos lados), entonces genera ruido. Y depende la calidad del aparato va a aproximar mejor o no la superficie." Este último punto **no está en las filminas**.

Modelado por barrido. Se especifica:

- **Sección de barrido:** representación 2D (línea o área).
- **Trayectoria:** línea en el espacio y distancia a recorrer (desplazamiento).

Se puede **variar la forma y el tamaño** de la sección de barrido, y **variar la orientación** de la sección a medida que se desplaza.

Consideraciones: orientación de la sección a lo largo de la trayectoria · intervalo de puntos en dirección de la sección y en dirección de la trayectoria.

Caso particular: barrido por **revolución alrededor de un eje**.

Ventajas y desventajas:

Ventajas	Desventajas
El proceso de creación es simple	Difícil representar superficies curvas: hay que ajustar el tamaño de los polígonos en función de la curvatura local
Los polígonos pueden ser tratados como entidades independientes	Puede implicar gran cantidad de tiempo y costo para objetos complejos
Ampliamente utilizado por la eficiencia de los algoritmos	Problemas de escalabilidad: a gran distancia, objetos con muchos polígonos tienden a verse como uno solo
	Se deben almacenar las relaciones topológicas en una base asociada

5.4.2 Parches paramétricos bicúbicos

Generalidades:

- **4 vértices unidos por 4 lados** que constituyen **curvas bicúbicas**. El interior del parche es una **superficie curva bicúbica**.
- Usualmente el parche se define como $Q(u, v)$, donde u, v son **parámetros** y Q es un **polinomio cúbico**.
- La superficie del objeto puede ser **interpolada o aproximada** por la malla del parche.
- Un tamaño conveniente es de **16 puntos 3D**, denominados **puntos de control**.
- Estos puntos de control son utilizados por **funciones polinomiales predefinidas** denominadas **funciones bases**.

Cómo se modela con este método:

- **Trabajar con un único parche** como objeto en sí mismo: directo, aunque no sencillo.
- **Crear una malla de parches a partir del objeto real:** se establecen puntos sobre la superficie del objeto y luego el parche se **regenera** a partir de ellos.
- **Utilizar técnicas de barrido de parches:** el parche paramétrico es considerado como la **sección que debe ser barrida** a lo largo de una trayectoria.

Ejemplo conceptual — la Tetera de Utah (*Unidad V p.26*):

Representación	Datos
Malla de 32 parches × 16 puntos de control, compartiendo 12 puntos para asegurar la continuidad entre parches	306 vértices
Malla poligonal "razonable" de la misma tetera	2048 vértices

→ Los parches son una representación **mucho más compacta** de superficies curvas.

Ventajas y desventajas:

Ventajas	Desventajas
La representación es " fluida ", más aún si los puntos de control se ajustan por software	Modelar la estructura de datos es dificultoso
Permiten obtener información de propiedades de masa : volumen, superficie del área, momentos de inercia	Es difícil mantener la integridad del modelo
	Demanda grandes volúmenes de memoria , con penalidades por tiempo de transferencia y uso de memoria

5.4.3 Geometría sólida constructiva (CSG)

Cómo funciona:

- Utiliza **objetos sólidos elementales** denominados **primitivas geométricas**, combinadas entre sí a través de operaciones.
- Las operaciones son un conjunto de **operadores booleanos** y **transformaciones lineales**.
- Usualmente los objetos modelados son partes que serán **manufacturadas por fundición, mecánicamente o por extrusión**.
- Es una representación **volumétrica**.
- Es una representación **exacta** dentro de los límites de las formas específicas.

[NOTA DE CLASE] (Clase 9 p.1) — "Está muy asociado a los problemas de diseño mecánico. La idea es que partimos de sólidos básicos o primitivos y a través de transformaciones booleanas generar el objeto. Se arma una estructura de árbol donde están las operaciones booleanas que hay que hacer." **La estructura de árbol no está mencionada en las filminas oficiales**: es un aporte de tus apuntes.

Ventajas y desventajas:

Ventajas	Desventajas
Es simple e intuitiva	Se requiere mucho tiempo computacional para generar una imagen del modelo
La representación define tanto la forma del objeto como su historia de modelado	Las operaciones son globales : afectan al sólido en su totalidad
Los cambios en el modelo implican cambios triviales en el proceso de modelado	Las operaciones locales no son simples de implementar (ej.: radios de unión entre superficies)

5.4.4 Técnicas de subdivisión del espacio (voxels y octrees)

Contexto:

- Usado para representar **sólidos con sus interiores**.
- Algunos métodos de recolección de datos generan información que forma un sólido.
- Algunas aplicaciones **requieren** el uso de sólidos: **medicina**, aplicaciones **CAD/CAM**.

Estrategia: dividir el espacio en **celdas volumétricas denominadas voxels** (*volumetric pixels*).

- Se obtiene un **arreglo regular de muestras tridimensionales**, como una imagen.

- Cada celda **almacena las propiedades del objeto sólido**: color, densidad, temperatura, e información relevante para el modelo.

Extracción de una superficie a partir de datos espaciales. Metodología: **interpolan los datos de las celdas intersectadas por la superficie.**

[NOTA DE CLASE] (Clase 9 p.3) — "Me da la información dentro del volumen."

Octrees — procedimiento:

1. Se parte de un **volumen que encierra completamente al objeto.**
2. Se **divide la celda por la mitad en las tres direcciones espaciales** (x, y, z).
3. Se obtienen **8 celdas volumétricas** en las que se verifica cuáles están ocupadas y cuáles no.
4. Aquellas que **contengan partes del objeto se vuelven a subdividir** por mitades.
5. Se continúa hasta el **nivel de refinamiento deseado.**

El modelo resultante se almacena en una **estructura de datos tipo árbol en donde cada nodo puede tener 8 hijos.**

[NOTA DE CLASE] (Clase 9 p.3) — "Es una estrategia para subdividir el espacio que ocupa el objeto y poder determinar qué zonas están ocupadas y cuáles no."

Ventajas y desventajas:

Ventajas	Desventajas
Representación simple e intuitiva	Representación aproximada (discreta)
Todos los objetos tienen la misma complejidad	Proceso de visualización costoso
Representación adecuada para algunos sistemas de adquisición de datos	Grandes requerimientos de memoria de almacenamiento
Puede usarse como representación intermedia : establecer la distribución de los objetos en la escena	

Este esquema **se utiliza solamente en contextos donde los beneficios superan su costo**, por ejemplo la visualización de volúmenes en imágenes médicas.

5.5 Comparación de los cuatro métodos

Característica	Poligonal	Parches bicúbicos	CSG	Subdivisión (voxels)
Qué modela	Superficie	Superficie	Volumen	Volumen
Exactitud	Aproximada	Aproximada/interpolada, "fluida"	Exacta (en sus primitivas)	Aproximada (discreta)
Elemento básico	Faceta poligonal plana	Cuadrilátero curvo bicúbico $Q(u, v)$	Primitiva sólida + operador booleano	Voxel
Estructura de datos	Malla + relaciones topológicas	16 puntos de control por parche	Árbol de operaciones	Árbol de 8 hijos por nodo (octree)
Fortaleza	Simple, eficiente, es lo que dibuja la GPU	Compacta para superficies curvas; da propiedades de masa	Intuitiva; guarda la historia de modelado	Todos los objetos tienen la misma complejidad; representa interiores
Debilidad	Mala para curvas; escalabilidad	Estructura de datos difícil; mucha memoria	Costosa de visualizar; operaciones globales	Aproximada; visualización costosa; mucha memoria
Aplicación típica	Tiempo real, videojuegos, GPU	Diseño de superficies suaves	Diseño mecánico / CAD	Imágenes médicas, CAD/CAM

Diferencia fundamental

El corte principal es **superficie vs volumen**: la representación poligonal y los parches describen **sólo la cáscara** del objeto, mientras que CSG y la subdivisión del espacio describen **el objeto lleno**, con su interior. El segundo corte es **exacto vs aproximado**: CSG es exacto dentro de sus primitivas y además conserva la **historia de modelado**, mientras que las otras tres aproximan la forma, ya sea con facetas planas, con parches curvos o con celdas discretas.

Cómo respondería esto en un parcial

Los métodos de representación de objetos 3D se clasifican según **si modelan la superficie o el volumen** del objeto y según **si la representación es exacta o aproximada**.

La **representación poligonal** aproxima la superficie mediante una secuencia de polígonos estructurados jerárquicamente: polígonos → caras → superficies → objetos. Se la llama representación de contorno porque describe geométrica y topológicamente la superficie. Sus ventajas son la simplicidad del proceso de creación, que los polígonos pueden tratarse como entidades independientes y la eficiencia de los algoritmos asociados. Sus desventajas son la dificultad para representar superficies curvas, el costo de modelar objetos complejos, los problemas de escalabilidad y la necesidad de almacenar las relaciones topológicas.

Los **parches paramétricos bicúbicos** son similares a la malla poligonal, pero cada "polígono" es un cuadrilátero curvo definido como $Q(u, v)$ con u, v parámetros y Q un polinomio cúbico, a partir de 16 puntos de control usados por funciones base. Permiten una representación fluida y obtener propiedades de masa como volumen, área y momentos de inercia; pero la estructura de datos es dificultosa, es difícil mantener la integridad del modelo y demanda gran cantidad de memoria. El ejemplo de la tetera de Utah lo ilustra: 32 parches requieren 306 vértices frente a 2048 de una malla poligonal razonable.

La **geometría sólida constructiva** combina primitivas geométricas sólidas mediante operadores booleanos y transformaciones lineales, almacenándose como un árbol. Es volumétrica y exacta dentro de sus formas específicas, y se usa típicamente en diseño mecánico para piezas manufacturadas. Es simple e intuitiva y define tanto la forma como la historia de modelado, de modo que los cambios son triviales; pero requiere mucho tiempo computacional para generar la imagen y sus operaciones son globales, lo que dificulta las operaciones locales como los radios de unión.

Las **técnicas de subdivisión del espacio** dividen el espacio en celdas volumétricas llamadas voxels, cada una almacenando propiedades como color, densidad o temperatura. Para refinar el espacio se usan **octrees**: se parte de un volumen que encierra al objeto, se lo divide por la mitad en las tres direcciones obteniendo 8 celdas, se verifica cuáles están ocupadas y se subdividen sólo esas, hasta el nivel de refinamiento deseado, almacenándose en un árbol de 8 hijos por nodo. Su ventaja es que la representación es simple y todos los objetos tienen la misma complejidad; sus desventajas son que es discreta, costosa de visualizar y muy demandante de memoria. Se usa donde el beneficio supera el costo, como en visualización de volúmenes médicos.

Multiple Choice de práctica — Unidad V

1. Un modelo, según la definición de la materia, captura:

A) Todos los detalles del objeto representado B) Las características sobresalientes —dato y comportamiento— del objeto o fenómeno modelado C) Únicamente la geometría exterior del objeto D) La secuencia de operaciones booleanas que lo generaron

2. Un organigrama de una empresa es un ejemplo de:

A) Modelo real con representación física B) Modelo real con representación no física C) Modelo abstracto, visualizable por su disposición D) Modelo poligonal de contorno

3. La estrategia de **organización** en el modelado consiste en:

A) Reducir el objeto a un conjunto de primitivas simples B) Estructurar las formas primitivas estableciendo una jerarquía de componentes geométricos C) Transformar matemáticamente las primitivas para ensamblar el objeto D) Elegir entre representación superficial o volumétrica

4. La representación poligonal se denomina también **representación de contorno** porque:

A) Sólo almacena el contorno visible desde la cámara B) Es una descripción geométrica y topológica de la superficie del objeto C) Utiliza únicamente líneas, sin caras D) Los polígonos se generan por barrido de un contorno 2D

5. ¿Cuál de estas es una **desventaja** de la representación poligonal?

A) No permite representar objetos de forma arbitraria B) A gran distancia, los objetos con muchos polígonos tienden a verse como un solo polígono C) No puede implementarse eficientemente en hardware gráfico D) No permite tratar los polígonos como entidades independientes

6. En el modelado matemático por **barrido** hay que especificar:

A) Los 16 puntos de control y las funciones base B) La sección de barrido (representación 2D) y la trayectoria C) El árbol de operaciones booleanas D) El nivel de refinamiento del octree

7. Las **superficies de revolución** son:

A) Un método de representación independiente de los cuatro estudiados B) Un caso particular del modelado matemático por barrido, alrededor de un eje C) Una técnica exclusiva de los parches bicúbicos D) El resultado de aplicar operaciones booleanas a un cilindro

8. En un parche paramétrico bicúbico $Q(u, v)$:

A) u y v son coordenadas del mundo y Q una malla poligonal B) u y v son parámetros y Q es un polinomio cúbico C) u y v son los semiejes del parche D) Q representa el volumen encerrado por el parche

9. La comparación de la tetera de Utah (306 vértices con parches vs 2048 con malla poligonal) demuestra que:

A) Los parches son más rápidos de renderizar B) Los parches son una representación mucho más compacta de superficies curvas C) La malla poligonal es más exacta D) Los parches permiten representar interiores

10. ¿Cuál de las siguientes es una **ventaja exclusiva** de los parches paramétricos bicúbicos frente a la malla poligonal?

A) Permiten obtener propiedades de masa como volumen, área y momentos de inercia B) Tratan cada elemento como entidad independiente C) Son más simples de crear D) Requieren menos memoria en todos los casos

11. La **geometría sólida constructiva** se caracteriza por:

A) Ser una representación de superficie aproximada B) Combinar primitivas sólidas mediante operadores booleanos y transformaciones lineales, siendo volumétrica y exacta C) Dividir el espacio en celdas regulares llamadas voxels D) Definir cuadriláteros curvos a partir de puntos de control

12. Una **desventaja** característica de la CSG es que:

A) No conserva la historia de modelado B) Las operaciones son globales y las locales, como los radios de unión, no son simples de implementar C) No permite representar volúmenes D) Requiere digitalizadores 3D para capturar los puntos

13. En la construcción de un **octree**, cada subdivisión genera:

A) 4 celdas, dividiendo por la mitad en x e y B) 8 celdas, dividiendo por la mitad en las tres direcciones espaciales C) 16 celdas, una por punto de control D) Un número variable de celdas según la forma del objeto

14. ¿Cuál es una **ventaja** de la subdivisión del espacio mediante voxels?

A) Es una representación exacta del objeto B) Todos los objetos tienen la misma complejidad C) El proceso de visualización es muy económico D) Requiere poca memoria de almacenamiento

15. Los métodos que modelan el **volumen** del objeto, y no sólo su superficie, son:

A) Poligonal y parches bicúbicos B) CSG y subdivisión del espacio C) Parches bicúbicos y CSG D) Poligonal y subdivisión del espacio

Respuestas y explicación — Unidad V

1. B. Definición literal (*Unidad V p.3*). La opción A contradice la idea de "modelar es copiar con complejidad" y la nota de clase sobre saber qué copiar y qué no.

2. C. Un organigrama **no tiene geometría intrínseca** pero puede visualizarse por su disposición: es el ejemplo exacto de la filmina.

3. B. La opción A es **descomposición** y la C es **composición**: las tres se confunden con facilidad.

4. B. (*Unidad V p.13*). La palabra clave es **topológica**: no sólo dónde están los vértices, sino cómo se conectan.

5. B. Es el problema de escalabilidad que enuncia la filmina (*Unidad V p.23*). Las opciones A, C y D niegan ventajas reales del método.

6. B. Sección de barrido (línea o área 2D) y trayectoria (línea en el espacio y distancia a recorrer) (*Unidad V p.21*).

7. B. "Las superficies de revolución son un caso especial de este método" (*Unidad V p.20*).

8. B. (*Unidad V p.24*). Los parches modelan **superficie**, no volumen, lo que descarta D.

9. B. La conclusión es sobre **compacidad de la representación**, no sobre velocidad ni exactitud.

10. A. Es la ventaja que la filmina destaca específicamente (*Unidad V p.27*). La opción B y C son ventajas de la **poligonal**.

11. B. (*Unidad V p.28*). La opción D describe los parches y la C, los voxels.

12. B. (*Unidad V p.30*). La opción A invierte una **ventaja**: la CSG sí conserva la historia de modelado.

13. B. Por la mitad en x, y y z $\rightarrow 2^3 = 8$ celdas, y de ahí el nombre "octree" (*Unidad V p.33*).

14. B. (*Unidad V p.35*). Las opciones A, C y D son justamente sus **desventajas**.

15. B. CSG y subdivisión son volumétricas; poligonal y parches modelan superficie.

Preguntas de desarrollo posibles — Unidad V

1. Explique qué es un modelo en computación gráfica y describa las tres estrategias utilizadas para modelar un objeto complejo.
2. Compare los cuatro métodos de representación de objetos 3D indicando, para cada uno, qué modela, su exactitud, y sus ventajas y desventajas.
3. Describa la representación poligonal e indique las estrategias disponibles para generar un modelo poligonal.

4. Explique en qué consiste el modelado matemático por barrido y qué debe especificarse para aplicarlo.
5. Describa la representación por geometría sólida constructiva, indicando cómo se almacena el modelo.
6. Explique las técnicas de subdivisión del espacio y describa el procedimiento de construcción de un octree.
7. Indique de qué depende la elección de un método de representación y qué aspectos determina esa elección.

Guía de respuestas — Unidad V

1. Definición de modelo; la frase "modelar es copiar con complejidad"; modelo real (física y no física) vs abstracto (disposición y cuantitativo) con ejemplos; las tres estrategias con su definición y, si se puede, el ejemplo del tornillo.
 2. La tabla comparativa completa. Cerrar con los dos cortes conceptuales: **superficie vs volumen** y **exacto vs aproximado**.
 3. Aproximación por secuencia de polígonos; la jerarquía polígonos→caras→superficies→objetos; el nombre "representación de contorno" y su justificación; las cuatro estrategias con una frase cada una; ventajas y desventajas.
 4. El objeto se genera por una descripción matemática barrida a lo largo de una ruta; se especifica sección de barrido y trayectoria; se puede variar forma, tamaño y orientación de la sección; las dos consideraciones; el caso particular de revolución; la ventaja de que la resolución poligonal se controla desde el algoritmo.
 5. Primitivas geométricas sólidas + operadores booleanos + transformaciones lineales; representación volumétrica y exacta; se almacena como **árbol** (hojas = primitivas, nodos = operaciones); aplicación en diseño mecánico y piezas manufacturadas; ventajas y desventajas.
 6. Voxels como celdas volumétricas que almacenan propiedades (color, densidad, temperatura); arreglo regular de muestras 3D; extracción de superficie por interpolación de las celdas intersectadas; los cinco pasos del octree; almacenamiento en árbol de 8 hijos; ventajas, desventajas y el contexto de uso (imágenes médicas).
 7. Depende de la naturaleza del objeto, la técnica para modelar la apariencia y la aplicación. Determina: estructura de datos y algoritmos, costo de procesamiento en el pipeline, apariencia final y facilidad de edición. Mencionar los dos criterios de clasificación.
-

UNIDAD VI — Transformaciones geométricas

Prioridad MUY ALTA. Fue la **pregunta 5** del parcial anterior, con un formato muy específico: "dadas matrices de transformación en coordenadas homogéneas, indicar qué tipo de transformación representa cada una" y "explicar qué efecto producirá una composición $D = A \cdot B \cdot C$ ".

Es decir: no pidió **calcular**, pidió **reconocer e interpretar**. Esta unidad está organizada con ese criterio.

6.1 Definición y para qué sirve

Transformaciones geométricas: operaciones que se aplican a la **descripción geométrica** de un objeto para cambiar su **posición, orientación o tamaño**. También llamadas **transformaciones de modelado**. — *Unidad VI p.3*

Son fundamentales en: animación por computadora · diseño asistido por computadora (CAD) · rutinas de visualización · modelado de escenas complejas.

Relación con otros conceptos: son la etapa de **composición** de la Unidad V (ensamblar el objeto a partir de primitivas) y, en la Unidad VII, son también la herramienta con la que se construye la **transformación de normalización** de la cámara.

6.2 Preliminares matemáticos

Álgebra vectorial

Operación	Definición general	En $\mathbb{R}^2$
Suma	$w = u + v$	$w = (u_x + v_x, u_y + v_y)$
Producto por un escalar	$w = k u$	$w = (k u_x, k u_y)$
Producto punto o interno	$u \cdot v = u v \cos \theta$	$u \cdot v = u_x v_x + u_y v_y$
Producto vectorial	$u \times v = u v \sin \theta \hat{n}$	$u \times v = (u_x v_y - v_x u_y) \hat{k}$

Matrices

$$M = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad M \cdot P = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

Propiedades — las tres que hay que saber:

- **Asociativa:** $(AB)C = A(BC)$
- **NO conmutativa:** $AB \neq BA \leftarrow$ es la clave de la pregunta 5
- **Elemento identidad:** $I = M^{-1} \cdot M$

6.3 Transformaciones básicas 2D

Traslación

Acción: desplazar un punto distancias t_x y t_y .

$$x' = x + t_x \quad y' = y + t_y \quad \text{Forma vectorial: } P' = P + T$$

Características:

- **Transformación de cuerpo rígido.**
- **Todos los puntos se mueven la misma distancia.**
- **Preserva formas y tamaños.**

Rotación sobre el origen

Acción: rotar (sobre el origen) un punto un ángulo θ .

$$x' = x \cos \theta - y \sin \theta \quad y' = x \sin \theta + y \cos \theta$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Propiedades:

- $\theta > 0$: **rotación antihoraria.**
- $\theta < 0$: **rotación horaria.**
- **Transformación de cuerpo rígido.**
- **Preserva distancias y ángulos.**

Escalado respecto al origen

Acción: escalar (respecto al origen) un punto en s_x y s_y .

$$x' = s_x \cdot x \quad y' = s_y \cdot y \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \rightarrow P' = [S] \cdot P$$

Tipos de escalado:

Condición	Efecto
$s_x = s_y$	Escalado uniforme
$s_x \neq s_y$	Escalado diferencial
$s_x, s_y > 1$	Ampliación
$s_x, s_y < 1$	Reducción
$s_x, s_y < 0$	Reflexión y escalado

⚠ **Atención:** el escalado **también mueve el objeto** si no está centrado en el origen, porque **todas** las coordenadas se multiplican, incluidas las de posición. Por eso existe la versión con punto fijo. **El escalado no es una transformación de cuerpo rígido.**

Transformaciones respecto a un punto arbitrario

Rotación sobre un punto arbitrario. Secuencia de pasos:

1. **Trasladar** el objeto para que el punto de rotación esté en el origen.

2. **Rotar** sobre el origen.

3. **Trasladar** el objeto a la posición original.

$$x' = x_r + (x - x_r) \cos \theta - (y - y_r) \sin \theta$$

$$y' = y_r + (x - x_r) \sin \theta + (y - y_r) \cos \theta$$

$$P' = P_r + [R(\theta)] \cdot (P - P_r)$$

Escalado respecto a un punto fijo. Misma secuencia con (x_f, y_f) :

$$x' = x_f + (x - x_f)s_x = x s_x + x_f(1 - s_x)$$

$$y' = y_f + (y - y_f)s_y = y s_y + y_f(1 - s_y)$$

$$P' = P_f + [S(s_x, s_y)] \cdot (P - P_f)$$

El patrón universal: "ir al origen, hacer, volver". Vale para toda transformación respecto a un punto que no sea el origen, tanto en 2D como en 3D.

6.4 Coordenadas homogéneas

Motivación — por qué son necesarias

Las transformaciones geométricas se pueden plantear como:

$$P' = M_1 \cdot P + Q$$

- Las **matrices operan con transformaciones proporcionales** (multiplicación).
- Los **términos de traslación requieren una suma adicional**.

Desventajas de esto:

- La **composición de transformaciones es compleja**.
- **No hay una forma unificada de representación**.

Solución: usar **coordenadas homogéneas** para plantear las transformaciones.

Definición

Un punto 2D (x, y) se representa como un **vector de tres componentes** de la forma (x_h, y_h, h) donde:

$$x = \frac{x_h}{h}, \quad y = \frac{y_h}{h}$$

Elección conveniente: $h = 1$ — versión "normalizada" donde las coordenadas homogéneas **coinciden numéricamente** con las cartesianas.

$$\text{Punto } (x, y) \longrightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \text{En 3D: } (x, y, z) \rightarrow (x, y, z, 1)$$

Ventajas:

- **Todas** las transformaciones se pueden plantear como **multiplicación matricial**.
- Matrices 3×3 **uniformes** (en 3D, 4×4).
- **Simplifica el proceso de composición**.

⚠ **Cuándo $h \neq 1$:** en el modelado siempre se usa $h = 1$, pero **en la proyección en perspectiva h deja de valer 1** y hay que **homogeneizar**, es decir, dividir por h . Ése es el " $\div w$ " del pipeline (Unidad VII).

6.5 Cómo reconocer una matriz de transformación

Esta sección responde directamente al formato de la pregunta 5: "indicar qué tipo de transformación representa cada una".

Las tres matrices básicas 2D en coordenadas homogéneas

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Uso: } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Tabla de reconocimiento

Cómo la reconozco	Transformación	Parámetros	Efecto sobre un punto	Posición	Orientación	Tamaño
Diagonal de unos , con valores en la última columna	Traslación	$t_x, t_y =$ desplazamiento en cada eje	Suma un vector constante	Cambia	Se preserva	Se preserva
Bloque 2×2 con cos y sin, antisimétrico (– sin arriba a la derecha)	Rotación	$\theta =$ ángulo; > 0 antihorario	Gira alrededor del origen	Cambia (salvo el origen)	Cambia	Se preserva
Diagonal con valores distintos de 1, última columna en cero	Escalado	$s_x, s_y =$ factores por eje	Multiplica cada coordenada	Cambia si no está en el origen	Se preserva	Cambia
Diagonal con algún valor negativo (–1)	Reflexión (caso de escalado)	Signo negativo en el eje invertido	Espeja respecto de un eje o plano	Cambia	Se invierte	Se preserva si $ s = 1$
Diagonal con cos / sin y última columna no nula	Composición rotación + traslación	—	Rota y desplaza	Cambia	Cambia	Se preserva

Pistas rápidas para identificar en un examen:

- ¿La última columna (o cuarta en 3D) tiene valores distintos de cero? → hay **traslación** involucrada.
- ¿Hay senos y cosenos? → hay **rotación**. Si – sin está arriba a la derecha es R_z (o rotación 2D); si está abajo a la izquierda es R_y .
- ¿Es diagonal con valores $\neq 1$? → **escalado**. Si hay un negativo, además hay **reflexión**.
- ¿La última fila NO es (0 0 1) o (0 0 0 1)? → **no es una transformación afín**: es una **proyección en perspectiva**, que va a requerir homogeneizar.
- ¿La matriz es la identidad salvo un bloque? → la transformación sólo afecta a esos ejes.

Transformaciones inversas

$$T^{-1} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix} \quad R^{-1} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad S^{-1} = \begin{bmatrix} 1/s_x & 0 & 0 \\ 0 & 1/s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Verificación: $M \cdot M^{-1} = I$

Propiedad clave: $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$ — la inversa de una rotación es su **transpuesta**. Esta propiedad es **decisiva** en la Unidad VII.

⚠ **Error típico:** escribir S^{-1} con $-s_x$ en lugar de $1/s_x$.

6.6 Composición de transformaciones

Éste es el corazón de la pregunta 5.

La regla

$$P' = M_n \cdot M_{n-1} \cdots M_2 \cdot M_1 \cdot P$$
$$M = M_n \cdot M_{n-1} \cdots M_2 \cdot M_1 \Rightarrow P' = M \cdot P$$

Propiedades importantes (Unidad VI p.15):

- **Asociativa:** $(M_3 \cdot M_2) \cdot M_1 = M_3 \cdot (M_2 \cdot M_1)$
- **NO conmutativa:** el **orden de aplicación importa**.
- **Se lee de derecha a izquierda.**

Qué significa $D = A \cdot B \cdot C$

La convención del profesor. En *Unidad VI p.15* se establece que la composición **se lee de derecha a izquierda**, y en la secuencia $P' = M_n \cdots M_1 \cdot P$ la matriz M_1 —la más a la derecha— es la **primera** que se aplica.

Entonces, ante $D = A \cdot B \cdot C$ aplicada a un punto P :

$$D \cdot P = A \cdot (B \cdot (C \cdot P))$$

Orden de aplicación	Matriz	Posición en el producto
Primera en aplicarse	C	La más a la derecha
Segunda	B	En el medio
Última en aplicarse	A	La más a la izquierda

Interpretación geométrica: el punto P es transformado primero por C ; el resultado es transformado por B ; y ese resultado, por A . La matriz D resume las tres en una sola: se calcula **una vez** y se aplica a **todos los vértices del objeto**, que es la gran ventaja computacional.

Por qué no es conmutativa, y qué pasa si cambio el orden

La multiplicación de matrices no es conmutativa porque **cada matriz transforma el sistema en el que opera la siguiente**. Una rotación aplicada después de una traslación actúa sobre un objeto **que ya está desplazado**, y por lo tanto lo hace girar **alrededor de un punto distinto**.

El ejemplo canónico del material (*Práctico 04 p.12*):

Composición	Cómo se lee	Efecto geométrico
$M = T \cdot R$	Primero rota , después traslada	La pieza gira sobre sí misma y queda en su lugar
$M = R \cdot T$	Primero traslada , después rota	La pieza orbita alrededor del origen

Ninguno de los dos está mal. Producen resultados diferentes.

Explicación complementaria: si el objeto está en el origen, rotar y trasladar da lo mismo en un orden que en otro para la rotación — pero apenas el objeto se aleja del origen, la rotación posterior lo arrastra describiendo un arco. Ése es el contenido geométrico de la no conmutatividad.

Ejemplos desarrollados

Ejemplo 1 — Rotación sobre un punto arbitrario (*Unidad VI p.16*)

Secuencia: (1) T_1 trasladar a origen $(-x_r, -y_r)$ · (2) R rotar θ · (3) T_2 trasladar de vuelta (x_r, y_r) .

$$M = T_2 \cdot R \cdot T_1 = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$$

$$M = \begin{bmatrix} \cos \theta & -\sin \theta & x_r(1 - \cos \theta) + y_r \sin \theta \\ \sin \theta & \cos \theta & y_r(1 - \cos \theta) - x_r \sin \theta \\ 0 & 0 & 1 \end{bmatrix}$$

Ejemplo 2 — Escalado con punto fijo (*Unidad VI p.17*)

$$M = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f) = \begin{bmatrix} s_x & 0 & x_f(1 - s_x) \\ 0 & s_y & y_f(1 - s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

Ejemplo 3 — Escalado en direcciones arbitrarias (*Unidad VI p.18*)

Secuencia: (1) R_1 alinear ejes rotando θ · (2) S escalar (s_1, s_2) · (3) R_2 volver rotando $-\theta$.

$$M = R^{-1}(\theta) \cdot S(s_1, s_2) \cdot R(\theta) = \begin{bmatrix} s_1 \cos^2 \theta + s_2 \sin^2 \theta & (s_2 - s_1) \cos \theta \sin \theta & 0 \\ (s_2 - s_1) \cos \theta \sin \theta & s_1 \sin^2 \theta + s_2 \cos^2 \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Cómo respondería esto en un parcial

En coordenadas homogéneas, una matriz de **traslación** se reconoce porque tiene la **diagonal de unos** y los desplazamientos t_x, t_y en la **última columna**; desplaza todos los puntos la misma distancia, preservando forma, tamaño y orientación, por lo que es una transformación de cuerpo rígido. Una matriz de **rotación** se reconoce por el bloque de senos y cosenos con el $-\sin\theta$ arriba a la derecha; gira los puntos alrededor del origen un ángulo θ , antihorario si $\theta > 0$, y también es de cuerpo rígido porque preserva distancias y ángulos. Una matriz de **escalado** se reconoce porque es **diagonal** con valores distintos de uno y última columna nula; multiplica cada coordenada por su factor, modificando el tamaño y, si el objeto no está en el origen, **también su posición**. Si algún factor es negativo, además produce una **reflexión**.

Respecto de la composición $D = A \cdot B \cdot C$: según la convención utilizada, el producto **se lee de derecha a izquierda**, de modo que al aplicar $D \cdot P$ el punto es transformado **primero por C**, luego por **B** y **finalmente por A**. La matriz D resume las tres transformaciones en una sola, lo que permite calcularla una única vez y aplicarla a todos los vértices del objeto, con la consiguiente ventaja computacional.

Es fundamental notar que la multiplicación de matrices es **asociativa pero no conmutativa**: el orden de aplicación importa. Esto tiene una interpretación geométrica precisa: cada transformación actúa sobre el resultado de la anterior, de modo que una rotación aplicada después de una traslación gira al objeto **alrededor de un punto distinto** del que giraría si se aplicara antes. Por ejemplo, $T \cdot R$ hace que la pieza rote sobre sí misma y quede en su lugar, mientras que $R \cdot T$ hace que la pieza **orbite alrededor del origen**. Ninguno de los dos resultados es incorrecto: son transformaciones diferentes.

Esta propiedad es la que obliga a respetar el patrón "*trasladar al origen, transformar, volver*" cuando la transformación debe hacerse respecto de un punto que no es el origen, como en la rotación sobre un punto arbitrario o el escalado con punto fijo.

6.7 Transformaciones 3D

Extensión de 2D a 3D

- Las transformaciones 3D se pueden pensar como **extensión** de las 2D.
- Hay una **nueva coordenada** z en todas las operaciones.
- Coordenadas homogéneas: $(x, y, z, 1)$ en lugar de $(x, y, 1)$.
- Matrices de transformación pasan de 3×3 a 4×4 .

$$P' = T \cdot P = \begin{bmatrix} t_{11} & t_{12} & t_{13} & t_{14} \\ t_{21} & t_{22} & t_{23} & t_{24} \\ t_{31} & t_{32} & t_{33} & t_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Ventajas: unificación de transformaciones lineales y traslaciones · composición mediante multiplicación matricial · eficiencia computacional.

Sistemas de coordenadas en 3D

	Dextrógiro (<i>right-handed</i>)	Levógiro (<i>left-handed</i>)
Regla	Regla de la mano derecha	Regla de la mano izquierda
Uso	Sistema típico para descripciones geométricas	Utilizado para la representación en pantalla de una escena
Para qué	Modelar los objetos y el mundo	Describir la profundidad de objetos en escena
Ejemplo	Coordenadas del mundo en OpenGL	DirectX

Traslación 3D

$$T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow u' = T \cdot u$$

Propiedades: preserva formas y orientaciones. **Inversa:** $T^{-1} = T(-t_x, -t_y, -t_z)$.

Escalado 3D

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Tipos: **uniforme** si $s_x = s_y = s_z$ (preserva proporciones) · **no uniforme** con diferentes factores por eje.

Escalado respecto a punto fijo 3D:

$$M = T(x_f, y_f, z_f) \cdot S(s_x, s_y, s_z) \cdot T(-x_f, -y_f, -z_f) = \begin{bmatrix} s_x & 0 & 0 & x_f(1 - s_x) \\ 0 & s_y & 0 & y_f(1 - s_y) \\ 0 & 0 & s_z & z_f(1 - s_z) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

△ **ERRATA CONFIRMADA en Unidad VI p.24.** En la filmina, el elemento de la **fila 2, columna 3** aparece como **1**, cuando debe ser **0**. Lo verifiqué renderizando la página. Se deduce de la composición $T \cdot S \cdot T^{-1}$, cuyo bloque 3×3 es **diagonal**. **Si te piden esta matriz, escribí el 0.**

Rotaciones 3D básicas

Convención (Unidad VI p.25): rotación positiva = antihoraria mirando desde el extremo positivo del eje hacia el origen. Ayuda: usar la mano derecha para determinar el sentido positivo.

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{aligned} x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta \\ z' &= z \end{aligned}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{aligned} y' &= y \cos \theta - z \sin \theta \\ z' &= y \sin \theta + z \cos \theta \\ x' &= x \end{aligned}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{aligned} z' &= z \cos \theta - x \sin \theta \\ x' &= z \sin \theta + x \cos \theta \\ y' &= y \end{aligned}$$

△ R_y es la excepción: el $-\sin \theta$ va **abajo a la izquierda**, no arriba a la derecha.

Explicación complementaria — truco para no equivocarse: la permutación cíclica $x \rightarrow y \rightarrow z \rightarrow x$ convierte la fórmula de R_z (que usa x,y) en la de R_x (reemplazando $x \rightarrow y, y \rightarrow z$) y en la de R_y (con $x \rightarrow z, y \rightarrow x$). Por eso en R_y el signo "cambia de lugar".

[NOTA DE CLASE] (Clase 9 p.15) — "En el espacio, si yo definí un punto no estoy definiendo casi nada; si quiero rotar sobre un punto, ¿para qué dirección se rota? Se rota sobre un eje. O sea, las transformaciones básicas son las rotaciones sobre los 3 ejes coordenados." Es una muy buena justificación de por qué en 3D las rotaciones básicas son sobre ejes y no sobre puntos, y **no está en las filmas**.

Rotación 3D respecto a un eje arbitrario

Problema: rotar un objeto alrededor de un eje que **no es paralelo** a los ejes coordenados.

Estrategia: usar una secuencia de transformaciones para **alinear el eje arbitrario con alguno de los ejes coordenados**.

1. **Trasladar** el eje de rotación para que pase por el origen.
2. **Rotar** el objeto para alinear el eje con un eje coordenado (ej. Z).
3. **Aplicar la rotación deseada** alrededor del eje Z.
4. **Aplicar la rotación de alineación inversa** para restaurar la orientación original.
5. **Aplicar la traslación inversa** para restaurar la posición original.

$$R_{P_1 P_2}(\theta) = T^{-1} \cdot R_x^{-1}(\alpha) \cdot R_y^{-1}(\beta) \cdot R_z(\theta) \cdot R_y(\beta) \cdot R_x(\alpha) \cdot T$$

Con $u = (u_x, u_y, u_z)$ el vector **unitario** del eje:

$$\begin{aligned} \cos \alpha &= \frac{u_z}{\sqrt{u_y^2 + u_z^2}} & \sin \alpha &= \frac{u_y}{\sqrt{u_y^2 + u_z^2}} \\ \cos \beta &= \frac{u_x}{\sqrt{u_x^2 + u_z^2}} & \sin \beta &= -\frac{u_z}{\sqrt{u_x^2 + u_z^2}} \end{aligned}$$

Errores típicos: no normalizar u antes de calcular α y β ; olvidar alguna de las 7 matrices; poner las inversas en el orden equivocado (van **a la izquierda**, en orden inverso al de aplicación).

Reflexiones 3D

Reflexión respecto al plano XY (inversión de Z):

$$M_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Aplicaciones: conversión entre sistemas de coordenadas (dextrógiro $\Leftrightarrow$ levógiro) · efectos de espejo en aplicaciones gráficas · simetría en modelado de objetos.

Cambio de sistema de referencia

Problema: transferir la descripción de un objeto desde el sistema xyz al sistema $x'y'z'$.

Datos: origen (x_0, y_0, z_0) y vectores unitarios ortogonales $\vec{u}'_x, \vec{u}'_y, \vec{u}'_z$.

$$M = R \cdot T = \begin{bmatrix} u'_{x1} & u'_{x2} & u'_{x3} & 0 \\ u'_{y1} & u'_{y2} & u'_{y3} & 0 \\ u'_{z1} & u'_{z2} & u'_{z3} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow P_{x'y'z'} = M \cdot P_{xyz}$$

Explicación complementaria — por qué funciona: primero se lleva el nuevo origen al origen mediante la **traslación inversa**; luego, como los $\vec{u}'$ son **ortonormales**, cada **fila** de R realiza un producto punto que **proyecta** el punto sobre cada nuevo eje. Este mecanismo es **exactamente el paso 2.2 de la cámara** en la Unidad VII.

Multiple Choice de práctica — Unidad VI

1. Una transformación geométrica se define como una operación que se aplica a la descripción geométrica de un objeto para cambiar su:

- A) Color, textura o material B) Posición, orientación o tamaño C) Representación de superficie a volumétrica D) Sistema de coordenadas de dextrógiro a levógiro

2. ¿Cuál de estas transformaciones **NO** es de cuerpo rígido?

- A) Traslación B) Rotación sobre el origen C) Escalado D) Rotación sobre un punto arbitrario

3. La razón principal para introducir coordenadas homogéneas es:

- A) Permitir representar puntos en el infinito B) Que la traslación deje de ser una suma y pase a ser un producto de matrices, unificando la representación C) Reducir el tamaño de las matrices de 4×4 a 3×3 D) Eliminar el error de redondeo en punto flotante

4. En coordenadas homogéneas 2D, la elección $h = 1$ implica que:

- A) Las coordenadas homogéneas coinciden numéricamente con las cartesianas B) La transformación deja de ser invertible C) Sólo pueden representarse traslaciones D) El punto queda proyectado sobre el plano $z = 1$ del espacio 3D

5. Dada $M = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$, la transformación que representa es:

- A) Escalado con $s_x = 5$, $s_y = -3$ B) Traslación de 5 en x y -3 en y C) Rotación de un ángulo cuyo coseno vale 1 D) Reflexión respecto del eje x seguida de traslación

6. Dada $M = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ aplicada a un objeto **no centrado en el origen**, el efecto es:

- A) Sólo duplicar el tamaño del objeto, manteniéndolo en su lugar B) Duplicar el tamaño y alejar el objeto del origen, porque también se escalan sus coordenadas de posición C) Rotar el objeto 90° y duplicar su tamaño D) Trasladar el objeto dos unidades en cada eje

7. Una matriz cuya última fila **no** es $(0 \ 0 \ 0 \ 1)$ indica que la transformación:

- A) Está mal escrita B) Es una proyección en perspectiva, que requerirá homogeneizar dividiendo por w C) Es una rotación sobre un eje arbitrario D) Es un escalado no uniforme

8. En la convención utilizada en la materia, ante $P' = A \cdot B \cdot C \cdot P$ la primera transformación que se aplica al punto es:

A) A B) B C) C D) Depende de si las matrices son de rotación o traslación

9. La composición de transformaciones es:

A) Conmutativa y asociativa B) Asociativa pero no conmutativa C) Conmutativa pero no asociativa D) Ni conmutativa ni asociativa

10. La diferencia geométrica entre $M = T \cdot R$ y $M = R \cdot T$ es que:

A) $T \cdot R$ hace girar la pieza sobre sí misma; $R \cdot T$ la hace orbitar alrededor del origen B) $T \cdot R$ es incorrecta porque la traslación debe aplicarse siempre primero C) Ambas producen el mismo resultado si el ángulo es múltiplo de 90° D) $R \cdot T$ preserva el tamaño y $T \cdot R$ no

11. Para rotar un objeto alrededor de un punto arbitrario (x_r, y_r) , la matriz compuesta correcta es:

A) $T(-x_r, -y_r) \cdot R(\theta) \cdot T(x_r, y_r)$ B) $T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$ C) $R(\theta) \cdot T(x_r, y_r) \cdot T(-x_r, -y_r)$ D) $R(\theta) \cdot T(-x_r, -y_r)$

12. La inversa de una matriz de escalado $S(s_x, s_y)$ es:

A) $S(-s_x, -s_y)$ B) $S(1/s_x, 1/s_y)$ C) $S(s_y, s_x)$ D) S^T

13. Una propiedad particular de las matrices de rotación es que:

A) Su inversa es su transpuesta B) Su determinante es cero C) Son siempre diagonales D) Conmutan entre sí en 3D

14. En la matriz $R_y(\theta)$, el término $-\sin \theta$ se ubica:

A) En la fila 1, columna 2, como en R_z B) En la fila 3, columna 1 (abajo a la izquierda) C) En la fila 2, columna 3 D) En la diagonal principal

15. El sistema de coordenadas **levógiro** se utiliza típicamente para:

A) Modelar los objetos y el mundo B) La representación en pantalla de una escena y describir la profundidad C) Definir los ejes de una elipse en posición estándar D) Almacenar las relaciones topológicas de una malla poligonal

16. La rotación alrededor de un eje arbitrario requiere una composición de:

A) 3 matrices: traslación, rotación y traslación inversa B) 7 matrices: traslación, dos rotaciones de alineación, la rotación deseada, y las tres inversas correspondientes C) 4 matrices: una por cada eje coordenado más la traslación D) 2 matrices: la rotación y su inversa

17. En la matriz de cambio de sistema de referencia, las **filas** del bloque de rotación contienen:

A) Las coordenadas del nuevo origen B) Los vectores unitarios ortogonales del nuevo sistema C) Los ángulos de Euler de la transformación D) Los factores de escala de cada eje

18. Una matriz diagonal $\text{diag}(1, 1, -1, 1)$ en 3D representa:

A) Un escalado uniforme de factor -1 B) Una reflexión respecto al plano XY, que invierte z C) Una rotación de 180° alrededor del eje z D) Una traslación de -1 en z

Respuestas y explicación — Unidad VI

1. B. Definición literal (*Unidad VI p.3*).

2. **C.** Traslación y rotación **preservan forma y tamaño**, por eso son de cuerpo rígido. El escalado cambia el tamaño por definición. La D es una composición de traslaciones y rotación: sigue siendo de cuerpo rígido.
3. **B.** La motivación explícita de la filmina: en cartesianas la traslación requiere una suma adicional, lo que hace compleja la composición y deja sin una forma unificada de representación (*Unidad VI p.11*).
4. **A.** Es la versión "normalizada" (*Unidad VI p.12*).
5. **B.** Diagonal de unos + valores en la última columna = **traslación** de $t_x = 5$, $t_y = -3$. La opción A confunde última columna con diagonal.
6. **B.** El escalado multiplica **todas** las coordenadas, incluidas las de posición. Por eso existe la versión con punto fijo. Es una de las confusiones más frecuentes.
7. **B.** La última fila (0 0 0 1) caracteriza a las transformaciones **afines**. Si no lo es, se genera un w dependiente de la posición y hay que **homogeneizar**: es la firma de la proyección en perspectiva.
8. **C.** Se lee **de derecha a izquierda**: C es la más a la derecha, por lo tanto la primera en aplicarse (*Unidad VI p.15*).
9. **B.** Las dos propiedades que enuncia la filmina. La no conmutatividad es la que tiene consecuencias geométricas.
10. **A.** (*Práctico 04 p.12*). La opción B es falsa: **ninguna de las dos está mal**, producen resultados distintos.
11. **B.** Patrón "ir al origen, hacer, volver", y en el producto lo primero que ocurre va **más a la derecha** (*Unidad VI p.16*). La opción A invierte exactamente el orden.
12. **B.** $S^{-1} = S(1/s_x, 1/s_y)$ (*Unidad VI p.14*). La opción A es el error típico.
13. **A.** $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$. Esta propiedad es la que hace eficiente la matriz de vista en la Unidad VII.
14. **B.** R_y es la excepción entre las tres rotaciones básicas (*Unidad VI p.26*).
15. **B.** (*Unidad VI p.20*). La opción A corresponde al **dextrógiro**.
16. **B.** $T^{-1} \cdot R_x^{-1}(\alpha) \cdot R_y^{-1}(\beta) \cdot R_z(\theta) \cdot R_y(\beta) \cdot R_x(\alpha) \cdot T$ (*Unidad VI p.31*).
17. **B.** Los vectores unitarios ortogonales del sistema destino (*Unidad VI p.33*); cada fila proyecta el punto sobre un nuevo eje.
18. **B.** Es M_{xy} : invierte z, y sirve para convertir entre dextrógiro y levógiro (*Unidad VI p.32*).

Preguntas de desarrollo posibles — Unidad VI

1. Dadas matrices de transformación en coordenadas homogéneas, indique qué tipo de transformación representa cada una y explique qué efecto producirá una composición $D = A \cdot B \cdot C$ sobre los puntos de un objeto.
2. Explique por qué se utilizan coordenadas homogéneas en computación gráfica y qué ventajas aportan.
3. Describa las transformaciones básicas 2D, indicando sus ecuaciones, su forma matricial en coordenadas homogéneas y qué propiedades preserva cada una.
4. Explique por qué la composición de transformaciones no es conmutativa y dé un ejemplo con interpretación geométrica.
5. Describa el procedimiento para aplicar una transformación respecto a un punto que no es el origen, justificando cada paso.

6. Explique cómo se extienden las transformaciones 2D a 3D e indique las diferencias entre los sistemas de coordenadas dextrógiro y levógiro.
7. Describa el procedimiento para rotar un objeto alrededor de un eje arbitrario.
8. Explique en qué consiste un cambio de sistema de referencia y cómo se construye la matriz correspondiente.

Guía de respuestas — Unidad VI

1. Para cada matriz: cómo se la reconoce (dónde están los parámetros), qué transformación es, qué representa cada parámetro y qué le ocurre a posición/orientación/tamaño. Para la composición: la convención de lectura de derecha a izquierda; que C se aplica primero y A última; que D resume las tres en una sola matriz aplicable a todos los vértices; asociatividad y **no conmutatividad** con su consecuencia geométrica.
2. El problema de partida ($P' = M_1 P + Q$, la traslación como suma); las dos desventajas (composición compleja, sin representación unificada); la definición con h y la elección $h = 1$; las tres ventajas; mención de que en perspectiva $h \neq 1$ y hay que homogeneizar.
3. Las tres transformaciones con ecuaciones algebraicas, forma matricial homogénea, y qué preserva cada una. Destacar que T y R son de **cuerpo rígido** y el escalado no, y que el escalado **también desplaza** si el objeto no está en el origen.
4. Enunciar la propiedad; explicar que cada matriz transforma el sistema sobre el que opera la siguiente; el ejemplo $T \cdot R$ vs $R \cdot T$ con la interpretación (gira sobre sí misma vs órbita); aclarar que ninguna es incorrecta.
5. El patrón de tres pasos con justificación de por qué hace falta (las matrices básicas operan respecto del origen); el producto $M = T(p) \cdot X \cdot T(-p)$ con la aclaración del orden; los dos ejemplos (rotación sobre punto y escalado con punto fijo) con sus matrices resultantes; la verificación de que el punto de referencia queda fijo.
6. Aparición de z ; coordenadas homogéneas $(x, y, z, 1)$; matrices 4×4 ; la forma general con la última fila $(0 \ 0 \ 0 \ 1)$; los dos sistemas con la regla de la mano y el uso de cada uno; las tres rotaciones básicas con la convención de signo y la excepción de R_y .
7. El problema; la estrategia de alinear con un eje coordenado; los cinco pasos; la fórmula de las 7 matrices; las expresiones de $\cos \alpha$, $\sin \alpha$, $\cos \beta$, $\sin \beta$; la necesidad de normalizar el vector del eje.
8. El problema (transferir la descripción de un sistema a otro); los datos necesarios (origen y vectores unitarios ortogonales); $M = R \cdot T$; por qué la traslación es inversa y por qué las filas de R son los nuevos ejes; la conexión con el paso 2.2 de la cámara.

UNIDAD VII — Visualización en 3D

Prioridad MUY ALTA. Aportó **dos** de las siete preguntas del parcial anterior: la 6 (especificaciones mínimas de la cámara sintética para perspectiva) y la 7 (proceso completo de 3D a imagen 2D).

Parte 1 — Proyecciones y cámara sintética

7.1 Concepto de proyección geométrica

Antecedente histórico: en **1525 Durero** registra en un tallado de madera un método por el cual se puede crear un dibujo en perspectiva de cualquier objeto.

Definición

Una **proyección geométrica** es el **mapeo de puntos de un espacio 3D a un plano 2D** mediante **líneas de proyección** que **convergen en un centro de proyección** o son **paralelas entre sí**. — *Unidad VII-1 p.4*

Elementos fundamentales

Elemento	Qué es
Objeto 3D	Entidad a proyectar
Centro de proyección	Punto desde donde parten las líneas de proyección
Plano de proyección	Superficie donde se forma la imagen
Líneas de proyección	Rayos que conectan puntos del objeto con el plano

7.2 Clasificación de las proyecciones geométricas planares

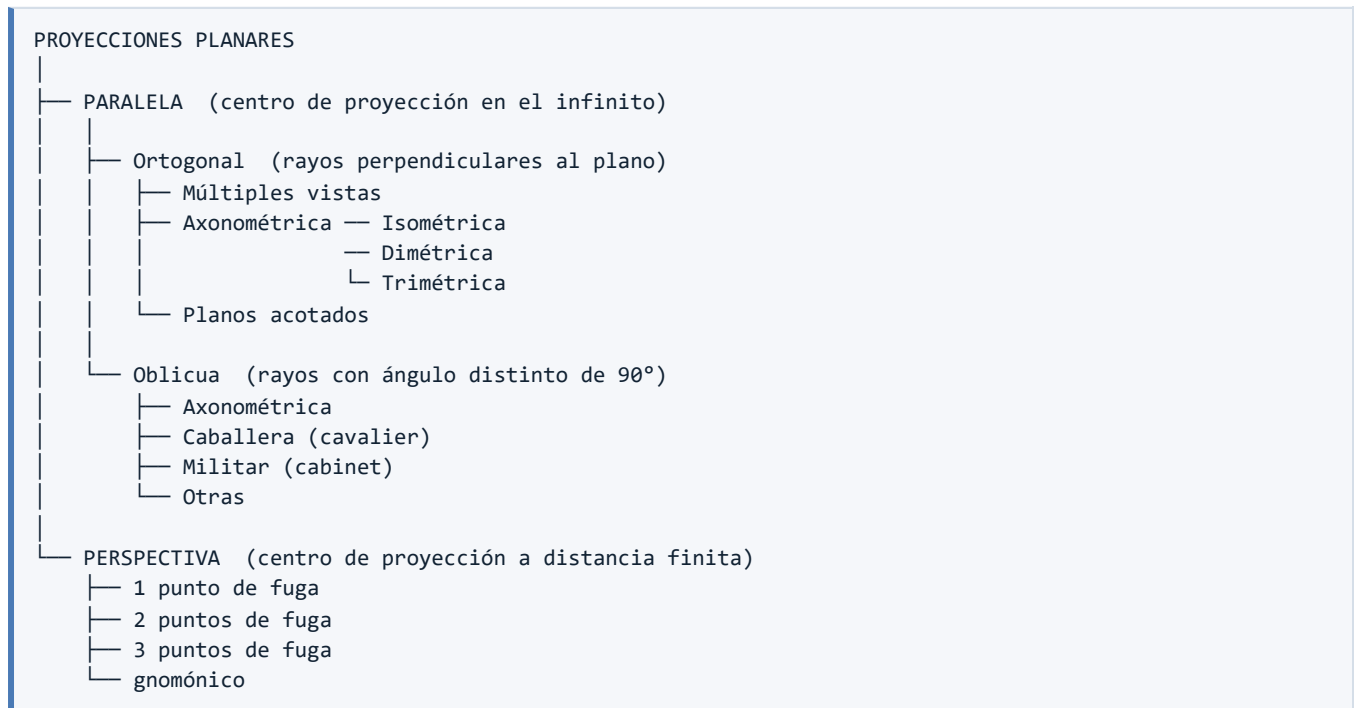
Primer criterio — distancia entre el centro de proyección y el plano de proyección:

	Paralela	Perspectiva
Centro de proyección	En el infinito	Cercano al plano de proyección (distancia finita)
Líneas de proyección	Paralelas entre sí	Convergentes
Por qué se usa	Utilizadas en ingeniería y arquitectura porque mantienen las proporciones relativas	Imitan lo que nuestros ojos ven , como si fueran una cámara → imágenes de aspecto más natural

Segundo criterio (sub-clasificación de las paralelas) — ángulo que forman las líneas de proyección con el plano:

- **Ortogonales:** líneas de proyección **normales** (perpendiculares) al plano de proyección.
- **Oblicuas:** ángulo de las líneas de proyección $\neq 90^\circ$.

Árbol completo de clasificación



7.3 Proyecciones paralelas ortogonales

Múltiples vistas

Definición: plano de proyección **paralelo a cualquier plano del sistema de referencia**; líneas de proyección **perpendiculares** al plano.

Características: ángulos y longitudes **se preservan exactamente** · cada vista muestra **sólo dos dimensiones** · sistema **estándar en dibujo técnico** · **no provee una imagen "real"** del objeto · la forma 3D del objeto **se debe componer mentalmente**.

Axonométricas

Definición: plano de proyección **no paralelo** con los planos del sistema de referencia; líneas de proyección **perpendiculares** al plano.

Características: se muestran **múltiples caras** del objeto simultáneamente · visualización 3D del objeto · **proporciones preservadas** · fácil interpretación.

Tipo	Ángulos entre ejes	Escalado	Adecuada para
Isométrica	Los tres iguales (120°)	Igual en los tres ejes	Perspectiva simple
Dimétrica	Dos iguales (ej. 131,5°)	Igual en dos ejes	Objetos con una cara preponderante
Trimétrica	Los tres diferentes	Diferente en los tres ejes	Destacar una cara del objeto del resto

7.4 Proyecciones paralelas oblicuas

Definición: plano de proyección **paralelo a uno de los planos** del sistema de referencia, pero las líneas de proyección forman un **ángulo oblicuo** con el plano.

Características: visualización 3D del objeto · **se representa de forma exacta una cara** del objeto · forma 3D **no realista** · los objetos **se pueden distorsionar**.

Parámetros de definición:

- **Ángulo α :** entre la línea de proyección y el plano.
- **Ángulo ϕ :** dirección en el plano de proyección.

Ecuaciones generales:

$$x_p = x + L_1(z_{vp} - z) \cos \phi \quad y_p = y + L_1(z_{vp} - z) \sin \phi \quad \text{donde } L_1 = \cot \alpha$$

Tipo	α	L_1	Efecto en la profundidad	ϕ típico
Caballera (<i>cavalier</i>)	45° (por tanto $\tan \alpha = 1$)	$\cot 45^\circ = 1$	Las dimensiones perpendiculares al plano no cambian de magnitud . Adecuada para perspectiva rápida	30° y 45°
Militar (<i>cabinet</i>)	$\approx 63,4^\circ$ (por tanto $\tan \alpha = 2$)	$\cot \alpha = 0,5$	Las dimensiones perpendiculares se reducen a la mitad . Aspecto más realista que cavalier (reduce la distorsión en profundidad)	30° y 45°

Explicación complementaria — memotecnia: *cabinet* significa "mueble"; los muebles se dibujan con la profundidad **achatada** para que no se vean deformados → cabinet es el de 0,5.

Error típico: confundir α con ϕ . α **decide cuánto se acorta la profundidad**; ϕ **decide hacia dónde se dibuja**.

7.5 Proyecciones en perspectiva

Características principales: centro de proyección en **posición finita** · líneas de proyección **convergen** · los **objetos lejanos aparecen más pequeños** · efecto de **escorzo** (*foreshortening*).

Ventajas: vista más realista · simula la visión humana · sensación de profundidad.

Puntos de fuga

Punto de fuga: punto donde **convergen las proyecciones de líneas paralelas que no son paralelas al plano de proyección**. — Unidad VII-1 p.15

Los **puntos de fuga principales** corresponden a los **ejes X, Y, Z del objeto** y **controlan la orientación de la vista**.

Clasificación	Condición
1 punto de fuga	Un eje principal intersecta el plano
2 puntos de fuga	Dos ejes principales intersectan el plano
3 puntos de fuga	Los tres ejes principales intersectan el plano

Error típico: contar los ejes que **son paralelos** al plano. Se cuentan los que **lo intersectan**.

Matemática de la perspectiva

Descripción paramétrica de un punto en el proyector:

$$x' = x - (x - x_{prp})t \quad y' = y - (y - y_{prp})t \quad z' = z - (z - z_{prp})t \quad 0 \leq t \leq 1$$

Resolviendo para $z' = z_{vp}$ (el plano de vista):

$$x_p = x \frac{z_{prp} - z_{vp}}{z_{prp} - z} + x_{prp} \frac{z_{vp} - z}{z_{prp} - z}$$

$$y_p = y \frac{z_{prp} - z_{vp}}{z_{prp} - z} + y_{prp} \frac{z_{vp} - z}{z_{prp} - z}$$

donde **prp** = *projection reference point* (el centro de proyección) y **vp** = *view plane*.

Concepto clave: la **división por** $(z_{prp} - z)$ es lo que hace que lo lejano se vea más chico. **La perspectiva no es lineal en z** — por eso más adelante se resuelve con coordenadas homogéneas y una **división por w**.

7.6 Resumen comparativo de proyecciones

Tipo	Ventajas	Aplicaciones
Ortogonal	Medidas exactas, múltiples vistas	Dibujo técnico, planos
Axonométrica	Vista 3D, proporciones	Ilustraciones técnicas
Oblicua	Una cara en verdadera forma	Diagramas, esquemas
Perspectiva	Realismo visual	Visualización, juegos, CAD

Criterios de selección:

- **Precisión vs. realismo:** paralelas para medición, perspectiva para visualización.
- **Complejidad computacional:** paralelas más eficientes.
- **Propósito:** documentación técnica vs. presentación visual.

Diferencia fundamental

En la **paralela** el centro de proyección está en el infinito, los rayos son paralelos y por eso **las proporciones se conservan a cualquier distancia**: sirve para medir. En la **perspectiva** el centro está a distancia finita, los rayos convergen y la proyección incluye una **división por la profundidad**, de modo que lo lejano se achica: sirve para ver con naturalidad, pero **ya no se puede medir sobre el dibujo**.

7.7 La cámara sintética

Ésta es la pregunta 6 del parcial anterior: "Indique cuáles son las especificaciones mínimas necesarias para un modelo de cámara sintética para una visualización en perspectiva."

Definición

Cámara: mecanismo para transformar un mundo 3D en una imagen 2D, proyectando puntos 3D en un plano de proyección. Se basa en el **comportamiento físico de la luz**.

Modelo base: cámara tipo "**pinhole**" (estenopeica).

Conceptos previos

Concepto	Definición
Ángulos de visión	Campo visual que un observador puede captar. Para la vista humana determina la nitidez de lo observado
Volumen de visualización	Todo lo que se ve desde una posición, en una dirección y con una orientación
Cono de visualización	Aproximación de lo que ven nuestros ojos

Modelo conceptual del proceso de visualización 3D

Especificar un observador el cual **ve una parte del mundo contenida en un volumen de visualización (VV)** a través de una **ventana gráfica** que contiene la **imagen proyectada del VV en un plano de proyección**. — *Unidad VII-1 p.20*

Pasos del proceso:

1. Los objetos se posicionan en la escena 3D.
2. Se posiciona la cámara y se le da una orientación.
3. Se determina qué objetos "ve" la cámara.
4. Se transforman coordenadas al volumen de visualización.
5. Se determina cuáles objetos están dentro del volumen (**clipping**).
6. Se aplica la proyección para llevar los objetos al plano de proyección.
7. Se aplica la transformación para llevar la imagen a la ventana de visualización.

Coordenadas 3D de la escena (sist. mundo)	--->	Recorte usando el volumen de visualización (Clip)	--->	Proyectar en el plano de imagen	--->	Transformación a coords. 2D de la ventana del disp.	--->	Coordenadas 2D del dispositivo
---	------	--	------	------------------------------------	------	---	------	-----------------------------------

Las especificaciones mínimas de una cámara sintética

Unidad VII-1 p.21 y p.32. Ésta es la lista que hay que saber.

Especificación	Qué define	Detalle
1. Tipo de proyección	Paralela o perspectiva	Determina la forma del volumen de visualización
2. Posición	Dónde está la cámara	Punto (x_c, y_c, z_c) en el sistema de referencia del mundo (dextrógiro)
3. Dirección — vector <i>Look at</i>	Hacia dónde apunta	Indica hacia dónde mira; 3 grados de libertad : vector hacia un punto en el espacio 3D
4. Orientación — vector <i>Up</i>	Cómo se gira la cámara alrededor del vector dirección	Indirectamente fija la orientación del plano de proyección (vertical, apaisado, etc.)
5. Campo de visión	Qué parte del mundo es visible	Relación de aspecto (AR) y ángulo de visión (θ_h)
6. Profundidad de campo	Hasta dónde se ve	Plano de recorte frontal y plano de recorte trasero
7. Distancia focal	(<i>opcional</i>) Simula el enfoque de lentes reales	La filmina la marca explícitamente como "característica opcional"
8. Plano de proyección	Dónde se forma la imagen	Su posición queda definida al elegir el campo de visión y el tamaño de la ventana

Con estos parámetros se define en la cámara un **sistema de coordenadas propio** (u, v, w) .

Volumen de visualización

Tipo de proyección	Volumen infinito	Volumen cerrado por los planos de recorte
Perspectiva	Se aproxima el cono de visualización mediante una pirámide infinita	Pirámide trunca (<i>frustum</i>)
Paralela	Prisma rectangular infinito	Prisma rectangular

Campo de visión

Relación de aspecto (AR): determina la **proporción entre el ancho y el alto** de la ventana de visualización. Una ventana cuadrada tiene relación de aspecto 1:1. Es similar al tamaño de la "película" o sensor CCD utilizado en una cámara.

$$AR = \frac{width}{height}$$

Ángulo de visión: afecta la **distorsión por perspectiva** que tendrá la proyección.

- Proyección **paralela**: ninguna distorsión.
- Proyección en **perspectiva**: puede ser grande (lentes gran angular).

Seleccionar un ángulo de visión es **equivalente a cuando un fotógrafo selecciona el tipo de lente**. Los lentes **gran angular** causan gran distorsión de la perspectiva; los **teleobjetivo**, para tomas de larga distancia, **recortan la profundidad** aunque generan muy poca distorsión.

Para el volumen piramidal se definen **dos ángulos de apertura**: el **ángulo en altura** θ_h y el **ángulo en ancho** θ_w . Usualmente se especifica el de altura y se calcula el de ancho. **Para proyección paralela no es necesario especificar estos ángulos.**

⚠ **CONTRADICCIÓN ENTRE DOCUMENTOS — importante.**

Fórmula	Dónde aparece
$\theta_w = AR \cdot \theta_h$	Unidad VII-1 p.27 y p.32
$\tan(\theta_w/2) = AR \cdot \tan(\theta_h/2)$	Guía práctica 05 p.2 y Práctico 06 p.13

No son equivalentes; coinciden sólo aproximadamente para ángulos pequeños. La segunda corresponde a la geometría real del frustum y es la que implementa `glm::perspective`. **Si la pregunta es sobre la teoría de la Unidad VII, la filmina dice la primera.** Conviene consultarlo.

Profundidad de campo — planos de recorte

Debido al **costo computacional** de plasmar una escena hay que **acotar el volumen de visualización**. Se definen **dos planos normales a la dirección del vector *Look at*: frontal y trasero**. Se denominan **planos de recorte** porque definen los límites para visualizar o descartar partes de la escena.

Situación del objeto	Qué se hace	Nombre
Completamente fuera del volumen	Se descarta	culling
Intersecta un plano del volumen	Se recorta	clipping
Completamente dentro	Se dibuja	—

Justificación del plano de recorte frontal:

- No es conveniente dibujar objetos **cercanos** a la cámara: podrían **bloquear el resto del escenario** y los objetos **podrían distorsionarse**.
- No se desea dibujar objetos que se encuentren **detrás** de la cámara: no es lógico verlos y, en perspectiva, **aparecerán invertidos** como consecuencia de la transformación.

Justificación del plano de recorte trasero:

- No se desea dibujar objetos **muy lejanos**: aparecerán tan pequeños que serán **insignificantes visualmente** e implicarán tiempo de procesamiento.
- Descartar objetos distantes implica **perder detalle**, pero **se ahorra tiempo** de procesamiento.

Distancia focal (opcional)

Busca **aproximar el comportamiento de las lentes** de una cámara real. Es la medida de la **distancia de foco ideal**: los objetos que se encuentran a la distancia focal se muestran **enfocados**; los que están a distancia menor o mayor se muestran **borrosos**. Se combina con la profundidad de campo.

Plano de proyección

Es el **equivalente al plano donde se ubica la "película" o sensor CCD** de las cámaras de fotografía. Su posición queda definida al elegir el campo de visión y el tamaño de la ventana de recorte:

$$z_{prp} - z_{vp} = \frac{height}{2} \cot\left(\frac{\theta_h}{2}\right) = \frac{width \cdot \cot(\theta_h/2)}{2 \cdot aspect}$$

Cómo respondería esto en un parcial

Una **cámara sintética** es el mecanismo que permite transformar un mundo 3D en una imagen 2D proyectando puntos sobre un plano de proyección, basándose en el comportamiento físico de la luz; su modelo base es la cámara estenopeica o *pinhole*. El modelo conceptual consiste en especificar un observador que ve una parte del mundo contenida en un **volumen de visualización** a través de una ventana gráfica que contiene la imagen proyectada de ese volumen sobre el plano de proyección.

Para una visualización **en perspectiva**, las especificaciones mínimas son las siguientes.

En primer lugar, el **tipo de proyección**, que en este caso es perspectiva y determina que el volumen de visualización sea una **pirámide** que aproxima el cono de visualización.

En segundo lugar, la **posición** de la cámara: un punto (x_c, y_c, z_c) expresado en el sistema de referencia del mundo, que es dextrógiro.

En tercer lugar, la **dirección y la orientación**. La dirección se especifica con el vector **Look at**, que indica hacia dónde apunta la cámara y aporta tres grados de libertad. La orientación se especifica con el vector **Up**, que establece cómo debe girarse la cámara alrededor del vector dirección y fija indirectamente la orientación del plano de proyección, por ejemplo vertical o apaisado. Con estos datos se define el sistema de coordenadas propio de la cámara, (u, v, w) .

En cuarto lugar, el **campo de visión**, caracterizado por la **relación de aspecto** —la proporción entre ancho y alto de la ventana, equivalente al tamaño de la película o sensor— y el **ángulo de visión**, que en perspectiva determina la distorsión y equivale a la elección del tipo de lente: un gran angular produce mucha distorsión, mientras que un teleobjetivo genera poca pero recorta la profundidad. Se especifica el ángulo en altura θ_h y a partir de él se obtiene el ángulo en ancho θ_w .

En quinto lugar, la **profundidad de campo**, dada por los **planos de recorte frontal y trasero**, normales a la dirección del *Look at*. Éstos cierran el volumen, que pasa de ser una pirámide infinita a una **pirámide truncada** o *frustum*. El plano frontal evita dibujar objetos muy cercanos —que bloquearían la escena o se distorsionarían— y objetos detrás de la cámara, que en perspectiva aparecerían invertidos. El plano trasero descarta objetos tan lejanos que resultarían visualmente insignificantes, ahorrando tiempo de procesamiento a costa de perder detalle.

Opcionalmente puede especificarse la **distancia focal**, que aproxima el comportamiento de las lentes reales mostrando enfocados los objetos ubicados a esa distancia y borrosos los demás.

Respecto de los objetos y el volumen, aquellos que quedan completamente fuera se **descartan** (*culling*), los que intersectan un plano del volumen se **recortan** (*clipping*) y los que están completamente dentro se dibujan.

Parte 2 — Transformación de normalización, recorte y proyección

Ésta es la pregunta 7 del parcial anterior: *"Describe el proceso y las transformaciones necesarias para convertir una visualización 3D arbitraria en una imagen 2D que pueda luego ser mostrada en un dispositivo de visualización."*

7.8 El problema y la estrategia

Objetivo: obtener una imagen 2D del modelo del mundo a partir de los parámetros de visualización 3D.

Problema: la proyección y el recorte en una vista arbitraria son difíciles de resolver.

Solución: reducir a problemas más simples.

Metodología: usar una **visualización canónica**, definida mediante una **posición de visualización canónica** y un **volumen de visualización canónico**.

Procedimiento — los tres pasos:

1. **Obtener los parámetros** de especificación de la visualización.
2. **Transformar** la visualización especificada en una visualización **canónica**.
3. **Generar la imagen 2D** usando el volumen de visualización canónico para **proyectar y recortar** la escena.

7.9 La visualización canónica

	Especificación
Posición	$(0, 0, 0)$
Vector Look at	$(0, 0, -1)$
Vector Up	$(0, 1, 0)$
Tipo de proyección del volumen	Paralela (prisma rectangular)
Tamaño del volumen	$-1 \leq x, y \leq 1$ y $0 \leq z \leq -1$

Por qué se elige así:

- **Es simple aplicar los métodos de recorte** usando planos que se encuentran **alineados con los ejes** (basta comparar coordenadas contra ± 1).
- **La proyección en un volumen canónico es simple:** proyectar (x, y) , **no es necesario considerar z**.

7.10 La transformación de normalización

Transformación de normalización: se denomina así a la **composición de transformaciones necesarias para pasar de una visualización arbitraria a una visualización canónica**.

Objetivo: transformar la visualización arbitraria del mundo a una visualización canónica **manteniendo la relación entre el volumen de visualización y el mundo**.

Secuencia — vale para volúmenes de proyecciones tanto paralelas como en perspectiva:

- **2.1** Trasladar la cámara al origen del sistema (x, y, z) .
- **2.2** Aplicar transformación para **alinear el sistema de referencia de la cámara** (u, v, w) con el sistema (x, y, z) .
- **2.3 Ajustar la escala** del volumen de visualización para que quede contenido en $-1 \leq x, y \leq 1$ y $0 \leq z \leq -1$.
- **2.4 (sólo perspectiva) Distorsionar la pirámide truncada (frustum) a un prisma rectangular.**

Concepto clave. Cada paso aclara que *"se aplica a todos los vértices de los objetos para preservar las distancias entre cámara y escena"*. Es decir: **la cámara nunca se mueve. Se mueve la escena**, para que la cámara quede en la posición canónica. Por eso las dos primeras transformaciones son **inversas** (Práctico 06 p.9).

Paso 2.1 — Traslación

Acción: trasladar los objetos de la escena dentro del volumen de visualización **respecto al origen** del sistema de referencia.

Transformación: traslación inversa usando la posición definida para la visualización, P_c .

$$T(-P_c) = \begin{bmatrix} 1 & 0 & 0 & -P_{cx} \\ 0 & 1 & 0 & -P_{cy} \\ 0 & 0 & 1 & -P_{cz} \\ 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow q' = T(-P_c) \cdot q$$

Paso 2.2 — Alinear (u,v,w) con (x,y,z)

Acción: alinear los ejes (u, v, w) con los ejes (x, y, z) para lograr el volumen de visualización en $-z$.

Implicancias: se debe usar una **rotación**, aplicada a los objetos de la escena para mantener su orientación relativa respecto al volumen de visualización.

Transformación: rotación inversa a partir de la orientación relativa entre (u, v, w) y (x, y, z) .

Opción 1: alinear el vector w con el eje z mediante $M' = R_y(\beta) \cdot R_x(\alpha)$ (las de la Unidad VI). **Desventaja: costosa computacionalmente;** hay que realizar el producto de matrices en **cada cambio de visualización**.

Opción 2 (la que se usa): usar **propiedades de las matrices de rotación**.

Propiedad clave (Unidad VII-2 p.11). Sea $M = [v_1 \ v_2 \ v_3]$ una matriz de rotación (vectores como columnas); entonces $M\hat{e}_1 = v_1$, $M\hat{e}_2 = v_2$, $M\hat{e}_3 = v_3$: **M rota los vectores canónicos para que coincidan con (v_1, v_2, v_3)** . Como las columnas son **unitarias** ($\|v_i\| = 1$) y **perpendiculares** ($v_i \cdot v_j = 0$ para $i \neq j$), se demuestra que:

$$M^T M = I \implies M^{-1} = M^T$$

¡Gran ventaja computacional! Invertir es transponer.

$$M = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad M^T = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

M (con u, v, w como **columnas**) rota $(x, y, z) \rightarrow (u, v, w)$. Lo que se necesita es **lo inverso**: M^T , en cuyas **filas** están u, v y w .

$$q' = M^T \cdot T(-P_c) \cdot q$$

Construcción de la base ortonormal (u,v,w)

Condiciones que debe cumplir el sistema: el vector *Look at* arbitrario sobre los **negativos del eje w** · la **proyección del vector Up** sobre el plano perpendicular a w **colineal con el eje v** · el eje u **mutuamente perpendicular** a v y w · (u, v, w) forma un sistema coordenado **dextrógiro**.

Primer paso — calcular w. El vector *Look at* en la visualización canónica se encuentra sobre $-z$:

$$w = -\frac{\text{Look}}{\|\text{Look}\|}$$

Segundo paso — calcular u. Restricciones: (1) debe ser **perpendicular al plano definido por w y Up** ; (2) (u, Up, w) debe formar un **sistema de mano derecha**. Solución: usar producto cruz. Tanto $w \times Up$ como $Up \times w$ son perpendiculares al plano, pero **los productos cruz son de mano derecha**, así que se usa $Up \times w$:

$$u = \frac{Up \times w}{\|Up \times w\|}$$

Tercer paso — calcular v. Restricciones: perpendicular al plano definido por u y w ; (u, v, w) dextrógiro:

$$v = w \times u$$

¿Por qué no hay que normalizar v ? Porque w y u son **vectores unitarios mutuamente perpendiculares**, así que **su producto cruz también es unitario**.

⚠ **Nota sobre convenciones.** Unidad VII-2 p.15 define $w = -Look/\|Look\|$, mientras que Práctico 06 p.10 escribe $w = (P_c - P_{ref})/\|P_c - P_{ref}\|$, **sin** el signo menos. Son **equivalentes**, porque parten de vectores opuestos. Lo importante es **ser coherente** con una de las dos.

Paso 2.3 — Ajustar proporciones en x e y

Planteo: se une el origen con los bordes del plano de recorte trasero. Para proyección **perspectiva** las líneas de unión son los **bordes del volumen**; para **paralela** quedan **contenidas dentro** del volumen.

Se plantea hacer que el **ángulo que forman las líneas trazadas sea de 90°** , lo que logra que pasen por los **vértices traseros del volumen canónico**. Para el plano xz, se escala en dirección x:

$$\tan\left(\frac{\theta_w}{2}\right) \cdot k_x = 1 \implies k_x = \frac{1}{\tan(\theta_w/2)} = \cot\left(\frac{\theta_w}{2}\right)$$

Transformación: escalado en las direcciones x e y.

$$S_{xy} = \begin{bmatrix} \cot(\theta_w/2) & 0 & 0 & 0 \\ 0 & \cot(\theta_h/2) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \implies q' = S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$$

Paso 2.3 — Ajustar proporciones en z

Planteo: el plano trasero se encuentra en algún lugar con $z \neq -1$. La distancia en z desde el origen hasta el plano se denomina **far**. Es necesario mover el plano llevándolo a $z = -1$. **Para mantener las proporciones hay que aplicarlo en los ejes x e y también.**

Transformación: escalado **proporcional en las tres direcciones**.

$$(S_2)_{xyz} = \begin{bmatrix} 1/far & 0 & 0 & 0 \\ 0 & 1/far & 0 & 0 \\ 0 & 0 & 1/far & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \implies q' = (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$$

Error típico: escalar **sólo** en z. Si se hace eso, se deforman los ángulos que se acababan de ajustar.

Paso 2.4 — Deformar la pirámide truncada (sólo perspectiva)

Se define el parámetro $k = near/far$: distancia del plano frontal. Transformar los puntos del volumen de visualización en perspectiva en el volumen de visualización paralelo implicará:

- **Proyectar los puntos.**
- **Trasladar el plano frontal al origen** para lograr $0 \leq z \leq -1$.

La matriz de distorsión resultante, con \$0

$$D = \begin{bmatrix} k-1 & 0 & 0 & 0 \\ 0 & k-1 & 0 & 0 \\ 0 & 0 & -1 & -k \\ 0 & 0 & -k-1 & 0 \end{bmatrix}$$

Luego, **al homogeneizar el vector resultante**, se termina de aplicar la transformación de proyección.

⚠ **Ojo — esta matriz no verifica los extremos del volumen.** Aplicando D a $(x, y, z, 1)$ se obtiene $w' = (-k - 1)z$; evaluando el plano trasero $z = -1$ resulta $z_{ndc} = (1 - k)/(1 + k)$, cuando debería ser -1 ; y las esquinas $x = \pm 1$ van a $(k - 1)/(k + 1)$ en vez de ± 1 . Con la última fila $(0, 0, 1 - k, 0)$ **todos los extremos cierran correctamente. No reemplaces la filmina por tu cuenta:** conviene consultarlo. Lo que sí es seguro, y es lo conceptual que se evalúa, es que **la última fila no es $(0, 0, 0, 1)$** , que eso genera una w dependiente de z , y que **al homogeneizar (dividir por w) se obtiene la división por la profundidad** que achica lo lejano.

(Nota adicional: la misma filmina escribe el intervalo como $k \leq z \leq -1$, lo cual es imposible con \$0

Resultado final

$$q' = D \cdot (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$$

Sólo es necesario conocer los **parámetros de la definición de cámara sintética** (posición, Look at, Up, AR, planos de recorte frontal y trasero) para determinar la transformación.

7.11 Recorte y proyección

Recorte (clipping)

Eliminar objetos que están fuera del volumen normalizado.

- Puede realizarse **antes o después** de aplicar la matriz de proyección.
- **La normalización simplifica la operación de recorte.**
- Está **implementado en el hardware** del pipeline de renderizado.

Vértices: evaluar las coordenadas de los vértices (**itransformados!**) contra los intervalos $-1 \leq x, y \leq 1$ y $0 \leq z \leq 1$, y descartar lo que cae fuera.

⚠ **Inconsistencia interna del material.** El volumen canónico se define como $0 \leq z \leq -1$ (p.5, p.7, p.19, p.21) pero el recorte y la proyección lo evalúan contra 0 y 1 positivos (p.24, p.25, p.27). Las filminas no lo resuelven. Como contraste, Práctico 06 p.7 es explícito: "Teórico: $-1 \leq x, y \leq 1$; $0 \leq z \leq -1$. OpenGL: $-1 \leq x, y \leq 1$; $-1 \leq z \leq 1$."

Bordes: se recortan calculando las coordenadas del punto de intersección del borde con los planos del volumen, usando la **forma paramétrica del segmento**:

$$x = (1 - t)x_0 + tx_1 \quad y = (1 - t)y_0 + ty_1 \quad z = (1 - t)z_0 + tz_1$$

Ejemplo de despeje — sustituir $x = 1$ y resolver para t :

$$1 = (1 - t)x_0 + tx_1 \Rightarrow 1 - x_0 = t(x_1 - x_0) \Rightarrow t = \frac{1 - x_0}{x_1 - x_0}$$

Si $0 \leq t \leq 1$, el segmento efectivamente cruza el plano en ese punto.

Superficies: hay que determinar las intersecciones de las superficies con los planos y luego **generar nuevas superficies** a partir de los puntos calculados.

Proyección y mapeo a pantalla

Con el volumen canónico (paralelo), se puede generar un plasmado **simplemente descartando el valor z y proyectando sobre el plano xy** . Un punto dentro de los intervalos hay que **mapearlo al espacio de pantalla**, por ejemplo $0 \leq x' < 1024$ y $0 \leq y' < 768$:

$$z \rightarrow \text{se ignora} \quad x' = 1023 \cdot \frac{x+1}{2} \quad y' = 767 \cdot \frac{y+1}{2}$$

- Si la visualización se encuentra dentro de una **ventana**, será necesario **escalar y trasladar** los valores a las coordenadas de ventana.
- **Nota de la filmina:** por ser proyección paralela **se puede hacer la proyección sobre cualquier plano de recorte** (frontal o trasero): **la imagen será siempre la misma.**

Error típico: usar 1024 y 768 en vez de 1023 y 767. Los índices de píxel van de 0 a $W - 1$.

7.12 Resumen del proceso completo

ENTRADA: escena 3D en coordenadas del mundo + parámetros de cámara sintética

```

|
v
[1] T(-Pc)      Traslación inversa: Lleva la cámara al origen
|              Recibe: vértices en coords. del mundo
|              Produce: vértices relativos a la posición de la cámara
v
[2] M^T         Rotación inversa: alinea (u,v,w) con (x,y,z)
|              Se construye con u, v, w como FILAS. M^-1 = M^T
|              Produce: vértices en coordenadas de CÁMARA
v              (juntas, [1]+[2] son la MATRIZ DE VISTA)
[3] Sxy         Escalado en x e y: hace que las líneas al plano
|              trasero formen 90°, ajustando al ancho canónico
v
[4] (S2)xyz     Escalado proporcional en las 3 direcciones:
|              Lleva el plano trasero a z = -1
v
[5] D           (SOLO PERSPECTIVA) Deforma el frustum en prisma:
|              proyecta y traslada el plano frontal al origen.
|              Genera w dependiente de z
v              (juntas, [3]+[4]+[5] son la MATRIZ DE PROYECCIÓN)
[6] Homogeneizar División por w: completa la proyección perspectiva
|              (es lo que achica lo lejano)
v
[7] RECORTE     Descartar vértices fuera de -1..1 ; recortar bordes
|              con la forma paramétrica y el parámetro t
v
[8] PROYECCIÓN  Descartar z, proyectar sobre el plano xy
|
v
[9] VIEWPORT    Mapear de [-1,1] a coordenadas de pantalla:
|              x' = (W-1)(x+1)/2 , y' = (H-1)(y+1)/2
v
SALIDA: coordenadas 2D del dispositivo, listas para el frame buffer

```

El problema consiste en que **proyectar y recortar en una visualización arbitraria es difícil de resolver**, porque el volumen de visualización puede tener cualquier posición, orientación y forma. La estrategia es **reducirlo a un problema más simple** mediante una **visualización canónica**, y el procedimiento tiene tres pasos: obtener los parámetros de la visualización, transformar esa visualización en la canónica, y finalmente proyectar y recortar usando el volumen canónico.

La **visualización canónica** se define con la cámara en el origen $(0, 0, 0)$, el vector Look at en $(0, 0, -1)$ y el Up en $(0, 1, 0)$, con un volumen de proyección paralela —un prisma rectangular— de tamaño $-1 \leq x, y \leq 1$ y $0 \leq z \leq -1$. Se elige así porque **el recorte contra planos alineados con los ejes es trivial**, ya que basta comparar coordenadas contra ± 1 , y porque **proyectar se reduce a descartar z**.

La **transformación de normalización** es la composición de transformaciones que lleva de la visualización arbitraria a la canónica. Es importante notar que se aplica **a todos los vértices de los objetos**, no a la cámara: conceptualmente **la cámara no se mueve, se mueve la escena** para que la cámara quede en la posición canónica. Por eso las dos primeras transformaciones son **inversas**.

El **primer paso** es una **traslación inversa** $T(-P_c)$, que lleva la cámara al origen del sistema usando su posición.

El **segundo paso** es una **rotación inversa** que alinea el sistema de referencia de la cámara (u, v, w) con el sistema (x, y, z) . En lugar de componer rotaciones sucesivas, que sería costoso, se aprovecha una propiedad de las matrices de rotación: como sus columnas son vectores unitarios y mutuamente perpendiculares, se cumple que $M^T M = I$ y por lo tanto $M^{-1} = M^T$, de modo que **invertir es simplemente transponer**. La matriz que se usa es M^T , con u, v y w como **filas**. Esa base ortonormal se construye a partir del Look at y el Up: $w = -\text{Look} / \|\text{Look}\|$, porque la cámara canónica mira hacia $-z$; $u = (\text{Up} \times w) / \|\text{Up} \times w\|$, usando ese orden para que el sistema resulte de mano derecha y normalizando porque Up no es necesariamente perpendicular a w ; y $v = w \times u$, que no necesita normalizarse porque es el producto cruz de dos vectores unitarios perpendiculares.

El **tercer paso** ajusta las proporciones del volumen. Primero se escala en x e y mediante S_{xy} , con factores $\cot(\theta_w/2)$ y $\cot(\theta_h/2)$, de modo que las líneas que unen el origen con los bordes del plano trasero formen 90° y pasen por los vértices traseros del volumen canónico. Luego se aplica $(S_2)_{xyz}$, un escalado de factor $1/\text{far}$ **en las tres direcciones**, que lleva el plano trasero a $z = -1$; se escala también en x e y para no deformar los ángulos recién ajustados.

El **cuarto paso** se aplica **sólo en proyección perspectiva**: la matriz D **deforma la pirámide truncada en un prisma rectangular**, proyectando los puntos y trasladando el plano frontal al origen. Su característica distintiva es que **su última fila no es** $(0, 0, 0, 1)$, por lo que genera una coordenada homogénea w que depende de z ; al **homogeneizar**, es decir, dividir por w , se obtiene la división por la profundidad que hace que los objetos lejanos se vean más pequeños. En proyección paralela este paso no es necesario, porque el volumen ya es un prisma.

La transformación completa resulta entonces $q' = D \cdot (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$.

Una vez normalizada la escena se realiza el **recorte**: se evalúan las coordenadas de los vértices transformados contra los intervalos del volumen canónico y se descarta lo que queda fuera; los bordes se recortan hallando la intersección con los planos mediante la forma paramétrica del segmento, y las superficies se intersectan con los planos generando nuevas superficies. Finalmente se **proyecta** descartando la coordenada z y se **mapea a coordenadas de pantalla** escalando el rango $[-1, 1]$ al

rango de píxeles del dispositivo, por ejemplo mediante $x' = 1023(x + 1)/2$ e $y' = 767(y + 1)/2$ para una pantalla de 1024×768 .

7.13 Equivalencia con la implementación

(Práctico 06 p.5-7.) Conecta la teoría con lo que se hace en los prácticos, y ayuda a fijar el proceso.

```
objeto -> mundo -> cámara -> coord.de recorte -> NDC -> pantalla
      M       V           P           ÷w      VP
      |----- glm_Position -----|   |--- función fija ---|
      vertex shader (programable)      (no se programa)
```

Teórico	OpenGL / glm
—	Model → <code>uModel</code>
$M^T \cdot T(-P_c)$	View → <code>glm::lookAt()</code>
$D \cdot (S_2)_{xyz} \cdot S_{xy}$	Projection → <code>glm::perspective()</code>
$q' = D \cdot (S_2) \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$	$MVP = P \cdot V \cdot M$

Por qué se divide en tres matrices: por **reutilización**, no por imposición de la API. P cambia al redimensionar (casi nunca); V cambia una vez por cuadro; M cambia una vez por objeto.

Concepto importante: "La matriz de vista no ubica la cámara: es la inversa de la pose de la cámara" (Práctico 06 p.9). Si se usara la pose directamente, la escena se desplazaría **exactamente en sentido contrario** al esperado.

Multiple Choice de práctica — Unidad VII

1. Una proyección geométrica se define como el mapeo de puntos de un espacio 3D a un plano 2D mediante líneas de proyección que:

A) Siempre convergen en un centro de proyección B) Convergen en un centro de proyección **o** son paralelas entre sí C) Son siempre perpendiculares al plano de proyección D) Parten del objeto en dirección normal a su superficie

2. En una proyección **paralela**, el centro de proyección se encuentra:

A) En el plano de proyección B) En el infinito C) A la distancia focal de la cámara D) En el origen del sistema de coordenadas del mundo

3. Las proyecciones paralelas se utilizan en ingeniería y arquitectura porque:

A) Producen imágenes de aspecto natural B) Mantienen las proporciones relativas C) Requieren menos parámetros de cámara D) Permiten definir tres puntos de fuga

4. La diferencia entre una proyección ortogonal y una oblicua es:

A) La posición del centro de proyección B) El ángulo que forman las líneas de proyección con el plano de proyección C) El número de puntos de fuga D) Si se conserva o no la relación de aspecto

5. En una proyección **isométrica**:

A) Los ángulos entre los tres ejes son iguales (120°) y las dimensiones se escalan igual en los tres ejes B) Dos ángulos son iguales y las dimensiones se escalan igual en dos ejes C) Los tres ángulos y las tres escalas son diferentes D) Una cara se representa en verdadera forma y las demás se distorsionan

6. En la proyección *caballera (cavalier)*:

A) $\alpha \approx 63,4^\circ$ y las dimensiones perpendiculares al plano se reducen a la mitad B) $\alpha = 45^\circ$, $L_1 = 1$, y las dimensiones perpendiculares al plano no cambian de magnitud C) Las líneas de proyección son perpendiculares al plano D) El centro de proyección está a distancia finita

7. La diferencia entre cavalier y cabinet está determinada por el valor de:

A) ϕ , la dirección en el plano de proyección B) $L_1 = \cot \alpha$, que vale 1 y 0,5 respectivamente C) La relación de aspecto de la ventana D) La distancia entre los planos de recorte

8. Un *punto de fuga* es el punto donde convergen las proyecciones de líneas paralelas que:

A) Son paralelas al plano de proyección B) **No** son paralelas al plano de proyección C) Pasan por el centro de proyección D) Coinciden con los planos de recorte

9. Una perspectiva de *dos puntos de fuga* se produce cuando:

A) Dos ejes principales del objeto intersectan el plano de proyección B) El objeto tiene dos caras visibles C) Se usan dos centros de proyección D) Dos ejes principales son paralelos al plano de proyección

10. Lo que hace que en perspectiva los objetos lejanos se vean más chicos es:

A) El recorte contra el plano trasero B) La división por $(z_{pp} - z)$, que hace que la transformación no sea lineal en z C) El escalado S_{xy} de la normalización D) La relación de aspecto del volumen de visualización

11. Los objetos que *intersectan* un plano del volumen de visualización son objeto de:

A) Culling B) Clipping C) Homogeneización D) Escorzo

12. ¿Cuál de estos parámetros de la cámara sintética la propia *filmina* define como *opcional*?

A) El vector Up B) La relación de aspecto C) La distancia focal D) El plano de recorte trasero

13. La justificación del *plano de recorte frontal* incluye que:

A) Los objetos muy lejanos serían visualmente insignificantes B) Los objetos detrás de la cámara aparecerían invertidos en perspectiva C) Permite calcular la relación de aspecto D) Define la distancia focal del sistema

14. El vector Up de la cámara sintética:

A) Indica hacia dónde apunta la cámara B) Establece cómo se gira la cámara alrededor del vector dirección, fijando la orientación del plano de proyección C) Define la distancia al plano de recorte frontal D) Determina el ángulo de visión vertical

15. El volumen de visualización de una proyección en perspectiva, una vez cerrado por los planos de recorte, tiene forma de:

A) Prisma rectangular B) Pirámide truncada (*frustum*) C) Cono truncado D) Esfera de radio far

16. La *visualización canónica* se define con la cámara en el origen, Look at en $(0, 0, -1)$ y Up en $(0, 1, 0)$, porque:

A) Es la única posición en la que la proyección es exacta B) El recorte contra planos alineados con los ejes es trivial y proyectar se reduce a descartar z C) Permite eliminar la necesidad de coordenadas homogéneas D) Minimiza el error de redondeo en punto flotante

17. En la transformación de normalización, la razón por la que las dos primeras transformaciones son **inversas** es:

A) Para deshacer la transformación de modelado aplicada previamente B) Porque se aplica a los vértices de la escena y no a la cámara: la cámara no se mueve, se mueve la escena C) Porque el sistema levógiro requiere invertir el signo de z D) Para compensar la división por w posterior

18. La propiedad $M^{-1} = M^T$ de las matrices de rotación se cumple porque:

A) Su determinante vale 1 B) Sus columnas son vectores unitarios y mutuamente perpendiculares C) Son matrices diagonales D) Sus filas suman 1

19. Al construir la base (u, v, w) de la cámara, **no** hace falta normalizar v porque:

A) v es siempre el vector $(0, 1, 0)$ B) $v = w \times u$, y el producto cruz de dos unitarios perpendiculares ya es unitario C) La normalización de v se realiza automáticamente al construir M^T D) v no interviene en la matriz de vista

20. La matriz D de la normalización se aplica:

A) Siempre, tanto en proyección paralela como en perspectiva B) Sólo en proyección en perspectiva, porque en paralela el volumen ya es un prisma C) Sólo en proyección paralela D) Sólo cuando la relación de aspecto es distinta de 1

21. El rasgo distintivo de la matriz D es que:

A) Es diagonal B) Su última fila no es $(0, 0, 0, 1)$, por lo que genera un w dependiente de z C) Su determinante es cero D) Es la inversa de S_{xy}

22. En el recorte de bordes, el parámetro t obtenido debe verificar:

A) $t > 1$ para que la intersección exista B) $0 \leq t \leq 1$ para que la intersección caiga dentro del segmento C) $t = 0$ exactamente D) t negativo si el punto está fuera del volumen

23. Al mapear el rango $[-1, 1]$ a una pantalla de 1024×768 , el factor correcto para x es:

A) 1024 B) 1023 C) 512 D) 2048

24. En la equivalencia con OpenGL, la matriz de **proyección** (`glm::perspective`) corresponde a:

A) Sólo la matriz D B) $D \cdot (S_2)_{xyz} \cdot S_{xy}$, es decir, los tres factores de proyección C) $M^T \cdot T(-P_c)$ D) La matriz de modelo compuesta con la de vista

Respuestas y explicación — Unidad VII

1. B. La definición contempla **las dos posibilidades**, que son justamente las que dan origen a la clasificación en perspectiva y paralela.

2. B. Es la característica que define a la proyección paralela (Unidad VII-1 p.5).

3. B. "Porque mantienen las proporciones relativas" (Unidad VII-1 p.5). La opción A corresponde a la perspectiva.

4. B. Es el **segundo criterio** de clasificación: ortogonales con rayos normales al plano, oblicuas con ángulo $\neq 90^\circ$. La opción A es el **primer** criterio, que separa paralela de perspectiva.

5. A. (Unidad VII-1 p.9). La B es dimétrica y la C trimétrica.

6. B. (Unidad VII-1 p.13). La opción A describe la **cabinet**.

7. B. $L_1 = \cot \alpha$ controla cuánto se acorta la profundidad. ϕ (opción A) sólo decide la **dirección** del dibujo.

- 8. B.** Definición literal. Las líneas **paralelas al plano** no producen punto de fuga.
- 9. A.** Se cuentan los ejes principales que **intersectan** el plano (Unidad VII-1 p.15).
- 10. B.** La perspectiva **no es lineal en z**; por eso requiere homogéneas y división por w (Unidad VII-1 p.16).
- 11. B.** Clipping = recortar lo que intersecta; culling = descartar lo que está completamente afuera (Unidad VII-1 p.28).
- 12. C.** "Característica opcional para el modelo de cámara" (Unidad VII-1 p.30).
- 13. B.** Entre las justificaciones del plano frontal está que los objetos detrás de la cámara **aparecerían invertidos** en perspectiva (Unidad VII-1 p.29). La opción A justifica el plano **trasero**.
- 14. B.** El Look at indica hacia dónde apunta (opción A); el **Up** fija el giro alrededor de esa dirección (Unidad VII-1 p.23).
- 15. B.** Pirámide infinita $\rightarrow$ cerrada por los planos $\rightarrow$ **pirámide trunca (frustum)**. El prisma (A) corresponde a la paralela.
- 16. B.** Las dos razones que da la filmina (Unidad VII-2 p.5).
- 17. B.** Es el concepto central: "se aplica a todos los vértices de los objetos para preservar las distancias entre cámara y escena" — la cámara nunca se mueve.
- 18. B.** De la ortonormalidad de las columnas se demuestra $M^T M = I$ (Unidad VII-2 p.11).
- 19. B.** w y u son unitarios y perpendiculares, entonces $\|w \times u\| = 1$ (Unidad VII-2 p.16). En cambio u **sí** debe normalizarse, porque Up no es necesariamente perpendicular a w .
- 20. B.** En paralela el volumen ya es un prisma: no hay pirámide que deformar.
- 21. B.** Es lo que genera la coordenada homogénea dependiente de z y, al homogeneizar, produce la perspectiva.
- 22. B.** Si t cae fuera de $[0, 1]$, la intersección con el plano existe pero **fuera del segmento**.
- 23. B.** $x' = 1023(x + 1)/2$: los índices de píxel van de 0 a $W - 1$ (Unidad VII-2 p.27).
- 24. B.** "perspective no es D: es $D \cdot (S_2)_{xyz} \cdot S_{xy}$, los tres factores de proyección" (Práctico 06 p.7). La opción C es la matriz de **vista**.

Preguntas de desarrollo posibles — Unidad VII

- Indique cuáles son las especificaciones mínimas necesarias para un modelo de cámara sintética para una visualización en perspectiva.
- Describa el proceso y las transformaciones necesarias para convertir una visualización 3D arbitraria en una imagen 2D que pueda mostrarse en un dispositivo de visualización.
- Explique el concepto de proyección geométrica y describa la clasificación de las proyecciones planares.
- Compare las proyecciones paralelas con las proyecciones en perspectiva.
- Explique qué son los puntos de fuga y qué determina su cantidad.
- Describa las proyecciones paralelas oblicuas y compare la caballera con la militar.
- Explique qué es la visualización canónica y justifique por qué se la utiliza.
- Explique cómo se construye la base ortonormal (u, v, w) de la cámara a partir de los vectores Look at y Up, y por qué la matriz de rotación se utiliza transpuesta.
- Justifique el uso de los planos de recorte frontal y trasero, e indique la diferencia entre culling y clipping.

10. Explique cómo se realiza el recorte de bordes y el mapeo a coordenadas de pantalla.

Guía de respuestas — Unidad VII

1. Definición de cámara sintética y modelo pinhole; el modelo conceptual del proceso; la lista de **ocho** especificaciones con qué define cada una; aclarar que la distancia focal es **opcional**; que el volumen en perspectiva es una **pirámide trunca**; y la distinción culling/clipping con la justificación de los planos de recorte.

2. El problema y la estrategia; los tres pasos del procedimiento; la definición del volumen canónico con su justificación; los cuatro pasos de la normalización con **qué matriz corresponde a cada uno y qué hace**; la aclaración de que se aplica a los vértices y no a la cámara; la construcción de (u, v, w) ; la propiedad $M^{-1} = M^T$; la fórmula final completa; y el recorte, la proyección y el mapeo a pantalla.

3. Definición; los cuatro elementos; los dos criterios de clasificación; el árbol completo; características de cada tipo.

4. Tabla: posición del centro, dirección de los rayos, conservación de proporciones, realismo, aplicaciones, costo computacional. Cerrar con la diferencia fundamental: la división por la profundidad.

5. Definición (líneas paralelas que **no** son paralelas al plano); que los principales corresponden a los ejes X , Y , Z y controlan la orientación de la vista; la clasificación 1/2/3 según cuántos ejes **intersectan** el plano.

6. Definición (plano paralelo a un plano del sistema, rayos oblicuos); características; los parámetros α y ϕ con qué controla cada uno; las ecuaciones generales con $L_1 = \cot \alpha$; la tabla comparativa cavalier/cabinet con valores; y por qué cabinet es más realista.

7. El problema (proyectar y recortar en vista arbitraria es difícil); la definición completa de la visualización canónica; las **dos** razones: recorte trivial contra planos alineados y proyección reducida a descartar z .

8. Las condiciones que debe cumplir el sistema; los tres pasos con sus fórmulas y **la justificación de cada decisión**: el signo menos en w , el orden $Up \times w$ por mano derecha, la normalización de u y por qué v no la necesita. Para la transpuesta: M con columnas rota $(x, y, z) \rightarrow (u, v, w)$ y se necesita lo inverso; la propiedad de ortonormalidad da $M^{-1} = M^T$; y la ventaja computacional.

9. El motivo general (costo computacional de plasmar la escena); las dos justificaciones del plano frontal y las dos del trasero; la tabla de las tres situaciones con culling, clipping y dibujo.

10. Recorte de vértices contra los intervalos; forma paramétrica del segmento; el despeje de t con el ejemplo; la condición $0 \leq t \leq 1$; recorte de superficies; proyección descartando z ; el mapeo con $(W - 1)$ y $(H - 1)$; y la observación de que en paralela da igual proyectar sobre el plano frontal o trasero.

BLOQUE PRÁCTICO — Conceptos que iluminan la teoría

Prioridad MEDIA. El profesor indicó que la evaluación está **principalmente orientada a teoría**. Esta sección es deliberadamente breve: incluye sólo aquello que **ayuda a comprender la teoría** o que puede aparecer como pregunta conceptual.

Los prácticos mencionados el año anterior

El año pasado se mencionaron como prácticos el **"Práctico 2.2 de la guía 1"** y el **"Práctico 4 de la guía 2"**.

⚠ **Nota sobre la correspondencia.** Las guías de **2026** no usan esa numeración: se organizan en "Partes". La correspondencia más razonable es:

Referencia anterior	Correspondencia probable en 2026	Qué ejercita
Práctico 2.2 de la guía 1	Guía-practico-01.pdf — Parte 2: el módulo <code>ResourceManager</code>	Gestión de recursos en C++ puro: leer shaders de disco, cachear, avisar los errores
Práctico 4 de la guía 2	Guía-practico-02.pdf — Parte 4: "El programa que los usa"	Armar el <code>main.cpp</code> con las cuatro etapas del dibujado, usando <code>glDrawElements</code>

No puedo confirmar esta correspondencia: la numeración del año anterior no coincide con la de las guías entregadas. Lo señalo como inferencia.

Qué conceptos teóricos ejercitan estos prácticos

El de la Guía 1, Parte 2 (ResourceManager) es C++ puro, sin OpenGL. Conceptualmente ejercita **arquitectura y separación de responsabilidades**, no computación gráfica. Su valor teórico es escaso para un parcial de teoría.

El de la Guía 2, Parte 4 sí ilumina la teoría, porque hace explícitas **las cuatro etapas del dibujado**:

1. **Decidir los datos** — los vértices y sus atributos.
2. **Enviar los bytes a la GPU y decir cómo se leen** — VBO (los bytes) y VAO (el instructivo de lectura).
3. **Armar el programa que los va a procesar** — compilar y linkear los shaders.
4. **Dar la orden** — `glDrawElements`, que es donde **aparece el primer píxel**.

Conceptos prácticos con valor teórico

El pipeline programable

Tres puntos programables (los datos, el vertex shader y el fragment shader) y **cuatro etapas fijas** (ensamblado de primitivas, recorte, rasterizado y escritura en el framebuffer).

Conexión clave con la Unidad II: "En la Unidad II, trazar el segmento con Bresenham decidía, píxel por píxel, cuáles se prenden. Esa decisión se escribía. Hoy se le pide un triángulo a la GPU: la decisión es la misma, pero ya no se escribe: está fija en el hardware, dentro de una etapa que no se programa." (Práctico 01 p.3-4)

Qué es un shader

[NOTA DE CLASE] (Clase 6 p.1) — "Un programa que corre una vez por vértice o una vez por fragmento. Corren en paralelo, o sea la invocación es una pero muchos núcleos. Cada invocación no sabe nada de la otra invocación que se está ejecutando en paralelo."

Consecuencia conceptual: **adentro de un shader no hay un bucle sobre todos los vértices ni una variable que se acumule entre invocaciones**. Se escribe el comportamiento para **una** invocación.

GLSL (OpenGL Shading Language): sintaxis parecida a C, con **vectores y matrices como tipos nativos**. Lo que **no** tiene: punteros, `new` / `malloc`, recursión, ni librería estándar de C.

La conexión con coordenadas homogéneas

En el vertex shader mínimo aparece `gl_Position = vec4(aPos, 1.0);`. Ese `1.0` es la **coordenada homogénea** $h = 1$ de la Unidad VI: el constructor arma un `vec4` a partir de un `vec3` más el 1.

La conexión con la Unidad VII

Durante todo el cuatrimestre los prácticos usaron una constante `uAjuste` como "caja negra", que resultó ser `scale(alto/ancho, 1, -1)`: **sólo un escalado** que corrige la relación de aspecto e invierte z. No había perspectiva.

Lo que le faltaba para ser una perspectiva real (Práctico 06 p.13): el $\cot(\theta_h/2)$ —por eso no había campo visual— **y la cuarta fila, la que hace $w = -z$** : sin ella no hay división por w , y **sin división por w no hay perspectiva**.

Composición jerárquica y pose

$$M_{\text{mundo}}(\text{pieza}) = M_{\text{pose}} \cdot M_{\text{local}}(\text{pieza}) \quad M_{\text{pose}} = T(\text{posición}) \cdot R(\text{ángulos}) \cdot T(-\text{ref})$$

Es una aplicación directa de la **composición de transformaciones** de la Unidad VI, y de las estrategias de **organización y composición** de la Unidad V. El $T(-\text{ref})$ va **a la derecha** porque es lo primero que ocurre: lleva el punto de referencia al origen antes de rotar.

PARCIAL DEL AÑO ANTERIOR — RESPUESTAS MODELO

Pregunta 1

Realice una comparación de los dos displays precursores de los dispositivos de visualización modernos: display caligráfico o vectorial y display digital de trama o raster.

Respuesta modelo

Ambos displays derivan del **tubo de rayos catódicos**, en el que un cañón de electrones emite un rayo dirigido por elementos deflectores que, al impactar en una película de fósforo, la hace emitir luz. La diferencia está en **cómo se controla ese rayo y cómo se representa la imagen**.

El **display caligráfico o vectorial** surge de la modificación que Sutherland realizó al CRT en 1963: alteró el control de los elementos deflectores y el de la emisión del rayo, de modo de poder **dibujar líneas en cualquier orden**, con la misma metodología que un pen-plotter. La imagen se almacena como una **lista de comandos** en memoria, que controla el rayo de electrones. Como el fósforo se apaga, esa lista debe **re-ejecutarse en un loop** para que las líneas persistan en pantalla.

De esa arquitectura se desprende su limitación central: **la velocidad de actualización depende de la cantidad de comandos**, es decir, de la complejidad de la imagen representada. El límite práctico es que la actualización debe ser de **30 a 60 veces por segundo** para evitar el problema del **parpadeo**; con imágenes complejas ese límite se supera y la imagen titila. Además, **rellenar formas resulta muy costoso**, porque hay que generar nuevos segmentos de línea para cubrir el área.

El **display digital de trama o raster** parte de interpretar la imagen como una **secuencia de intensidades de luz** que puede muestrearse. El proceso de **barrido** consiste en muestrear una secuencia de funciones de intensidad continuas, generando una función por línea de la imagen. Su hardware es similar al del vectorial pero incorpora una **máscara de sombra**, requiere precisión geométrica del rayo y exige mantener correspondencia entre la persistencia del fósforo, la relación de aspecto y la resolución; con tres cañones se obtienen imágenes en color.

En su funcionamiento, el rayo **recorre toda la pantalla con un patrón regular**. Esto lo convierte en un **proceso determinístico e invariante**: no depende de la complejidad de la imagen ni del código de pintado, y el tiempo se mantiene constante, lo que permite repintar la pantalla 60 veces por segundo. La imagen se almacena en el **buffer de refresco** como una matriz de bits. Sus ventajas son las transiciones suaves y la mayor resolución, y puede acelerarse duplicando los cañones en modo entrelazado.

Resumiendo la comparación: el vectorial **dibuja segmentos de línea** y el raster **dibuja puntos**; en el vectorial la velocidad de pintado **depende de la complejidad** y en el raster es **independiente**; en el vectorial el relleno es **muy costoso** y en el raster es **sencillo y sin costo adicional**. El problema típico del vectorial es el parpadeo, mientras que el del raster es el **aliasing**, es decir, la apariencia escalonada producto de la discretización, además del consumo de memoria del frame buffer.

La diferencia de fondo es que en el vectorial **la imagen es un procedimiento** —una lista de instrucciones que hay que ejecutar—, mientras que en el raster **la imagen es un dato** almacenado en memoria que se recorre siempre del mismo modo. Ese cambio es lo que independiza el costo de la complejidad y hace gratuito el relleno, y a la vez lo que introduce la discretización y con ella el aliasing.

Conceptos que no puedo olvidarme

- Los dos derivan del CRT; lo que cambia es **cómo se controla el rayo**.
- Vectorial: **lista de comandos** re-ejecutada en loop; velocidad **dependiente de la complejidad**; **30-60 veces/s** o hay parpadeo; relleno muy costoso.
- Raster: **barrido con patrón regular**; proceso **determinístico e invariante**; tiempo constante; relleno sencillo y sin costo adicional.
- Raster incorpora **máscara de sombra** y almacena la imagen en el **buffer de refresco**.
- Problemas típicos: **parpadeo** vs **aliasing**.

De dónde estudiarlo

CGyAV-2026-Unidad-I (1).pdf p.12-14 (CRT y vectorial), p.15-19 (raster), p.25 (tabla resumen) · Clase_2 p.2 (notas) · CGyAV-2026-Unidad-II-1 (1).pdf p.3 (repaso).

Posibles variantes de la pregunta

- "Indique ventajas y limitaciones del display caligráfico frente al display raster."
- "Explique por qué en un display vectorial la velocidad de actualización depende de la complejidad de la imagen y en uno raster no."
- "¿Qué papel cumple el buffer de refresco en cada uno de estos sistemas?" (pista: el vectorial guarda **comandos**, el raster guarda **bits**)
- "Explique por qué el rellenado de áreas es costoso en un sistema vectorial y sencillo en uno raster."

Pregunta 2

Explique qué es un buffer de refresco en los dispositivos de trama, indicando su función en la arquitectura del dispositivo.

Respuesta modelo

El **buffer de refresco** o frame buffer es la memoria que **almacena la imagen en formato de bits**, es decir, un valor por cada píxel de la pantalla. Conceptualmente se ubica **entre el proceso que genera la imagen y el proceso que la pinta**.

Su existencia responde a un problema concreto. Los **primeros sistemas raster** utilizaban un **único controlador** tanto para la generación como para el pintado de la imagen, lo que obligaba a **sincronizar ambas velocidades** y limitaba la velocidad de cambio de imágenes: el componente más lento imponía su ritmo al otro.

La solución fue **ampliar la arquitectura** separándola en tres dispositivos: un **controlador de display**, encargado de la **generación** de la imagen; un **controlador de video**, encargado del **control del pintado**; y el **buffer de refresco**, que **almacena la imagen en formato de bits** entre ambos. De este modo se **permite tener velocidades de generación y pintado diferentes**, que es exactamente la función que cumple el buffer dentro de la arquitectura.

Su rol en el refresco es central: como el fósforo de la pantalla se apaga, el controlador de video debe **releer el buffer** para repintar la imagen, típicamente sesenta veces por segundo. El buffer es entonces la **fuentes de datos de cada uno de esos repintados**, y su contenido determina lo que se ve.

Según su organización, el buffer puede ser **bi-level de 1 bit**, la implementación más simple, en la que el fósforo está encendido o apagado y las imágenes son monocromáticas; **n-level de n bits**, que permite 2^n intensidades, de modo que un cañón de 8 bits produce 256 tonos de gris y tres cañones de 8 bits producen $2^{24} \approx 16,7$ millones de colores; **con tabla de colores** o look-up table, donde se aprovecha que una imagen no utiliza todos los colores posibles: se los organiza en una tabla separada y el buffer almacena únicamente los **índices** a esa tabla, ahorrando memoria; o **"true color" de 24 bits**, donde cada canal R, G y B tiene su propio frame buffer de 8 bits con su propia tabla de colores.

Finalmente, cuando las escenas son complejas y tienen objetos en movimiento, los tiempos de procesamiento altos pueden generar errores visuales denominados **artifacts**. Para evitarlos se emplea la estrategia del **doble frame buffer**: el controlador de video **lee del front buffer** mientras el dispositivo de procesamiento **escribe en el back buffer**, y ambos se intercambian al completar el cuadro. Esto puede implementarse tanto a nivel de hardware como de software, y evita el **screen tearing**, el artifact en el que la pantalla muestra partes de dos cuadros distintos simultáneamente por desincronización entre lectura y escritura.

Conceptos que no puedo olvidarme

- **Qué contiene:** la imagen en **formato de bits**, un valor por píxel.
- **Qué problema resuelve:** la sincronización entre generación y pintado en la arquitectura de **un solo controlador**.
- **Los tres componentes:** controlador de **display** (genera), controlador de **video** (pinta), **buffer de refresco** (almacena).
- **Función en el refresco:** el controlador de video lo **relee** porque el fósforo se apaga.
- Los **cuatro tipos** con la fórmula 2^n y los números concretos.
- **Doble buffer:** front = lee video, back = escribe procesamiento; evita el **screen tearing**.

De dónde estudiarlo

CGyAV-2026-Unidad-I (1).pdf p.20 (arquitectura), p.21-22 (tipos), p.23-24 (doble buffer y tearing) · Clase_2 p.1-2.

Posibles variantes de la pregunta

- "Describa la arquitectura de hardware de un sistema raster y justifique por qué fue necesario separarla en distintos componentes."
 - "Explique qué tipos de buffer de refresco existen y qué ventaja aporta cada uno."
 - "¿Qué es una tabla de colores y por qué permite ahorrar memoria?"
 - "Explique qué es el screen tearing, cuál es su causa y cómo se lo evita."
 - "¿Cuántos colores puede representar un frame buffer de tres cañones de 8 bits? Justifique."
-

Pregunta 3

Describa a qué hace referencia el concepto de pipeline de rendering en computación gráfica y realice un esquema general del mismo indicando las operaciones en cada etapa del pipeline.

Respuesta modelo

El **pipeline de rendering** es el conjunto de procesos que permiten visualizar un modelo 3D abstracto por medio de una representación 2D. Consiste en la transformación del modelo matemático del mundo en píxeles en la pantalla. Este proceso requiere múltiples pasos que se realizan en secuencia, donde la información de salida de una etapa sirve como información de entrada de la siguiente.

El esquema general es el siguiente:

Modelado del Proyección Modelo de Muestreo / Frame Dispositivo mundo virtual --> 2D --> imagen 2D --> procesa- --> buffer --> de 3D miento visualización

En la etapa de **modelado del mundo virtual**, en tres dimensiones, se construye la **descripción geométrica** de los objetos de la escena mediante alguno de los métodos de representación disponibles, y se los **ubica, orienta y dimensiona** aplicando transformaciones geométricas. Recibe la descripción del objeto a representar y **produce la escena 3D completa en coordenadas del mundo**. Es necesaria porque sin una descripción de las formas no hay nada que visualizar.

En la etapa de **proyección** se especifica un observador mediante los parámetros de una **cámara sintética** —posición, dirección, orientación, tipo de proyección, campo de visión y planos de recorte—. Se determina qué objetos caen dentro del **volumen de visualización**, se **descartan** los que quedan completamente fuera y se **recortan** los que lo intersectan, y finalmente se **proyecta** la geometría sobre un plano. Recibe la escena 3D y **produce primitivas bidimensionales**. Es necesaria porque hay que reducir las tres dimensiones del mundo a las dos de la pantalla.

El **modelo de imagen 2D** es la descripción resultante, todavía en términos **continuos**: segmentos de recta, circunferencias, polígonos. Constituye la frontera entre el dominio continuo y el discreto.

La etapa de **muestreo y procesamiento** realiza la **digitalización o conversión por barrido** de esas primitivas: decide **qué píxeles deben encenderse** para aproximar cada primitiva, mediante algoritmos como el de Bresenham o el de punto medio; **rellena** las primitivas de área mediante algoritmos de scanline o de relleno por semilla; y aplica técnicas de **antialiasing** para compensar la pérdida de información propia del muestreo. Recibe el modelo continuo y **produce una matriz de valores de píxel**. Es necesaria porque la pantalla es una grilla discreta y no un plano continuo.

Esos valores se escriben en el **frame buffer**, que **almacena la imagen en formato de bits y desacopla** la velocidad de generación de la de pintado.

Finalmente, el **dispositivo de visualización lee repetidamente** el contenido del buffer y **refresca la pantalla**, ya que el fósforo se apaga y la imagen debe repintarse.

Cabe destacar que cada unidad de la materia corresponde a una etapa de este pipeline: el modelado de objetos a la Unidad V, las transformaciones geométricas a la VI, la proyección y visualización 3D a la VII, la rasterización y el antialiasing a la II, y el almacenamiento y la visualización a la I.

Conceptos que no puedo olvidarme

- **La definición textual** y las tres propiedades: múltiples pasos, en secuencia, salida de una = entrada de la siguiente.
- **El esquema hay que dibujarlo**: la consigna lo pide explícitamente.
- Para **cada etapa**: qué recibe, qué operación hace, qué produce.
- La correspondencia **etapa ↔ unidad de la materia**.
- Que el recorte pertenece a la etapa de **visualización**, no al modelado.

De dónde estudiarlo

CGyAV-2026-Unidad-II-1 (1).pdf p.4-5 · CGyAV-2026-Unidad-V (1).pdf p.2 · CGyAV-2026-Unidad-VI.pdf p.2 · CGyAV-2026-Unidad-VII-1.pdf p.2 · 01-...Presentacion practico... p.4 · Clase_3 p.1 · Clase_7 p.3.

Posibles variantes de la pregunta

- "Indique qué operación se realiza en la etapa de muestreo del pipeline y por qué es necesaria."
- "Explique en qué etapa del pipeline intervienen los algoritmos de Bresenham y de relleno de polígonos."
- "Describa el pipeline de rendering en su versión programable, indicando qué etapas son fijas y cuáles programables."
- "Relacione cada unidad del programa de la materia con la etapa del pipeline que le corresponde."

Pregunta 4

Explique cómo funcionan los algoritmos de relleno de áreas: relleno de contorno y relleno por difusión.

Respuesta modelo

Los **algoritmos de relleno de áreas** se utilizan para rellenar figuras con **bordes curvos o irregulares** que no son polígonos y, por lo tanto, no pueden tratarse mediante el algoritmo de scanline. Usualmente son empleados por artistas gráficos, que bosquejan los bordes de una figura y luego seleccionan el color o patrón de relleno desde un menú; en los paquetes de pintado interactivo constituyen la herramienta conocida como **balde de pintura**.

Ambos algoritmos comparten la misma mecánica general: **comienzan con un punto conocido dentro de la figura** —el punto semilla— y **proceden a rellenar hacia el exterior** de ese punto, expandiéndose por **conectividad**. La conectividad puede ser **4-conectada**, considerando los vecinos de arriba, abajo, izquierda y derecha, u **8-conectada**, que incluye además las diagonales. Lo que distingue a un algoritmo del otro es **la condición que utilizan para detenerse**.

El **relleno de contorno** (boundary fill) parte del punto interior conocido y **expande hasta encontrar el color de contorno**. En cada píxel lee su color actual y verifica **dos condiciones**: que ese color **no sea el color de contorno**, que es la condición que efectivamente **detiene** la expansión al llegar al borde de la figura; y que **no sea el color de relleno**, que impide volver sobre píxeles ya pintados y caer en una recursión infinita. Si ambas condiciones se cumplen, pinta el píxel con el color de relleno y se llama recursivamente sobre sus píxeles vecinos.

Sus **ventajas** son que funciona con cualquier forma de contorno, que su implementación es intuitiva y que resulta ideal para aplicaciones interactivas. Sus **desventajas** son que requiere un stack recursivo profundo, que puede resultar lento para áreas grandes y que presenta problemas si hay píxeles previamente coloreados dentro de la región.

El **relleno por difusión o inundación** (flood fill) se diferencia en que **no busca un color de contorno específico**, sino que **reemplaza el color interior existente**. En cada píxel verifica si su color **coincide con el color interior** que se desea reemplazar; si coincide, lo pinta con el color de relleno y continúa recursivamente sobre sus vecinos. Se detiene ante **cualquier píxel cuyo color no sea el interior buscado**, sin importar cuál sea.

Esto lo hace especialmente útil para **regiones cuyo borde es multicolor**, donde el boundary fill no podría aplicarse porque no existe un único color de contorno. Sus aplicaciones típicas son el cambio de color en regiones existentes, la herramienta de balde de pintura y la selección de regiones similares. Debe verificarse que el color de relleno sea **distinto** del color interior, ya que de lo contrario también se produce recursión infinita; y deben manejarse los límites del área de dibujo.

En ambos casos, la recursión píxel a píxel consume gran cantidad de memoria de stack, por lo que se aplica una **optimización por spans**: en lugar de apilar cada píxel individualmente, se **rellenan líneas horizontales completas** y se **apilan únicamente las posiciones iniciales de los spans** correspondientes a las filas superior e inferior, reduciendo notablemente el uso de memoria.

En síntesis, la diferencia esencial entre ambos está en la condición de detención: el **boundary fill mira hacia el borde** y avanza mientras no encuentre el color de contorno, por lo que requiere un contorno de color definido y uniforme; el **flood fill mira hacia adentro** y avanza mientras el píxel sea del color interior que reemplaza, por lo que tolera bordes multicolor pero exige un interior homogéneo.

Conceptos que no puedo olvidarme

- Ambos parten de un **punto semilla interior** y expanden **hacia afuera** por **conectividad** (4 u 8).

- **Boundary fill:** se detiene en el **color de contorno**; **doble condición** ($\neq$ contorno **y** $\neq$ relleno) y para qué sirve cada una.
- **Flood fill:** **reemplaza un color interior**; se detiene ante **cualquier** otro color; sirve para bordes multicolor.
- Ventajas y desventajas de cada uno.
- La **optimización por spans** y por qué hace falta.
- La diferencia esencial está en la **condición de detención**.

De dónde estudiarlo

CGyAV-2026-Unidad-II-2 (1).pdf p.18 (conectividad), p.19-20 (boundary fill), p.21-22 (flood fill).

Posibles variantes de la pregunta

- "Compare el relleno de contorno con el relleno por difusión, indicando en qué situaciones conviene cada uno."
- "¿Qué es la conectividad de píxeles y qué consecuencias tiene elegir 4 u 8 conectividad?"
- "Explique por qué el algoritmo de relleno de contorno verifica dos condiciones y qué ocurriría si se omitiera una de ellas."
- "Explique la optimización por spans en los algoritmos de relleno por semilla."
- "Explique el algoritmo de relleno de polígonos por scanline y compárelo con los algoritmos de relleno por semilla." (variante que **amplía** hacia el otro grupo de algoritmos)

Pregunta 5

Dadas matrices de transformación en coordenadas homogéneas: indicar qué tipo de transformación representa cada una; explicar qué efecto producirá una composición $D = A \cdot B \cdot C$ sobre los puntos de un objeto.

Respuesta modelo

Identificación de las matrices. En coordenadas homogéneas, cada tipo de transformación tiene una **firma reconocible**.

Una matriz de **traslación** presenta la **diagonal principal con unos** y los desplazamientos t_x , t_y (y t_z en 3D) ubicados en la **última columna**. Su efecto es **sumar un vector constante** a las coordenadas de cada punto: **todos los puntos se mueven la misma distancia**. Modifica la posición del objeto, pero **preserva su forma, su tamaño y su orientación**, por lo que constituye una **transformación de cuerpo rígido**.

Una matriz de **rotación** se reconoce por el bloque formado por **senos y cosenos**, con el término $-\sin \theta$ ubicado arriba a la derecha en el caso de la rotación 2D o de R_z . El parámetro θ es el **ángulo de giro**, siendo la rotación **antihoraria** si $\theta > 0$ y **horaria** si $\theta < 0$. Su efecto es **girar los puntos alrededor del origen**, modificando su orientación y, salvo para el propio origen, también su posición. **Preserva distancias y ángulos**, por lo que también es una transformación de cuerpo rígido.

Una matriz de **escalado** es **diagonal**, con los factores s_x , s_y (y s_z) en la diagonal principal y la última columna nula. Su efecto es **multiplicar cada coordenada** por el factor correspondiente, modificando el **tamaño** del objeto. Si $s_x = s_y$ el escalado es **uniforme** y preserva las proporciones; si son distintos es **diferencial**. Valores mayores que uno amplían, menores que uno reducen, y valores **negativos producen además una reflexión**. Es importante notar que el escalado, al multiplicar **todas** las coordenadas incluidas las de posición, **también desplaza el objeto** si éste no se encuentra centrado en el origen; por ello **no es una transformación de cuerpo rígido**.

Como criterio general de identificación: si la **última columna** tiene valores no nulos hay traslación involucrada; si hay **senos y cosenos** hay rotación; si la matriz es **diagonal con valores distintos de uno** hay escalado; y si la **última fila no es** $(0\ 0\ 1)$ o $(0\ 0\ 0\ 1)$ la transformación **no es afín** y corresponde a una **proyección en perspectiva**, que requerirá homogeneizar dividiendo por w .

Efecto de la composición $D = A \cdot B \cdot C$. La composición de transformaciones se **lee de derecha a izquierda**: al aplicar D a un punto P , se obtiene $D \cdot P = A \cdot (B \cdot (C \cdot P))$, de modo que el punto es transformado **primero por** C , el resultado es transformado por B , y finalmente ese resultado es transformado por A . Es decir, **la matriz ubicada más a la derecha es la primera en aplicarse** y la ubicada más a la izquierda es la última.

La matriz D **resume las tres transformaciones en una sola**. Esto constituye la principal ventaja práctica de las coordenadas homogéneas: la matriz compuesta **se calcula una única vez** y luego **se aplica a todos los vértices del objeto**, en lugar de aplicar tres transformaciones sucesivas a cada uno.

Ahora bien, la multiplicación de matrices es **asociativa pero no conmutativa**, de modo que **el orden de aplicación importa**. Esto tiene una interpretación geométrica precisa: **cada transformación actúa sobre el resultado de la anterior**, y por lo tanto sobre un objeto que ya fue modificado. Una rotación aplicada después de una traslación hace girar al objeto **alrededor de un punto distinto** del que giraría si se aplicara antes.

El caso característico es el de una traslación y una rotación: la composición $T \cdot R$ **rota primero** y traslada después, de modo que la pieza **gira sobre sí misma** y luego se desplaza a su lugar; la composición $R \cdot T$ **traslada primero** y rota después, de modo que la pieza **orbita alrededor del origen**. Ninguna de las dos es incorrecta: **producen transformaciones distintas**.

Esta propiedad es la que obliga a respetar el patrón **"trasladar al origen, transformar, volver"** cuando la transformación debe realizarse respecto de un punto que no es el origen. Así, para rotar

alrededor de un punto arbitrario (x_r, y_r) la composición correcta es $M = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$, donde la traslación al origen aparece **a la derecha** por ser la primera en aplicarse.

Conceptos que no puedo olvidarme

- **Dónde mirar** para identificar cada matriz: última columna $\rightarrow$ traslación; senos/cosenos $\rightarrow$ rotación; diagonal $\rightarrow$ escalado; última fila distinta de $(0\ 0\ 0\ 1)$ $\rightarrow$ perspectiva.
- Qué **preserva** cada una: T y R son de cuerpo rígido; el escalado **no**.
- Que el escalado **también desplaza** si el objeto no está en el origen.
- **Se lee de derecha a izquierda**: C primero, A última.
- **Asociativa pero NO conmutativa**, con la interpretación geométrica de $T \cdot R$ vs $R \cdot T$.
- La ventaja: **una matriz calculada una vez, aplicada a todos los vértices**.

De dónde estudiarlo

CGyAV-2026-Unidad-VI.pdf p.6-10 (transformaciones básicas), p.11-14 (homogéneas e inversas), p.15 (composición), p.16-18 (ejemplos) · Clase_9 p.5-12 · 04-...Presentacion practico... p.12 ($T \cdot R$ vs $R \cdot T$).

Posibles variantes de la pregunta

- "Explique por qué se utilizan coordenadas homogéneas y qué ventajas aportan en la composición de transformaciones."
- "Justifique por qué la composición de transformaciones no es conmutativa, dando un ejemplo con interpretación geométrica."
- "Describa el procedimiento para rotar un objeto alrededor de un punto que no es el origen y escriba la matriz compuesta resultante."
- "Dada una matriz, indique qué transformación representa y qué le ocurre a la posición, la orientación y el tamaño de un objeto al aplicarla."
- "¿Qué indica que la última fila de una matriz de transformación 4×4 no sea $(0\ 0\ 0\ 1)$?"

Pregunta 6

Indique cuáles son las especificaciones mínimas necesarias para un modelo de cámara sintética para una visualización en perspectiva.

Respuesta modelo

(Ver la respuesta modelo completa desarrollada en la sección 7.7 La cámara sintética, que responde exactamente a esta consigna.)

En síntesis, las especificaciones son: **tipo de proyección** (perspectiva, lo que determina un volumen piramidal); **posición** de la cámara en coordenadas del mundo; **dirección** mediante el vector Look at; **orientación** mediante el vector Up; **campo de visión**, dado por la relación de aspecto y el ángulo de visión θ_h ; **profundidad de campo**, dada por los planos de recorte frontal y trasero, que cierran la pirámide convirtiéndola en un **frustum**; y opcionalmente la **distancia focal**. Con estos parámetros queda definido el sistema de coordenadas propio de la cámara (u, v, w) y el **plano de proyección**.

Conceptos que no puedo olvidarme

- La **lista completa**, y que la **distancia focal es opcional** (lo dice la filmina).
- Que Look at da la **dirección** y Up la **orientación** (el giro alrededor de la dirección).
- Que el volumen en perspectiva es una **pirámide**, que al cerrarse con los planos de recorte se vuelve **pirámide trunca (frustum)**.
- Las **justificaciones** de los dos planos de recorte.
- La diferencia **culling / clipping**.
- Que en **paralela no hacen falta los ángulos de visión**.

De dónde estudiarlo

CGyAV-2026-Unidad-VII-1.pdf p.18-21 (concepto y parámetros), p.22-23 (posición, dirección, orientación), p.24-27 (volumen y campo de visión), p.28-29 (planos de recorte), p.30-31 (focal y plano de proyección), p.32 (resumen) · CGyAV-2026-Unidad-VII-2.pdf p.2 y p.4.

Posibles variantes de la pregunta

- "Indique las especificaciones de una cámara sintética para una visualización paralela y señale en qué se diferencian de las de una perspectiva." (pista: **no hacen falta los ángulos de visión**; el volumen es un **prisma**)
- "Explique qué es el volumen de visualización y qué forma adopta según el tipo de proyección."
- "Justifique la necesidad de los planos de recorte frontal y trasero."
- "Explique la diferencia entre los vectores Look at y Up y qué define cada uno."
- "¿Qué es la relación de aspecto y cómo se relaciona con los ángulos de visión?"

Pregunta 7

Describa el proceso y las transformaciones necesarias para convertir una visualización 3D arbitraria en una imagen 2D que pueda luego ser mostrada en un dispositivo de visualización.

Respuesta modelo

(Ver la respuesta modelo completa desarrollada en la sección 7.12 Resumen del proceso completo, que responde exactamente a esta consigna.)

En síntesis, el proceso consiste en: **reducir la visualización arbitraria a una visualización canónica**, porque proyectar y recortar en una vista arbitraria es difícil. Los tres pasos son obtener los parámetros, transformar a la visualización canónica, y proyectar y recortar en ese volumen.

La **transformación de normalización** se compone de: $T(-P_c)$ (traslación inversa que lleva la cámara al origen); M^T (rotación inversa que alinea (u, v, w) con (x, y, z) , aprovechando que $M^{-1} = M^T$); S_{xy} (escalado que ajusta el volumen al ancho canónico mediante $\cot(\theta/2)$); $(S_2)_{xyz}$ (escalado proporcional de factor 1/far que lleva el plano trasero a $z = -1$); y, **sólo en perspectiva**, D (que deforma el frustum en prisma y genera la coordenada homogénea que, al **homogeneizar**, produce la división por la profundidad).

$$q' = D \cdot (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$$

Luego se **recorta** contra el volumen canónico, se **proyecta** descartando z , y se **mapea a coordenadas de pantalla**.

Conceptos que no puedo olvidarme

- **Por qué** se normaliza: recortar y proyectar en vista arbitraria es difícil.
- La **definición del volumen canónico** y sus dos justificaciones.
- **Los cuatro pasos con su matriz**, y que D es **sólo para perspectiva**.
- Que se aplica **a los vértices, no a la cámara**: la cámara no se mueve, se mueve la escena. Por eso las dos primeras son **inversas**.
- La construcción de (u, v, w) y **por qué** $M^{-1} = M^T$.
- Que la **última fila de** D genera el w que produce la perspectiva al homogeneizar.
- El recorte con el parámetro t y el mapeo con $(W - 1)$.

De dónde estudiarlo

CGyAV-2026-Unidad-VII-2.pdf completo: p.3 (procedimiento), p.5 (canónica), p.7 (secuencia), p.8 (traslación), p.9-13 (rotación y M^T), p.14-16 (u, v, w) , p.17-21 (escalados), p.22-23 (D), p.24-27 (recorte y proyección), p.28 (resumen) · 06-...Presentacion practico... p.4-9 (equivalencia).

Posibles variantes de la pregunta

- "Explique qué es la transformación de normalización y por qué se la utiliza."
- "Describa la visualización canónica y justifique la elección de sus parámetros."
- "Explique cómo se construye la matriz de vista a partir de los parámetros de la cámara sintética."
- "Indique qué diferencia existe entre la normalización de una proyección paralela y la de una perspectiva." (pista: **la paralela no lleva D**)
- "Explique el papel de la coordenada homogénea w en la proyección en perspectiva."
- "Describa cómo se realiza el recorte en el volumen canónico y por qué resulta más simple que en una vista arbitraria."

OJO CON ESTO EN EL MULTIPLE CHOICE

Confusiones reales detectadas en el material. Son los lugares donde un distractor plausible puede hacerte perder puntos.

Sobre displays y frame buffer

1. **El frame buffer NO dibuja ni genera la imagen: sólo la almacena.** Quien genera es el **controlador de display**; quien pinta es el **controlador de video**.
2. **La tabla de colores (LUT) NO aumenta la cantidad de colores** que el dispositivo puede mostrar. Reduce la cantidad de **bits por píxel** necesarios, porque el buffer guarda índices en lugar de colores. La ganancia es de **memoria**, no de gama.
3. **El doble buffer NO acelera el sistema.** Elimina **artifacts** de sincronización, en particular el screen tearing.
4. **No confundir bits por canal con bits totales.** 8 bits **por cañón** con 3 cañones dan $2^{24} \approx 16,7$ millones, **no** $2^8 = 256$.
5. **El parpadeo es problema del VECTORIAL; el aliasing es problema del RASTER.** No al revés.
6. **En el raster el tiempo de refresco NO depende de la complejidad de la imagen.** Es al revés en el vectorial.
7. **El LCD NO es emisor de luz: la modifica.** Los emisores son el LED y el OLED.
8. **"Matriz activa (TFT)" no es lo mismo que "emisor".** Son dos ejes independientes: uno describe cómo se sostiene el estado del píxel, el otro si el píxel produce luz.
9. **La fotografía digital es una imagen CAPTURADA, no sintetizada.** Lo que define la categoría es **de dónde viene la información**, no si el soporte es digital.
10. **"Realista" y "tiempo real" son ejes independientes.** Un render de cine es realista y **en diferido**.

Sobre algoritmos de digitalización

11. **Con la F de la recta, $F > 0$ significa DEBAJO de la línea**, no arriba. Es contraintuitivo y aparece como distractor.
12. **En Bresenham, $D < 0 \rightarrow$ adyacente (E); $D \geq 0 \rightarrow$ adyacente superior (NE).** Si el punto medio quedó **por encima** de la recta, la recta pasa más abajo y **no** se sube.
13. **El factor es $\times 2$ en la recta y $\times 4$ en circunferencia y elipse.** Sale del denominador que aparece: $\frac{1}{2}$ en un caso, $\frac{5}{4}$ y $\frac{1}{4}$ en los otros.
14. **En la recta los incrementos son CONSTANTES; en circunferencia y elipse se RECALCULAN** en cada paso. Es la diferencia más preguntable entre los tres algoritmos.
15. **DDA y Bresenham producen los MISMOS píxeles.** Lo que cambia es cómo se llega: flotante con error acumulado vs enteros exactos.
16. **La circunferencia tiene simetría de OCTANTES (8 puntos); la elipse sólo de CUADRANTES (4 puntos).** Por eso la elipse recorre un cuadrante entero dividido en dos regiones.
17. **La elipse se divide en dos regiones por el cambio de PENDIENTE, no porque tenga dos focos ni porque los semiejes sean distintos.**
18. **El punto medio de la región 1 de la elipse es $(x_k + 1, y_k - \frac{1}{2})$ y el de la región 2 es $(x_k + \frac{1}{2}, y_k - 1)$.** No son intercambiables: dependen de la dirección de avance.

19. **El algoritmo de punto medio para circunferencias NO dibuja toda la circunferencia: calcula sólo el 2.º octante.**

Sobre relleno y antialiasing

20. **No confundir boundary fill con flood fill.** Boundary se detiene en el **color de contorno**; flood reemplaza un color interior y se detiene ante **cualquier** otro color.
21. **En el scanline se PINTA ANTES de eliminar** los lados terminados. Invertir el orden deja spans sin rellenar.
22. **En la tabla de lados se guarda $1/m$, no m .** Y al actualizar se **suma $1/m$** , porque se avanza en **y**.
23. **Un vértice de "continuación" cuenta como 1 intersección y se acorta el lado inferior; un "extremo local" cuenta como 2 y no se toca.** Confundirlos invierte el relleno.
24. **Los lados horizontales se IGNORAN**, no se procesan de otro modo.
25. **Par-impar y nonzero winding coinciden en polígonos simples.** Difieren **sólo** en los auto-intersectados.
26. **Nyquist es el doble de la frecuencia máxima de la SEÑAL**, no el doble de la frecuencia de muestreo.
27. **Al pasar de frecuencia a intervalo la desigualdad se invierte:** $f_s \geq 2f_{max}$ pero $\Delta x_s \leq \Delta x_{ciclo}/2$.
28. **Aumentar la resolución NO elimina el aliasing.** Lo atenúa, con costo de memoria y fps.
29. **Supersampling = POSTfiltering (subdivide y cuenta); area sampling = PREfiltering (calcula el área directamente).** Los nombres suelen aparecer invertidos como distractor.
30. **En una grilla $n \times n$ el total de subpíxeles es n^2 , no n .** El " n niveles" de la filmina corresponde a una línea de **ancho cero**.
31. **La máscara de pesos hay que NORMALIZARLA** por la suma (16 en la de 3×3). El peso central es $4/16$, no 4 ni $4/9$.

Sobre modelado

32. **Poligonal y parches modelan SUPERFICIE; CSG y voxels modelan VOLUMEN.**
33. **CSG es EXACTA (dentro de sus primitivas); las otras tres son aproximadas.**
34. **CSG SÍ conserva la historia de modelado — es una ventaja, no una desventaja.** Su desventaja es que las operaciones son **globales**.
35. **El octree divide en 8, no en 4.** Por la mitad en las **tres** direcciones: $2^3 = 8$.
36. **En voxels, "todos los objetos tienen la misma complejidad" es una VENTAJA.** La visualización costosa y la memoria son las desventajas.
37. **Las superficies de revolución NO son un quinto método: son un caso particular del modelado matemático por barrido.**
38. **No confundir descomposición, organización y composición.** Descomponer = reducir a primitivas; organizar = jerarquía; componer = transformar y ensamblar.

Sobre transformaciones

39. **$A \cdot B \cdot C$ aplica PRIMERO C — la matriz más a la derecha.** Se lee de derecha a izquierda.
40. **$T \cdot R \neq R \cdot T$.** $T \cdot R$ gira sobre sí misma; $R \cdot T$ orbita alrededor del origen. **Ninguna es incorrecta.**

41. **La composición es asociativa PERO NO conmutativa.** Los distractores suelen negar la asociatividad o afirmar la conmutatividad.
42. **El escalado NO es una transformación de cuerpo rígido.** Traslación y rotación sí lo son.
43. **El escalado TAMBIÉN desplaza el objeto** si no está centrado en el origen, porque multiplica todas las coordenadas.
44. $S^{-1} = S(1/s_x, 1/s_y)$, **no** $S(-s_x, -s_y)$.
45. **La inversa de una rotación es su TRANSPUESTA:** $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$.
46. **En R_y el $-\sin \theta$ va ABAJO A LA IZQUIERDA.** Es la excepción entre las tres rotaciones básicas.
47. **En 3D no se rota "sobre un punto": se rota sobre un EJE.** (Nota de clase, Clase 9 p.15.)
48. **Dextrógiro para modelar el mundo; levógiro para la pantalla y la profundidad.** No al revés.
49. **Errata confirmada:** la matriz de escalado 3D con punto fijo de Unidad VI p.24 tiene un **1** donde debe ir un **0** (fila 2, columna 3).
50. **Si la última fila NO es (0001), la transformación no es afín: es una proyección en perspectiva.**

Sobre visualización 3D

51. **Culling DESCARTA lo que está completamente afuera; clipping RECORTA lo que intersecta.** No son sinónimos.
52. **Los puntos de fuga se cuentan por los ejes que INTERSECTAN el plano,** no por los que son paralelos a él.
53. **Cavalier: $L_1 = 1$, profundidad sin cambio. Cabinet: $L_1 = 0,5$, profundidad a la mitad.** Cabinet = mueble = achatado.
54. α **decide cuánto se acorta la profundidad; ϕ decide hacia dónde se dibuja.** No son intercambiables.
55. **La distancia focal es OPCIONAL** en la cámara sintética: la propia filmina lo dice.
56. **En proyección paralela NO hacen falta los ángulos de visión.**
57. **El volumen en perspectiva es una PIRÁMIDE (trunca al cerrarla); en paralela es un PRISMA.**
58. **La matriz de vista es la INVERSA de la pose de la cámara.** "La cámara no se mueve: se mueve la escena."
59. **En M^T los vectores u, v, w van como FILAS.** Como columnas se obtiene M , que rota en el sentido contrario.
60. **u SÍ se normaliza; v NO hace falta.** Porque U_p no es necesariamente perpendicular a w , mientras que $w \times u$ ya es unitario.
61. **w lleva SIGNO MENOS** respecto del Look at (en la convención de la Unidad VII-2), porque la cámara canónica mira hacia $-z$.
62. **La matriz D se aplica SÓLO en perspectiva.** En paralela el volumen ya es un prisma.
63. **`glm::perspective` NO es sólo D : es $D \cdot (S_2)_{xyz} \cdot S_{xy}$.**
64. **En el recorte paramétrico hay que verificar $0 \leq t \leq 1$.** Si no, la intersección existe pero fuera del segmento.
65. **El mapeo a pantalla usa $(W - 1)$ y $(H - 1)$,** no W y H : los índices de píxel empiezan en 0.
66. **La perspectiva NO es lineal en z .** Por eso requiere coordenadas homogéneas y división por w .

67. **La proyección ocurre ANTES de la rasterización** en el pipeline, no después.

Sobre el pipeline

58. **Tres etapas programables, cuatro fijas.** El **rasterizado es FIJO**: "acá vive Bresenham".

59. **El pipeline NO es un ciclo**: es una secuencia unidireccional.

70. **El recorte pertenece a la etapa de visualización**, no al modelado.

REPASO FINAL — LO QUE NO PUEDO IR SIN SABER

Lectura estimada: 20-30 minutos.

Definiciones imprescindibles

Concepto	Definición
Computación gráfica	La ciencia (y el arte) de comunicar visualmente por medio de una pantalla y los dispositivos de interacción de una computadora
Rendering	Proceso que permite obtener una representación estática 2D (imagen) de un mundo abstracto 3D
Pipeline de rendering	Conjunto de procesos que permiten visualizar un modelo 3D abstracto por medio de una representación 2D; múltiples pasos en secuencia donde la salida de una etapa es la entrada de la siguiente
Buffer de refresco	Memoria que almacena la imagen en formato de bits; desacopla generación y pintado; es releída en cada refresco
Screen tearing	Artifact por desincronización entre lectura y escritura del frame buffer: se muestran partes de dos cuadros a la vez
Conversión por barrido	Proceso de determinar qué píxeles deben pintarse para representar una primitiva continua en un sistema raster
Aliasing	Pérdida de información causada por submuestreo. Sus errores se llaman artifacts
Coherencia	Relación o conexión entre partes de la escena que permite ahorrar cálculo
Modelo	Representación real o abstracta que captura las características sobresalientes (dato y comportamiento) del objeto o fenómeno modelado
Transformación geométrica	Operación aplicada a la descripción geométrica de un objeto para cambiar su posición, orientación o tamaño. También llamada de modelado
Proyección geométrica	Mapeo de puntos de un espacio 3D a un plano 2D mediante líneas de proyección que convergen en un centro de proyección o son paralelas entre sí
Punto de fuga	Punto donde convergen las proyecciones de líneas paralelas que no son paralelas al plano de proyección
Cámara sintética	Mecanismo para transformar un mundo 3D en una imagen 2D proyectando puntos sobre un plano; se basa en el comportamiento físico de la luz
Transformación de normalización	Composición de transformaciones necesaria para pasar de una visualización arbitraria a una visualización canónica
Culling / Clipping	Culling descarta lo completamente fuera del volumen; clipping recorta lo que lo intersecta

Comparaciones imprescindibles

1. **Vectorial vs raster** — qué dibuja, cómo almacena, dependencia de la complejidad, costo del relleno, problema típico.
2. **DDA vs Bresenham** — aritmética, error, aptitud para hardware; **dan los mismos píxeles**.
3. **Recta vs circunferencia vs elipse** — factor ($\times 2/\times 4/\times 4$), incrementos (constantes/recalculados), simetría ($-$ /octantes/cuadrantes).
4. **Par-impar vs nonzero winding** — difieren sólo en auto-intersectados.
5. **Boundary fill vs flood fill** — la **condición de detención**.
6. **Supersampling vs area sampling** — post vs prefiltering; conteo vs cálculo analítico.
7. **Los cuatro métodos de representación 3D** — superficie/volumen, exacto/aproximado, ventajas y desventajas.
8. **Paralela vs perspectiva** — centro en el infinito o finito, proporciones vs realismo.
9. **Cavalier vs cabinet** — $L_1 = 1$ vs 0,5.
10. **Ortogonal vs oblicua** — ángulo de los rayos con el plano.
11. **Dextrógiro vs levógiro** — modelar el mundo vs representar la profundidad.
12. **LCD vs LED/OLED** — modula vs emite; **matriz pasiva vs activa** es otro eje.

Procesos que tengo que poder explicar

1. **El pipeline de rendering completo**, etapa por etapa, con qué recibe, qué hace y qué produce.
2. **La arquitectura del sistema raster** y por qué se separó en tres componentes.
3. **El planteo de un algoritmo de punto medio**: forma implícita $\rightarrow$ punto medio entre candidatos $\rightarrow$ multiplicar para eliminar la fracción $\rightarrow$ incrementos $\rightarrow$ simetría.
4. **El algoritmo de scanline optimizado**, con sus 6 pasos **en orden**.
5. **Boundary fill y flood fill**, con sus condiciones y por qué cada una.
6. **El proceso completo 3D $\rightarrow$ 2D**: normalización (4 pasos), recorte, proyección, viewport.
7. **La construcción de la base** (u, v, w) a partir de Look at y Up, con la justificación de cada decisión.
8. **La rotación sobre un eje arbitrario**: los 5 pasos y las 7 matrices.
9. **La construcción de un octree**: los 5 pasos.
10. **El modelado por barrido** y el caso particular de revolución.

Diagramas que tengo que poder reconstruir

1. **El pipeline de rendering abstracto** (modelado 3D $\rightarrow$ proyección $\rightarrow$ modelo de imagen $\rightarrow$ muestreo $\rightarrow$ frame buffer $\rightarrow$ display), con las unidades asociadas. $\leftarrow$ **Lo pidió el parcial anterior**.
2. **La arquitectura raster**: controlador de display $\rightarrow$ frame buffer $\rightarrow$ controlador de video $\rightarrow$ pantalla.
3. **El árbol de clasificación de las proyecciones planares**.
4. **El volumen de visualización** con posición, Look at, Up, θ_h , planos frontal y trasero.
5. **El volumen canónico** con sus límites $-1 \leq x, y \leq 1, 0 \leq z \leq -1$.
6. **La cadena de transformaciones** $q' = D \cdot (S_2) \cdot S_{xy} \cdot M^T \cdot T(-P_c) \cdot q$.
7. **El pipeline programable**: tres etapas programables y cuatro fijas.

Matrices que tengo que reconocer

TRASLACIÓN

$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$

↑ diagonal de 1,
valores en la
última columna

ROTACIÓN

$\begin{bmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix}$

↑ senos y cosenos,
antisimétrico

ESCALADO

$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$

↑ diagonal $\neq 1$,
última columna 0

INVERSAS: $T^{-1} = T(-t)$ $R^{-1} = R(-\theta) = R^T$ $S^{-1} = S(1/s)$

3D: $R_z \rightarrow$ -sin arriba a la derecha (como en 2D)

$R_x \rightarrow$ igual patrón, en el bloque y-z

$R_y \rightarrow$ ¡EXCEPCIÓN! el -sin va ABAJO A LA IZQUIERDA

REFLEXIÓN XY: $\text{diag}(1, 1, -1, 1)$

SI LA ÚLTIMA FILA NO ES $(0 \ 0 \ 0 \ 1)$ $\rightarrow$ PROYECCIÓN EN PERSPECTIVA

Fórmulas importantes

Frame buffer:	$niveles = 2^n$ (3 cañones $\times$ 8 bits $\rightarrow 2^{24} \approx 16,7$ M)
Recta:	$F(x,y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b$ $F > 0$ DEBAJO
DDA:	$ m \leq 1 \rightarrow y(i+1) = y(i) + m$; $ m > 1 \rightarrow x(i+1) = x(i) + 1/m$
Bresenham:	$D_0 = 2\Delta y - \Delta x$ $D < 0 \rightarrow E, D += 2\Delta y$ $D \geq 0 \rightarrow NE, D += 2(\Delta y - \Delta x)$ [CONSTANTES]
Circunferencia:	$F = x^2 + y^2 - r^2$; $D_0 = 5 - 4r$ $\Delta D_{ady} = 8(xk+1) + 4$ [inicial 12] $\Delta D_{inf} = 8(xk+1) - 8yk + 12$ [inicial $2\theta - 8r$] [SE RECALCULAN]
Elipse:	$F = ry^2x^2 + rx^2y^2 - rx^2ry^2$ cambio de región: $2ry^2x \geq 2rx^2y$ (es $dy/dx = -1$) R1: avanza en x, punto medio $(xk+1, yk-\frac{1}{2})$ R2: disminuye en y, punto medio $(xk+\frac{1}{2}, yk-1)$
Scanline:	$xk = (\Delta x / \Delta y) \cdot yk - (\Delta x / \Delta y) \cdot b$ $x(k+1) = xk + 1/m$ GET/AET: $(ymax, x(ymin), 1/m)$
Nyquist:	$fs \geq 2 \cdot f_{max} \Leftrightarrow \Delta xs \leq \Delta x_{ciclo} / 2$
Cobertura:	$\% = m \cdot xk + b - yk + \frac{1}{2}$
Mezcla:	$color = (n_{línea} \cdot c_{línea} + n_{fondo} \cdot c_{fondo}) / n_{total}$
Máscara 3x3:	$\{1, 2, 1; 2, 4, 2; 1, 2, 1\}$, $\Sigma = 16 \rightarrow w_{central} = 1/4$
Composición:	$P' = M_n \dots M_2 \cdot M_1 \cdot P$ (se lee de DERECHA a IZQUIERDA)
Punto arbitrario:	$M = T(p) \cdot X \cdot T(-p)$
Rotación:	$[\cos -\sin \quad xr(1-\cos)+yr \cdot \sin ; \sin \cos \quad yr(1-\cos)-xr \cdot \sin ; 0 \ 0 \ 1]$
Escalado:	$[sx \ 0 \ xf(1-sx) ; 0 \ sy \ yf(1-sy) ; 0 \ 0 \ 1]$
Eje arbitrario:	$R = T^{-1} \cdot Rx^{-1}(\alpha) \cdot Ry^{-1}(\theta) \cdot Rz(\vartheta) \cdot Ry(\theta) \cdot Rx(\alpha) \cdot T$ [7 matrices]
Oblicua:	$xp = x + L1(zvp-z)\cos \varphi$; $yp = y + L1(zvp-z)\sin \varphi$ $L1 = \cot \alpha$ cavalier: 1 cabinet: 0,5
AR:	$AR = width/height$
Plano proyección:	$zprp - zvp = (height/2) \cdot \cot(\vartheta h/2)$
Base cámara:	$w = -Look / \ Look\ $ (con signo menos) $u = (Up \times w) / \ Up \times w\ $ (SE normaliza) $v = w \times u$ (NO hace falta)
Matriz de vista:	$V = M^T \cdot T(-Pc)$ (u,v,w como FILAS)
Normalización:	$q' = D \cdot (S2)xyz \cdot Sxy \cdot M^T \cdot T(-Pc) \cdot q$ $Sxy = \text{diag}(\cot(\vartheta w/2), \cot(\vartheta h/2), 1, 1)$ $(S2) = \text{diag}(1/far, 1/far, 1/far, 1)$ $k = near/far$
Recorte:	$x = (1-t)x_0 + t \cdot x_1$; $t = (L-x_0)/(x_1-x_0)$; $0 \leq t \leq 1$
Pantalla:	$x' = (W-1)(x+1)/2$; $y' = (H-1)(y+1)/2$

Algoritmos importantes

Algoritmo	Para qué sirve	Clave conceptual
DDA	Digitalizar rectas	Incremental en flotante; dos estrategias según la pendiente
Bresenham	Digitalizar rectas	Sólo enteros ; evalúa el signo en el punto medio
Punto medio circunferencia	Digitalizar circunferencias	Factor $\times 4$; incrementos recalculados ; 2.º octante + 8 simetrías
Punto medio elipse	Digitalizar elipses	Dos regiones según $dy/dx = -1$; simetría de cuadrantes
Relleno por paridad	Rellenar polígonos	Bit de paridad; invertir en cada borde; pintar si impar
Scanline con GET/AET	Rellenar polígonos en general	Coherencia de bordes; 6 pasos en orden; casos especiales de vértices
Boundary fill	Rellenar regiones irregulares	Se detiene en el color de contorno ; doble condición
Flood fill	Rellenar regiones irregulares	Reemplaza un color interior ; sirve con bordes multicolor
Supersampling	Antialiasing	Postfiltering: subdivide en $n \times n$ y cuenta
Area sampling	Antialiasing	Prefiltering: calcula el área analíticamente
Octree	Subdividir el espacio	División en 8; sólo se refina donde hay objeto

Errores de multiple choice (los 12 más peligrosos)

1. $F > 0$ es **debajo** de la recta, no arriba.
2. $A \cdot B \cdot C$ aplica **primero C**.
3. La composición es asociativa pero **NO** conmutativa.
4. El escalado **no** es de cuerpo rígido y **además desplaza** si el objeto no está en el origen.
5. En la recta los incrementos son constantes; en circunferencia y elipse **se recalculan**.
6. La elipse tiene simetría de **cuadrantes**, no de octantes.
7. Boundary fill se detiene en el **color de contorno**; flood fill reemplaza un **color interior**.
8. Nyquist: el doble de la frecuencia de la **señal**.
9. **Culling** descarta; **clipping** recorta.
10. La matriz de vista es la **inversa** de la pose de la cámara.
11. u, v, w van como **filas** en M^T .
12. La matriz D se aplica **sólo en perspectiva**.

15 preguntas que definitivamente debería poder responder

1. ¿Cuál es la diferencia entre un display vectorial y uno raster, y qué consecuencias tiene?
2. ¿Qué es el buffer de refresco y qué función cumple en la arquitectura del dispositivo raster?
3. ¿Qué es el pipeline de rendering? Dibujarlo e indicar la operación de cada etapa.

4. *¿Cómo funcionan el relleno de contorno y el relleno por difusión, y en qué se diferencian?*
 5. *Dada una matriz en coordenadas homogéneas, ¿qué transformación representa y qué efecto produce?*
 6. *Ante $D = A \cdot B \cdot C$, ¿en qué orden se aplican las transformaciones y por qué importa?*
 7. *¿Cuáles son las especificaciones mínimas de una cámara sintética para perspectiva?*
 8. *¿Cómo se convierte una visualización 3D arbitraria en una imagen 2D?*
 9. *¿Por qué se utilizan coordenadas homogéneas?*
 10. *¿Cómo se plantea un algoritmo de punto medio, y por qué se multiplica por 2 o por 4?*
 11. *¿Por qué el algoritmo para elipses divide el cuadrante en dos regiones?*
 12. *¿Cuáles son los cuatro métodos de representación de objetos 3D y en qué se diferencian?*
 13. *¿Qué es el aliasing, por qué se produce y qué técnicas lo mitigan?*
 14. *¿Cómo se construye la base (u, v, w) de la cámara y por qué la rotación se usa transpuesta?*
 15. *¿Qué es la visualización canónica y por qué se la utiliza?*
-

SIMULACRO DE PARCIAL

Computación Gráfica y Ambientes Virtuales (0494) — Duración sugerida: 2 horas

Parte A: 22 preguntas de multiple choice (1 punto cada una) — Parte B: 6 preguntas de desarrollo (13 puntos cada una) — **Total: 100 puntos**

Las preguntas son **nuevas**: no reproducen las del parcial anterior, pero mantienen su profundidad, estilo y tipo de razonamiento.

Parte A — Multiple Choice

A1. El proceso determinístico e invariante del barrido raster implica que el tiempo de repintado:

A) Crece linealmente con la cantidad de primitivas dibujadas B) Es constante, independientemente de la complejidad de la imagen y del código de pintado C) Depende de la persistencia del fósforo únicamente D) Se reduce a la mitad al usar modo entrelazado

A2. La incorporación del buffer de refresco en la arquitectura raster permitió principalmente:

A) Aumentar la resolución máxima del dispositivo B) Que la generación y el pintado de la imagen funcionen a velocidades diferentes C) Eliminar el aliasing de las primitivas D) Reducir la cantidad de cañones de electrones necesarios

A3. Un sistema utiliza un frame buffer de 1 cañón de 6 bits. La cantidad de tonos que puede representar es:

A) 6 B) 36 C) 64 D) 128

A4. En el pipeline de rendering, la etapa que decide qué píxeles se encienden para representar una primitiva corresponde a:

A) Modelado del mundo virtual B) Proyección C) Muestreo / procesamiento D) Frame buffer

A5. Con la forma implícita de la recta utilizada en la materia, un punto medio que arroja $D \geq 0$ indica que:

A) El punto medio está por encima de la recta y corresponde pintar el adyacente B) El punto medio está por debajo de la recta y corresponde pintar el adyacente superior C) El punto pertenece exactamente a la recta D) La pendiente es mayor que 1 y hay que cambiar de estrategia

A6. Un segmento va de (1,1) a (7,9). Antes de aplicar Bresenham en su versión básica corresponde:

A) Aplicarlo directamente, ya que $\$x_0$

A7. La diferencia entre los incrementos de la variable de decisión en el algoritmo de rectas y en el de circunferencias es que en circunferencias:

A) Son siempre negativos B) Dependen de la posición actual y deben recalcularse en cada paso C) Se multiplican por 2 en lugar de por 4 D) Sólo se calculan una vez cada ocho píxeles

A8. En el algoritmo de punto medio para elipses, el criterio $2r_y^2x \geq 2r_x^2y$ indica que:

A) El punto está fuera de la elipse B) Debe pasarse a la región 2, donde se avanza disminuyendo en y C) Debe aplicarse la simetría de octantes D) La elipse no está en posición estándar

A9. Una región de un polígono auto-intersectado que queda "rodeada dos veces" será considerada:

A) Interior por par-impar y exterior por winding B) Exterior por par-impar e interior por winding C) Interior por ambas reglas D) Exterior por ambas reglas

A10. En el algoritmo de scanline optimizado, eliminar de la AET los lados con $y_{max} = y_k$ debe hacerse:

A) Antes de pintar los spans de esa scanline B) Después de pintar los spans de esa scanline C) Antes de ordenar la AET por x D) Sólo al finalizar todo el recorrido del polígono

A11. Si en el algoritmo `boundaryFill4` se omitiera la comparación contra el color de relleno, la consecuencia sería:

A) El relleno se escaparía de la figura B) Recursión infinita al volver sobre píxeles ya pintados C) No se rellenaría ningún píxel D) Sólo se rellenaría la primera fila horizontal

A12. Una figura tiene el borde compuesto por varios colores distintos y un interior de color uniforme. El algoritmo adecuado es:

A) Boundary fill, porque tolera contornos irregulares B) Flood fill, porque reemplaza un color interior y se detiene ante cualquier otro C) Relleno por paridad, procesando cada color por separado D) Scanline con GET y AET

A13. Un píxel es atravesado por una línea que cubre 5 de sus subpíxeles en una grilla de supersampling de 4×4 . La intensidad correspondiente es:

A) $5/4 = 125\%$ B) $5/16 \approx 31\%$ C) $4/5 = 80\%$ D) 5 niveles de intensidad

A14. ¿Cuál de estos métodos de representación permite obtener propiedades de masa como volumen, área superficial y momentos de inercia?

A) Representación poligonal B) Parches paramétricos bicúbicos C) Subdivisión del espacio por voxels D) Malla de alambre (wireframe)

A15. En un octree, una celda que **no contiene** ninguna parte del objeto:

A) Se subdivide igual, para mantener la uniformidad del árbol B) No se vuelve a subdividir C) Se marca como superficie y se interpola D) Se convierte en un voxel de densidad cero

A16. Una matriz 4×4 cuya última fila es $(0 \ 0 \ -1 \ 0)$ corresponde a:

A) Un escalado no uniforme B) Una transformación que no es afín: genera un w dependiente de z , propia de la proyección en perspectiva C) Una reflexión respecto del plano XY D) Una rotación alrededor del eje z

A17. Un objeto ubicado lejos del origen es transformado por $M = \text{diag}(3, 3, 1)$. El resultado es:

A) El objeto triplica su tamaño y permanece en el mismo lugar B) El objeto triplica su tamaño y además se aleja del origen C) El objeto rota y se amplía D) El objeto se traslada tres unidades en cada eje

A18. Se desea que un objeto gire sobre sí mismo sin desplazarse de su ubicación actual. La composición correcta es:

A) $M = R \cdot T$ B) $M = T \cdot R$ C) $M = S \cdot R$ D) $M = R \cdot S$

A19. El sistema de coordenadas levógiro se emplea típicamente para:

A) Describir la geometría de los objetos en el mundo B) Representar la escena en pantalla y describir la profundidad C) Definir la base ortonormal de la cámara D) Almacenar la tabla de colores del frame buffer

A20. El motivo por el cual la transformación de normalización aplica **traslación y rotación inversas** es:

A) Que el volumen canónico está definido con z negativo B) Que la transformación se aplica a los vértices de la escena y no a la cámara: la cámara no se mueve, se mueve la escena C) Compensar la posterior división por w D) Convertir el sistema de dextrógiro a levógiro

A21. Al construir la base de la cámara, el vector u debe normalizarse porque:

A) El producto cruz nunca produce vectores unitarios B) El vector U_p no es necesariamente perpendicular a w , por lo que $\|U_p \times w\| \neq 1$ C) u es el único vector que interviene en la matriz de vista D) La normalización corrige el signo del producto cruz

A22. Al mapear el rango normalizado a una pantalla de 800×600 , la coordenada $x = 1$ se transforma en:

A) 800 B) 799 C) 400 D) 1

Parte B — Desarrollo

B1. (13 pts) Explique qué es el aliasing, cuál es su causa en un sistema raster y por qué no puede eliminarse simplemente aumentando la resolución del dispositivo. Describa además las técnicas de antialiasing disponibles y los criterios para elegir entre ellas.

B2. (13 pts) Describa el algoritmo de relleno de polígonos por línea de scan en su versión optimizada. Indique qué estructuras de datos utiliza, qué información almacena cada una, el orden de los pasos del algoritmo y cómo se resuelven los casos especiales.

B3. (13 pts) Explique el planteo general de los algoritmos de punto medio para digitalizar primitivas. Indique los pasos comunes a los tres algoritmos estudiados y señale las diferencias entre el caso de la recta, el de la circunferencia y el de la elipse.

B4. (13 pts) Realice una comparación de los métodos de representación de objetos tridimensionales estudiados, indicando para cada uno qué aspecto del objeto modela, si la representación es exacta o aproximada, y sus principales ventajas y desventajas.

B5. (13 pts) Explique por qué se utilizan coordenadas homogéneas en computación gráfica. Describa cómo se compone una transformación respecto de un punto que no es el origen y justifique el orden de las matrices en el producto resultante.

B6. (13 pts) Describa el concepto de proyección geométrica y desarrolle la clasificación de las proyecciones planares. Explique la diferencia esencial entre las proyecciones paralelas y las proyecciones en perspectiva, indicando en qué aplicaciones conviene cada una.

RESPUESTAS DEL SIMULACRO

Parte A — Respuestas y explicación

A1. B. El barrido raster recorre la pantalla con un patrón regular; el proceso es determinístico e invariante: no depende de la complejidad de la imagen ni del código de pintado. La opción A describe el comportamiento del **vectorial**. El modo entrelazado (D) mejora la velocidad, pero no es la característica que define la invariancia.

A2. B. El problema original era que un único controlador obligaba a **sincronizar** generación y pintado. El buffer, junto con la separación en controlador de display y de video, **desacopla ambas velocidades**.

A3. C. 2^n con $n = 6$ da $2^6 = 64$. La opción A confunde n con el resultado; la B calcula 6^2 .

A4. C. La etapa de **muestreo/procesamiento** realiza la digitalización o conversión por barrido: decide qué píxeles se encienden. El frame buffer (D) **almacena** el resultado, no lo calcula.

A5. B. Con esta F , positivo significa **debajo** de la recta; si el punto medio quedó debajo, la recta pasa más arriba y corresponde el **adyacente superior (NE)**. La opción A invierte ambas cosas.

A6. B. $\Delta x = 6$, $\Delta y = 8$, entonces $m = 8/6 \approx 1,33 > 1$. La versión básica exige **soy** y calculando **x**, igual que en el DDA. La opción A verifica sólo dos de las tres hipótesis.

A7. B. En la recta los incrementos son **constantes**; en circunferencias dependen de x_k e y_k y **se recalculan**. Es la diferencia central entre ambos algoritmos. La opción C invierte los factores: la circunferencia usa $\times 4$.

A8. B. El criterio surge de igualar $dy/dx = -1$; a partir de ahí conviene avanzar **disminuyendo en y** para no dejar huecos.

A9. B. Dos cruces es un número **par** $\rightarrow$ exterior para par-impar. Pero el winding number vale ± 2 , **distinto de cero** $\rightarrow$ interior para winding. Es el caso donde ambas reglas difieren.

A10. B. El orden de la filmina es: mover $\rightarrow$ ordenar $\rightarrow$ **pintar** $\rightarrow$ eliminar $\rightarrow$ avanzar $\rightarrow$ actualizar. Eliminar antes de pintar dejaría la última scanline de cada lado sin rellenar.

A11. B. Sin esa comparación, al volver sobre un píxel ya pintado la condición seguiría cumpliéndose y la recursión no terminaría. La opción A describiría un problema de conectividad, no de esta condición.

A12. B. El flood fill reemplaza un **color interior** y se detiene ante cualquier otro, por lo que tolera bordes multicolor. El boundary fill (A) requiere un contorno de **un solo color definido**.

A13. B. El total de subpíxeles en una grilla 4×4 es $n^2 = 16$, no $n = 4$. La opción A divide por n y da un valor imposible, mayor al 100 %.

A14. B. Es la ventaja específica que la filmina atribuye a los **parches paramétricos bicúbicos**.

A15. B. El octree **sólo refina donde hay objeto**: las celdas vacías no se vuelven a subdividir. Ésa es precisamente la ventaja de la estructura frente a una grilla uniforme.

A16. B. La última fila (0 0 0 1) caracteriza a las transformaciones **afines**. Si no lo es, se genera una coordenada homogénea w dependiente de la posición, que al homogeneizar produce la división por la profundidad: es la firma de la **proyección en perspectiva**.

A17. B. El escalado multiplica **todas** las coordenadas, incluidas las de posición. Por eso existe la versión con **punto fijo**, que aplica el patrón "ir al origen, escalar, volver".

A18. B. $T \cdot R$ se lee de derecha a izquierda: **primero rota** (sobre el origen del objeto) y **después traslada**, de modo que la pieza gira sobre sí misma y queda en su lugar. $R \cdot T(A)$ la haría **orbitar** alrededor del origen.

A19. B. El dextrógiro se usa para modelar los objetos y el mundo; el **levógiro** para la representación en pantalla y describir la profundidad.

A20. B. Es el concepto central de la normalización: "se aplica a todos los vértices de los objetos para preservar las distancias entre cámara y escena". La cámara nunca se mueve.

A21. B. U_p no tiene por qué ser perpendicular a w , así que $U_p \times w$ no resulta unitario. En cambio $v = w \times u$ **sí** lo es, porque w y u son unitarios y perpendiculares entre sí.

A22. B. $x' = (W - 1)(x + 1)/2 = 799 \cdot 1 = 799$. Los índices de píxel van de 0 a 799 en una pantalla de 800.

Parte B — Respuestas modelo y puntos esenciales

B1 — Aliasing y antialiasing

Respuesta modelo.

El **aliasing** es la **pérdida de información causada por submuestreo** (undersampling). Los errores visuales que produce se denominan **artifacts**, y los más característicos son los **perfiles escalonados** (jagged), los **detalles faltantes** o renderizados incorrectamente, y las **texturas que se deforman**.

Su **causa** en un sistema raster es estructural: los algoritmos de digitalización de primitivas — Bresenham, punto medio— generan objetos con apariencia escalonada porque realizan un **muestreo discreto de modelos continuos**. El problema de fondo es que se pretende representar objetos con **parámetros continuos** utilizando **píxeles discretos**.

La teoría del muestreo establece el criterio: según el **teorema de Nyquist**, para no perder información hay que muestrear con una frecuencia de **al menos el doble de la frecuencia máxima de la señal**, es decir $f_s \geq 2f_{max}$, lo que equivale a un intervalo de muestreo $\Delta x_s \leq \Delta x_{ciclo}/2$.

De ahí surge la **limitación fundamental**: un borde nítido contiene **frecuencias arbitrariamente altas**, de modo que para representarlo exactamente harían falta **intervalos de muestreo arbitrariamente pequeños**, lo cual es imposible. Por eso **aumentar la resolución del dispositivo mejora el resultado pero no elimina el problema**. Además existen limitaciones prácticas: el tamaño máximo del frame buffer está limitado por la tecnología, mantener sesenta o más cuadros por segundo obliga a compromisos, y hay un costo computacional y de memoria asociado. En consecuencia, se requieren **técnicas de software** que actúen como solución complementaria: el antialiasing.

Existen **tres técnicas**. El **supersampling** o postfiltering muestrea a alta resolución y combina los resultados: divide cada píxel en una grilla de $n \times n$ subpíxeles, aplica el algoritmo de rasterización sobre ellos, cuenta cuántos solapan la primitiva y establece la intensidad proporcional al conteo. Si la línea tiene ancho finito, todos los subpíxeles pueden contribuir y puede mezclarse con el color de fondo mediante un promedio ponderado; también pueden emplearse **máscaras de peso** que otorguen mayor importancia a los subpíxeles centrales.

El **area sampling** o prefiltering calcula directamente el **área de solapamiento** entre la primitiva y el píxel, sin realizar cálculos subpíxel: trata la línea como un rectángulo de ancho finito, calcula la intersección polígono-píxel y determina el porcentaje de cobertura, estableciendo la intensidad en forma proporcional.

Las **técnicas de filtrado** utilizan funciones de peso continuas sobre la superficie del píxel e integran el producto de la función objeto por el filtro, lo que otorga mayor precisión matemática. Los filtros típicos son el **box** de peso uniforme, el **cono** de peso decreciente radial y el **gaussiano**.

Los **criterios de selección** entre ellas son la calidad visual obtenida, el costo computacional, la memoria requerida y la disponibilidad de la técnica en el hardware.

Puntos esenciales que deben aparecer: definición de aliasing como pérdida por submuestreo · artifacts concretos · causa (muestreo discreto de modelos continuos) · Nyquist en alguna de sus dos formas · la limitación fundamental (frecuencias arbitrariamente altas) · por qué el hardware no alcanza (frame buffer, fps, costo) · las tres técnicas con su nombre alternativo (post/prefiltering) y su mecanismo · los cuatro criterios de selección.

B2 — Scanline optimizado

Respuesta modelo.

El algoritmo de scanline generaliza el relleno a **todo tipo de polígonos** —cóncavos, convexos, auto-intersectados—, aprovechando el concepto de **coherencia**: la relación entre partes de la escena que permite ahorrar cálculo. En particular utiliza la **coherencia espacial**, según la cual las primitivas no cambian de píxel a píxel dentro de un intervalo ni de una línea de scan a otra, y la **coherencia de bordes**, según la cual un lado corta a scanlines consecutivas en puntos vecinos.

El planteo básico consiste en recorrer el polígono de abajo hacia arriba; para cada línea de scan determinar las intersecciones con los lados; ordenarlas en forma ascendente según x ; y recorrer la scanline de izquierda a derecha invirtiendo el estado en cada intersección y pintando cuando el estado es interior.

La versión optimizada introduce dos mejoras. La primera es el **cálculo incremental de las intersecciones**: en lugar de resolver la ecuación del lado en cada scanline, se calcula la intersección siguiente a partir de la actual mediante $x_{k+1} = x_k + 1/m = x_k + \Delta x / \Delta y$, ya que al avanzar una unidad en y la coordenada x se desplaza $1/m$.

La segunda es el uso de dos **estructuras de datos**. La **tabla de lados global (GET)** organiza los lados en **casilleros** indexados por la coordenada y_{min} de cada lado, empleando un ordenamiento de casilleros o bucket sort; dentro de cada casillero los lados se ordenan por su coordenada $x_{min} = f(y_{min})$. De cada lado se almacenan únicamente los **datos mínimos de reconstrucción**: su y_{max} , su x en y_{min} y su $1/m$. La **tabla de lados activos (AET)** contiene los datos de reconstrucción de **todos los lados que intersecta la scanline activa**, ordenados según el valor actual de la coordenada x ; cada entrada guarda y_{max} , la x actual y $1/m$.

El algoritmo procede entonces así: se **crea la GET**; se inicializa la scanline en el y_{min} correspondiente al primer casillero no vacío; se inicializa la AET vacía; y se repite hasta que ambas tablas queden vacías la siguiente secuencia: **mover** de la GET a la AET los lados cuyo y_{min} coincide con la scanline actual; **ordenar** la AET según x ; **pintar** los píxeles utilizando **pares** de coordenadas x de la AET; **eliminar** de la AET las entradas cuyo y_{max} coincide con la scanline actual, es decir los lados terminados; **avanzar** a la scanline siguiente; y **actualizar** la coordenada x de las entradas remanentes sumando $1/m$.

Respecto de los **casos especiales**, el primero se presenta cuando una scanline pasa por un **vértice compartido**, situación que puede generar un número **impar** de intersecciones y producir una determinación interior/exterior incorrecta. La solución es un **análisis topológico**: si el vértice constituye un **extremo local**, es decir si los dos lados quedan del mismo lado de la scanline formando un pico o un valle, se cuenta como **dos** intersecciones; si se trata de una **continuación**, con los lados en zonas opuestas, se cuenta como **una** sola, lo que se implementa **acortando el lado inferior en una unidad**.

El segundo caso especial son los **lados horizontales**, que generarían múltiples intersecciones con su propia scanline; la solución adoptada es simplemente **ignorarlos**, ya que quedan cubiertos por los lados adyacentes.

Como verificación, en cada scanline la cantidad de intersecciones debe resultar **par**; un número impar indica que falta aplicar el análisis topológico en algún vértice.

Puntos esenciales: el concepto de coherencia y sus tipos · el planteo básico de cuatro pasos · la optimización incremental con $1/m$ · GET y AET con **los tres campos** de cada una y sus criterios de ordenamiento · los **seis pasos en orden**, especialmente que se pinta **antes** de eliminar · los dos casos especiales con su solución · la verificación de paridad.

B3 — Planteo de los algoritmos de punto medio

Respuesta modelo.

(Ver la respuesta modelo desarrollada en la sección 2.6 El patrón común a los tres algoritmos.)

Puntos esenciales: los **cinco pasos** del patrón (forma implícita y sus tres regiones → evaluación en el punto medio entre los candidatos → multiplicación para eliminar la fracción → cálculo de los incrementos → aprovechamiento de la simetría) · que sólo interesa el **signo** de la variable de decisión y por eso puede multiplicarse · la tabla con las diferencias: forma implícita de cada primitiva, factor **×2** en la recta y **×4** en circunferencia y elipse, incrementos **constantes** en la recta pero **recalculados** en las otras dos, y simetría de **octantes** en la circunferencia frente a **cuadrantes** en la elipse · para la elipse, además, las **dos regiones** con su criterio de cambio y el hecho de que **el punto medio depende de la dirección de avance**.

B4 — Comparación de los métodos de representación 3D

Respuesta modelo.

(Ver la respuesta modelo desarrollada en la sección 5.5 Comparación de los cuatro métodos.)

Puntos esenciales: los **dos criterios de clasificación** (superficie vs volumen; exacta vs aproximada) · los cuatro métodos con su elemento básico, su estructura de datos y su aplicación típica · ventajas y desventajas de cada uno, cuidando no invertirlas (CSG **sí** conserva la historia de modelado; en voxels la complejidad uniforme **es** una ventaja) · al menos un dato concreto, como la tetera de Utah con 306 vértices frente a 2048 · cierre con la diferencia fundamental.

B5 — Coordenadas homogéneas y composición

Respuesta modelo.

Las transformaciones geométricas, expresadas en coordenadas cartesianas, adoptan la forma $P' = M_1 \cdot P + Q$. El inconveniente es que las **matrices operan con transformaciones proporcionales** mediante multiplicación, mientras que los **términos de traslación requieren una suma adicional**. Esto trae dos desventajas concretas: la **composición de transformaciones resulta compleja**, porque hay que ir alternando productos y sumas, y **no existe una forma unificada de representación** para las tres transformaciones básicas.

La solución son las **coordenadas homogéneas**. Un punto 2D (x, y) se representa mediante un vector de **tres** componentes (x_h, y_h, h) , donde $x = x_h/h$ e $y = y_h/h$; la elección conveniente es $h = 1$, que da una versión normalizada en la que las coordenadas homogéneas **coinciden numéricamente** con las cartesianas. En 3D el punto pasa a ser $(x, y, z, 1)$.

Las ventajas son tres: **todas** las transformaciones, incluida la traslación, pueden plantearse como una **multiplicación matricial**; las matrices resultan **uniformes**, de 3×3 en 2D y 4×4 en 3D; y se **simplifica el proceso de composición**, ya que componer se reduce a multiplicar.

Para componer una transformación **respecto de un punto que no es el origen** se aplica el patrón "trasladar al origen, transformar, volver", que consta de tres pasos: primero se **traslada el objeto** de modo que el punto de referencia quede en el origen; luego se **aplica la transformación** deseada, que las matrices básicas realizan siempre respecto del origen; y finalmente se **traslada el objeto de vuelta** a su posición original.

El **orden de las matrices en el producto** se justifica por la convención de composición: en $P' = M_n \cdots M_2 \cdot M_1 \cdot P$ el producto **se lee de derecha a izquierda**, de manera que **la matriz ubicada más a la derecha es la primera en aplicarse**. Como la traslación al origen es lo primero que debe ocurrir, debe escribirse **a la derecha** del producto. Así, para rotar alrededor de un punto (x_r, y_r) la composición correcta es $M = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$, y para escalar manteniendo fijo un punto (x_f, y_f) es $M = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f)$, que da como resultado una matriz con los factores de escala en la diagonal y los términos $x_f(1 - s_x)$ e $y_f(1 - s_y)$ en la última columna.

Este orden **no es arbitrario ni intercambiable**, porque la multiplicación de matrices es **asociativa pero no conmutativa**. Cada transformación actúa sobre el resultado de la anterior, es decir, sobre un objeto ya modificado; invertir el orden produce una transformación geoméricamente distinta. Si la traslación al origen se escribiera a la izquierda, la rotación se aplicaría **antes** de llevar el punto de referencia al origen, de modo que el objeto giraría alrededor del origen del sistema y no alrededor del punto deseado, y además quedaría desplazado.

La ventaja práctica de todo esto es que la matriz compuesta **se calcula una sola vez** y luego **se aplica a todos los vértices del objeto**, en lugar de realizar las transformaciones sucesivamente sobre cada uno.

Puntos esenciales: el problema de partida ($P' = M_1 P + Q$ y la traslación como suma) · las **dos** desventajas · la definición con h y la elección $h = 1$ · las **tres** ventajas · el patrón de tres pasos con la justificación de por qué hace falta · la regla de lectura de derecha a izquierda · **por qué la traslación al origen va a la derecha** · al menos una matriz compuesta concreta · la no conmutatividad y qué pasaría al invertir · la ventaja computacional.

B6 — Proyecciones geométricas

Respuesta modelo.

Una **proyección geométrica** es el mapeo de puntos de un espacio 3D a un plano 2D mediante **líneas de proyección** que **convergen en un centro de proyección** o bien **son paralelas entre sí**. Sus elementos fundamentales son el **objeto 3D** a proyectar, el **centro de proyección** desde el cual parten las líneas, el **plano de proyección** donde se forma la imagen, y las propias **líneas de proyección**, que conectan los puntos del objeto con el plano.

La clasificación de las proyecciones planares responde a dos criterios sucesivos. El **primer criterio** es la **distancia entre el centro de proyección y el plano de proyección**, y separa dos grandes familias. En las **proyecciones paralelas** el centro de proyección se encuentra **en el infinito** y las líneas de proyección resultan **paralelas entre sí**; se utilizan en ingeniería y arquitectura porque **mantienen las proporciones relativas**. En las **proyecciones en perspectiva** el centro se encuentra a **distancia finita** y las líneas **convergen**; imitan lo que ven nuestros ojos, como si fueran una cámara, produciendo imágenes de aspecto **más natural**.

El **segundo criterio** subdivide a las paralelas según el **ángulo que forman las líneas de proyección con el plano de proyección**. Las **ortogonales** tienen líneas **perpendiculares** al plano, y se dividen a su vez en proyecciones de **múltiples vistas**, cuando el plano es paralelo a alguno del sistema de referencia —preservan exactamente ángulos y longitudes, son el estándar del dibujo técnico, pero cada vista muestra sólo dos dimensiones y la forma 3D debe componerse mentalmente—, y en **axonométricas**, cuando el plano no es paralelo a los del sistema, lo que permite ver varias caras simultáneamente conservando las proporciones. Las axonométricas se clasifican a su vez en **isométrica**, con los tres ángulos entre ejes iguales a 120° y la misma escala en los tres ejes; **dimétrica**, con dos ángulos iguales y la misma escala en dos ejes; y **trimétrica**, con los tres ángulos y las tres escalas diferentes.

Las **oblicuas** tienen el plano paralelo a uno de los del sistema pero las líneas de proyección formando un ángulo distinto de 90° ; representan de forma exacta una cara del objeto, aunque la forma 3D resultante no es realista y los objetos pueden distorsionarse. Se definen mediante el ángulo α entre la línea de proyección y el plano, del que se obtiene $L_1 = \cot \alpha$, y el ángulo ϕ que da la dirección en el plano. Los dos tipos principales son la **caballera** o cavalier, con $\alpha = 45^\circ$ y $L_1 = 1$, en la que las dimensiones perpendiculares al plano **no cambian de magnitud**; y la **militar** o cabinet, con $\tan \alpha = 2$ y $L_1 = 0,5$, en la que esas dimensiones **se reducen a la mitad**, lo que da un aspecto más realista al reducir la distorsión en profundidad.

Las **perspectivas**, por su parte, se clasifican según la cantidad de **puntos de fuga**, entendiendo por tales los puntos donde convergen las proyecciones de líneas paralelas que **no son paralelas al plano de proyección**. Los puntos de fuga principales corresponden a los ejes X, Y y Z del objeto, y su cantidad —uno, dos o tres— está determinada por **cuántos ejes principales intersectan el plano de proyección**.

La **diferencia esencial** entre ambas familias es la siguiente. En la paralela, al estar el centro de proyección en el infinito, los rayos no convergen y la transformación resulta **lineal**: las proporciones se conservan a cualquier distancia, de modo que **sobre el dibujo se puede medir**. En la perspectiva, en cambio, el centro está a distancia finita y la proyección incorpora una **división por la profundidad**, concretamente por $(z_{prp} - z)$, lo que hace que la transformación **no sea lineal en z** y que los objetos lejanos se vean más pequeños, produciendo el efecto de **escorzo**. Esa no linealidad es, justamente, la razón por la cual la perspectiva requiere coordenadas homogéneas y una división por w para poder expresarse matricialmente.

En cuanto a las aplicaciones, las **paralelas** se emplean donde importa la **precisión y la medición**: dibujo técnico y planos en el caso de las ortogonales de múltiples vistas, ilustraciones técnicas en el de

las axonométricas, y diagramas y esquemas en el de las oblicuas. Las **perspectivas** se emplean donde importa el **realismo visual**: visualización, videojuegos y CAD. Los criterios de selección son, entonces, precisión frente a realismo, complejidad computacional —las paralelas son más eficientes— y el propósito, según se trate de documentación técnica o de presentación visual.

Puntos esenciales: definición completa contemplando **las dos** posibilidades de las líneas · los cuatro elementos · los **dos criterios** de clasificación · el árbol completo con las tres axonométricas y los dos tipos de oblicuas con sus valores de α y L_1 · los puntos de fuga con su definición precisa y qué determina su cantidad · la diferencia fundamental centrada en la **división por la profundidad** y la no linealidad en z · las aplicaciones y los tres criterios de selección.

CHECKLIST DE VERIFICACIÓN

- [✓] **Todos los PDFs revisados.** Los 28 archivos de `teorico-practico/` fueron extraídos y leídos íntegramente en el análisis previo, y revisados nuevamente para este documento. Las páginas cuyo contenido está sólo en imágenes (apuntes de Clase 3, 7 y 9) se verificaron renderizándolas.
- [✓] **Todas las unidades cubiertas** — con la salvedad indicada: **sólo cinco de las nueve unidades del programa tienen material** entre los PDFs entregados (I, II, V, VI y VII). Las Unidades III, IV, VIII y IX no fueron entregadas y **no se inventó su contenido**.
- [✓] **Preguntas del parcial anterior utilizadas como referencia.** Se usaron para calibrar el nivel de profundidad, se identificó que **ninguna es numérica** y se ajustó el énfasis del documento en consecuencia; además, se las respondió una por una con variantes.
- [✓] **Respuestas modelo de desarrollo incluidas** — en cada tema relevante, bajo el título "Cómo respondería esto en un parcial".
- [✓] **Multiple choice de práctica incluido** — 15 preguntas en la Unidad I, 15 en la II.1, 20 en la II.2, 15 en la V, 18 en la VI y 24 en la VII, todas con explicación de la respuesta correcta y de los distractores.
- [✓] **Matrices y transformaciones explicadas** — con una **tabla de reconocimiento** orientada al formato de la pregunta 5, indicando cómo identificar cada matriz y qué le ocurre a posición, orientación y tamaño.
- [✓] **Pipeline explicado** — con esquema, y para cada etapa qué recibe, qué operación realiza, qué produce y por qué es necesaria.
- [✓] **Comparaciones importantes incluidas** — doce comparaciones en formato tabla, cada una seguida de su Diferencia fundamental.
- [✓] **Notas de clase utilizadas** — dieciséis aportes marcados como **[NOTA DE CLASE]**, incluyendo el único "hay que saber explicar" del material.
- [✓] **Repaso final incluido** — lectura de 20 a 30 minutos.
- [✓] **Simulacro incluido** — 22 preguntas de multiple choice y 6 de desarrollo, con respuestas en sección separada.
- [✓] **PDF generado correctamente.**

Salvedades que corresponde declarar

1. **Cuatro unidades del programa no tienen material** entre los PDFs entregados. Conviene confirmar el alcance del parcial.
2. **Dos formulaciones del material están en conflicto o no verifican**, y se presentan ambas versiones sin elegir: la relación entre θ_w y θ_h (Unidad VII-1 p.27 frente a Guía 05 p.2), y la matriz D de deformación del frustum (Unidad VII-2 p.22).
3. **Una errata confirmada visualmente:** la matriz de escalado 3D con punto fijo de Unidad VI p.24.
4. **Una inconsistencia interna** en Unidad VII-2 sobre el signo de z en el volumen canónico.
5. **Un contenido faltante en los apuntes:** el título "Otra forma de calcular el supersampling" (Clase 7 p.1) quedó sin desarrollo.
6. **La correspondencia de los prácticos mencionados el año anterior** ("Práctico 2.2 de la guía 1" y "Práctico 4 de la guía 2") con las guías de 2026 es una **inferencia**, porque la numeración no coincide.